ODT — On-Device Training

Complete Reference for the STM32 C Library

FQT · RigL · HAR · LibriSpeech Speaker ID

Chapters 1–39 | All Source Files | RigL Gap Analysis | STM32 Hardware Notes

Introduction: FQT, RigL, and the HAR Dataset

1. Fixed-Point Quantization Training (FQT)

Fixed-Point Quantization Training (FQT), introduced by Deutel et al., enables neural networks to be trained directly using low-precision integer arithmetic instead of 32-bit floating point. The core insight is that while full-precision floating point is convenient for research, embedded microcontrollers such as the STM32F7 family lack hardware floating-point units powerful enough to train networks in real time. By representing weights and activations as scaled integers, the computational cost drops dramatically.

FQT defines two primary quantization schemes used throughout the ODT library. Symmetric quantization (SYM) maps real values to signed integers symmetrically around zero: real = mantissa * scale. The scale factor is chosen so the maximum absolute value in a tensor fills the integer range. Asymmetric quantization (ASYM) additionally introduces a zero-point offset: real = (mantissa - zeroPoint) * scale. ASYM better represents distributions that are not centred at zero, which is common for activations after ReLU.

The SYM_INT32 type is a special hybrid used for accumulation. Operands are quantised to 12-bit integers (ODT_SYM_OPERAND_QMAXBITS=12) so that the product of two operands fits in a 32-bit accumulator without overflow: 2^12 * 2^12 = 2^24, well within the 2^31 range of int32. Gradients use a wider 16-bit target (ODT_SYM_GRAD_QMAXBITS=16) to preserve enough resolution for learning signals.

■ **CONCEPT**	Why 12-bit operands? Because int12 * int12 = int24, and a sum of up to 511 such products stays below INT32_MAX. This avoids the need for int64 accumulators, which are expensive on Cortex-M7.

Concretely: if a weight has real value 0.05 and the scale is 0.004, then mantissa = round(0.05 / 0.004) = round(12.5) = 13. The reconstructed real value is 13 * 0.004 = 0.052, introducing a quantisation error of 0.002. Over millions of training steps with stochastic rounding, these errors average out and do not bias the learned weights.

FQT is particularly important for the STM32 Nucleo-F746ZG target. This board features a Cortex-M7 core at 216 MHz with a single-precision FPU and DSP extensions. While the FPU handles float32 arithmetic efficiently for inference, the memory and bandwidth constraints (320 KB SRAM, no cache for the AHB bus region) make full-precision backward passes impractical. FQT reduces gradient storage by 2-4x and computation by 2-8x depending on the quantisation scheme.

The FQT training pipeline in ODT proceeds as follows: (1) forward pass in quantised arithmetic (SYM_INT32 or FLOAT32 depending on layer config); (2) loss computation in FLOAT32; (3) backward pass computing gradients in SYM_INT32 or FLOAT32; (4) optimizer step applying gradients to quantised weights using stochastic rounding. The quantisation scheme for each layer is set at model initialisation and does not change during training.

2. Rigged Lottery (RigL) Sparse Training

RigL (Rigged Lottery, Evci et al. 2020) is a dynamic sparse training algorithm. Instead of training a dense network and pruning afterwards, RigL maintains a fixed sparsity level throughout training by periodically updating which weights are active (non-zero) and which are inactive (zero, masked out).

The algorithm alternates between two phases. During the gradient descent phase, only active weights receive gradient updates — inactive weights are frozen at zero and skipped entirely. Periodically (every T_end / (delta_T * zeta) steps), a topology update step is executed: the bottom K active weights by absolute

value are DROPPED (set to zero), and the top K inactive weights by gradient magnitude are GROWN (unmasked and initialised). This swap preserves the sparsity level while allowing the network topology to evolve toward more useful connections.

RigL achieves accuracy close to dense training at the same computational cost as a network of the target sparsity. For on-device training on STM32 MCUs, this is transformative: a 90% sparse Linear layer requires only 10% of the multiply-accumulate operations and 10% of the weight memory.

■ **RigL: Core RigL loop**	Every T_drop_grow steps: (1) find the K smallest |w| among active weights and DROP them. (2) find the K largest |grad| among inactive weights and GROW them. (3) zero-initialise newly grown weights. K = alpha * numActive, where alpha decays from 0.3 to 0 following a cosine schedule.

The cosine decay schedule for alpha is: alpha(t) = 0.3 * 0.5 * (1 + cos(pi * t / T_total)). At t=0, alpha=0.3 (30% of active weights are swapped each update). At t=T_total, alpha=0 (no more topology changes, the network has converged to its final sparse structure). This annealing prevents late-training instability.

For a 90% sparse Linear layer with 256 input and 128 output features: total weights = 32768. Active weights = 3277 (10%). At t=0, K = 0.3 * 3277 = 983 weights are swapped. At t=T/2, K ≈ 491. This means the topology changes significantly early in training, allowing the network to explore connectivity patterns, then stabilises as training converges.

The GROW step uses gradient magnitude rather than gradient value. Inactive weights receive gradients through the backward pass even though they are zeroed in the forward pass — because the backward pass computes dL/dw = dL/dy * x (input activation), which is non-zero even when w=0. These phantom gradients indicate which connections would be most beneficial to activate.

3. HAR Dataset and LibriSpeech Speaker ID

The Human Activity Recognition (HAR) dataset provides accelerometer and gyroscope readings at 50 Hz from 30 subjects performing six activities: walking, walking upstairs, walking downstairs, sitting, standing, and laying. Each window is 128 samples across 9 sensor channels (3-axis accelerometer + 3-axis gyroscope + 3-axis body acceleration), giving a 128x9 input tensor per sample. The dataset is split 70/30 into training and test sets, giving 7352 training samples and 2947 test samples.

The ODT library targets LibriSpeech speaker identification as its primary continual-learning task. Raw audio is processed into mel-frequency cepstral coefficients (MFCCs), which form fixed-length feature vectors. Typically, 40 MFCC coefficients are computed per 25ms frame with 10ms hop, giving a 40-dimensional feature per 10ms of audio. A 1-second utterance produces a 40x100 tensor. A compact convolutional or linear network classifies the speaker identity.

Continual learning means new speakers are registered without full retraining, using PPCA-based generative replay (PpcaReplay.c) to prevent catastrophic forgetting. PPCA (Probabilistic Principal Component Analysis) fits a low-rank Gaussian model to the embeddings of each known speaker. During continual learning on new speaker data, the PPCA model generates synthetic samples from old speakers to augment the training batch, maintaining performance on previously learned speakers.

The NPYLoader module loads .npy files directly from the MCU flash or SD card, parsing the NumPy binary format header to extract shape and dtype, then feeding batches through the DataLoader abstraction. This allows the same Python-generated datasets to be consumed natively on the embedded device without any conversion step.

4. Architecture Overview: Module Dependency Graph

The ODT library is organised in layers, from lowest-level utilities to application-level training loops. Understanding the dependency graph is essential for RigL integration, since a change to one module (e.g., adding a mask parameter to linearConfig_t) propagates upward to all callers.

Layer 0 — Utilities: Common.h, DTypes.h, RNG.h. These have no ODT dependencies.

Layer 1 — Core types: Quantization.h, Rounding.h, Tensor.h, ArithmeticType.h, DataStorage.h. These depend only on Layer 0.

Layer 2 — Tensor operations: TensorConversion.h, Matmul.h, Add.h, Mul.h, Reduce.h, MinMax.h, Bernoulli.h. These depend on Layer 1.

Layer 3 — Op executor: ExecuteOp.h. Depends on Layer 2.

Layer 4 — Layers: Layer.h, Linear.h, Conv1d.h, Relu.h, Softmax.h, etc. Depend on Layer 3.

Layer 5 — Optimizers: Optimizer.h, Sgd.h, AdamW.h, LrScheduler.h. Depend on Layer 4.

Layer 6 — Data pipeline: Dataset.h, NPYLoader.h, DataLoader.h. Depend on Layers 1-2.

Layer 7 — Persistence: Serialize.h, Deserialize.h. Depend on Layers 4-5.

Layer 8 — Continual learning: PpcaReplay.h. Depends on Layers 4-6.

Layer 9 — RigL (MISSING): rigLStep() would live here, depending on Layers 4-5 and using MinMax.h from Layer 2.

Chapter 1

Common.h

Logging macros and compile-time configuration

1. Usage of This File

Common.h is the universal header included by nearly every .c file in the ODT library. It provides three categories of facilities: include guards, debug-level logging macros, and the PRINT_BYTE_ARRAY diagnostic helper. Every source file begins with #define SOURCE_FILE "FILENAME" then #include "Common.h" so that log messages are automatically tagged with the originating file and function name.

The primary problem Common.h solves is consistent, zero-overhead logging across a large embedded codebase. Without it, each developer would use their own printf calls, making it impossible to silence all debug output for a release build, and impossible to filter log output by severity. Common.h centralises this so that setting a single CMake flag controls all logging library-wide.

The functions (macros) exposed by Common.h are: PRINT_DEBUG(fmt, ...) for verbose tracing; PRINT_INFO(fmt, ...) for informational messages; PRINT_ERROR(fmt, ...) for error conditions; and PRINT_BYTE_ARRAY(arr, n) which prints a byte array as hex, used to inspect raw tensor buffers during debugging.

Every .c file in ODT uses Common.h as its very first include. A typical file header looks like: #define SOURCE_FILE "Matmul" followed by #include "Common.h". If SOURCE_FILE is not defined, Common.h provides a safe fallback string so compilation still succeeds.

A complete walkthrough of using PRINT_DEBUG from scratch: Suppose you are adding a new function checkMaskConsistency() to MinMax.c. You want to print the number of active weights when running at debug level. You would write: PRINT_DEBUG("Active weights: %zu / %zu\n", activeCount, totalCount). With -DDEBUG_MODE_DEBUG set in CMake, this produces: [MinMax: checkMaskConsistency] Active weights: 3277 / 32768. Without the flag, the preprocessor removes this line entirely — zero bytes of code, zero stack usage.

Common.h also defines the do { ... } while(false) idiom for all its macros. This is a C preprocessor best practice that allows the macro to be used safely after a bare if statement without braces. Without the do-while wrapper, the following code would be silently broken: if (err) PRINT_ERROR("fail"); else doSomething(); — the else would attach to the while, not the if.

2. Interactions with Other Files and the STM32 MCU

On the STM32 Nucleo-F746ZG, standard output (printf) is routed through the ITM (Instrumentation Trace Macrocell) or UART depending on the build configuration. The ITM is a Cortex-M7 debug peripheral that transmits printf output over the SWD (Serial Wire Debug) interface to a connected debugger. On release firmware without a debugger, printf is a no-op because all logging macros compile to nothing.

The PRINT_DEBUG, PRINT_INFO, and PRINT_ERROR macros are conditionally compiled based on DLEVEL, which is set at build time via CMake flags: -DDEBUG_MODE_DEBUG sets DLEVEL=3, -DDEBUG_MODE_INFO sets DLEVEL=2, -DDEBUG_MODE_ERROR sets DLEVEL=1, and the default (no flag) sets DLEVEL=0. In a release firmware build, DLEVEL=0 and all logging macros expand to nothing,

producing zero code overhead.

Common.h depends only on standard C headers: stdbool.h for bool, stdio.h for printf, and string.h for memcpy. These are universally available in the STM32 ARM GCC toolchain. No ODT-specific dependencies exist, making Common.h safe to include first.

The dependency chain is: Common.h is included by DTypes.h, Tensor.h, Quantization.h, and every .c file. Since Common.h has no ODT dependencies, it sits at the very bottom of the include hierarchy. If Common.h did not exist, every file would need its own logging mechanism, and silencing logs for production would require editing every source file individually.

From the STM32 hardware perspective, the only relevant component is the ITM FIFO. When the debugger is connected and ITM stimulus port 0 is enabled, _write() (the newlib printf output function) writes to ITM_STIM[0]. This is a 32-bit FIFO register; writing a byte puts it in the SWD trace stream. The ITM can transmit at up to 6 Mbit/s on the SWD interface, fast enough for debug logging but not for streaming large data. PRINT_BYTE_ARRAY should therefore be used sparingly — printing a 4MB weight tensor would take seconds.

STM32 ITM	The Instrumentation Trace Macrocell (ITM) is a Cortex-M debug feature. printf() output routes here when connected via SWD. In production firmware without a debugger, all PRINT_* macros compile to zero bytes because DLEVEL=0.

3. Line-by-Line Explanation

Include guard

`#ifndef ODT_COMMON_H`	`// If not already defined, enter this block`
`#define ODT_COMMON_H`	`// Define the guard so re-inclusion is skipped`
`#include <stdbool.h>`	`// Provides the bool type (used in while(false) below)`
`#include <stdio.h>`	`// Provides printf for all log output`
`#include <string.h>`	`// Provides memcpy, memset for byte manipulation`

The include guard is the standard C idiom to prevent a header from being processed more than once per translation unit. Without it, if FileA.c includes Tensor.h which includes Common.h, and FileA.c also directly includes Common.h, the compiler would see duplicate typedef declarations and error out. The guard makes re-inclusion a no-op.

The guard name ODT_COMMON_H follows the convention: project name + file name + H, all uppercase, with dots replaced by underscores. This namespacing prevents collisions with guard names in system headers or third-party libraries.

SOURCE_FILE fallback

`#ifndef SOURCE_FILE`	`// If the .c file forgot to define SOURCE_FILE before including this`
`#define SOURCE_FILE "no Source file defined!"`	`// Provide a safe fallback — compilation still succeeds`
`#endif`	`// End conditional`

The convention in ODT is that every .c file places #define SOURCE_FILE "MyModule" before #include "Common.h". This string literal is embedded into every log message. If a developer forgets, the fallback string

alerts them during log review. Without the fallback, forgetting SOURCE_FILE would cause a compilation error because PRINT_DEBUG uses it — the fallback converts a hard error into a runtime-visible warning.

DLEVEL selection

`#ifdef DEBUG_MODE_DEBUG`	`// If CMake was invoked with -DDEBUG_MODE_DEBUG`
`#define DLEVEL 3`	`// Level 3: debug + info + error all enabled`
`#elif defined(DEBUG_MODE_INFO)`	`// Else if -DDEBUG_MODE_INFO`
`#define DLEVEL 2`	`// Level 2: info + error only`
`#elif defined(DEBUG_MODE_ERROR)`	`// Else if -DDEBUG_MODE_ERROR`
`#define DLEVEL 1`	`// Level 1: error only`
`#else`	`// Default: release build, no debug flags`
`#define DLEVEL 0`	`// Level 0: ALL logging compiled out — zero overhead`
`#endif`	

The three-level system allows graduated verbosity. During initial development, DLEVEL=3 gives full tracing. During integration testing, DLEVEL=2 shows informational flow. For CI regression tests on the STM32, DLEVEL=1 catches errors without flooding the serial port. For production firmware flashed to end devices, DLEVEL=0 gives maximum performance.

PRINT_DEBUG macro

`#define PRINT_DEBUG(str, ...) \`	`// Begin macro definition, backslash continues line`
`do { \`	`// do-while(false) makes macro safe in if/else context`
`if (DLEVEL &gt;= 3) { \`	`// Only runs at debug level 3`
`printf("\033[0;33m[%s: %s] ", SOURCE_FILE, __FUNCTION__); \`	`// Yellow colour, file:function prefix`
`printf(str, ##__VA_ARGS__); \`	`// ##__VA_ARGS__: removes leading comma when no args`
`printf("\033[0m\n"); \`	`// Reset ANSI colour, add newline`
`} \`	
`} while (false)`	`// Semicolon after macro use becomes a harmless empty statement`

The ##__VA_ARGS__ token paste operator is a GCC extension (also supported by Clang/ARM GCC). When the macro is called with only the format string and no additional arguments — PRINT_DEBUG("Mask initialised") — the ## removes the trailing comma before __VA_ARGS__, which would otherwise be a syntax error in strict C. With arguments — PRINT_DEBUG("Count: %d", n) — ## has no effect and the variadic arguments pass through normally.

The ANSI escape code \033[0;33m sets the terminal foreground colour to yellow for debug messages. \033[0;31m (red) is used for errors, \033[0;36m (cyan) for info. \033[0m resets to default. These codes work on any ANSI-compatible terminal (Linux, macOS terminal, or the STM32CubeIDE console). Windows cmd.exe may not honour them, but STM32 developers typically use Tera Term or PuTTY which do.

The do-while(false) pattern is essential for safety. Consider: if (err) PRINT_DEBUG("fail %d", code); else recover();. If PRINT_DEBUG expanded to a bare { ... } block, the semicolon after the macro call would end the if statement, and else would be a dangling else attaching to the while — a subtle, hard-to-find bug. With do-while(false), the macro is a single statement and the else attaches correctly.

■ **WARNING** Never define PRINT_DEBUG as just a printf call without do-while. The following pattern breaks if/else: #define PRINT_DEBUG(s) printf(s); printf("\n") — the second printf always executes regardless of the if condition.

PRINT_BYTE_ARRAY macro

```
#define PRINT_BYTE_ARRAY(arr, n) \                  // Print n bytes of arr as hex
do { \
if (DLEVEL &gt;= 3) { \
for (size_t _i=0; _i&lt;(n); _i++) \                 // Iterate over all bytes
printf("%02x ", (arr)[_i]); \                       // Print each byte as 2-digit hex
printf("\n"); \                                     // Newline after the array
} \
} while (false)
```

PRINT_BYTE_ARRAY is used to inspect raw tensor data buffers during debugging. For example, after packing a quantised weight tensor, you can verify the bit pattern matches expectations: PRINT_BYTE_ARRAY(tensor->data, 16) prints the first 16 bytes as hex. The loop variable _i uses a leading underscore to avoid shadowing user variables named i in the enclosing scope.

4. Missing RigL Code and Implementation

Common.h has no direct RigL gap — it is pure infrastructure. However, one useful addition for RigL debugging would be a RIGL_ASSERT macro that validates mask-to-weight consistency in debug builds. This macro would verify that every position where mask[i]==0 also has weight[i]==0.0f, catching bugs where the mask and weight tensor fall out of sync.

```
#define RIGL_ASSERT_MASK(weights, mask, n) \        // Validate mask/weight consistency
do { \
if (DLEVEL &gt;= 3) { \                              // Only in debug builds
for (size_t _j=0; _j&lt;(n); _j++) { \               // Check every weight
if (!tensorBoolGet((mask), _j)) { \                 // If weight is inactive
float _w = ((float*)(weights)-&gt;data)[_j]; \       // Read weight value
if (_w != 0.0f) { \                                 // Should be exactly zero
PRINT_ERROR("Mask/weight mismatch at %zu: w=%f", _j, _w); \
} \
} \
} \
} \
} while (false)
```

This RIGL_ASSERT_MASK would be placed in the training loop after each rigLStep() call and after each optimizer step to catch mask/weight inconsistencies immediately. In DLEVEL=0 builds it compiles to nothing.

■ **TIP** Add RIGL_ASSERT_MASK calls to the training loop during development. Mask/weight inconsistencies are the most common RigL implementation bug and are silent — the network trains but with incorrect sparsity, giving unexpectedly poor accuracy.

Chapter 2

DTypes.h + DTypes.c

Raw byte serialisation and deserialisation primitives

1. Usage of This File

DTypes provides the lowest-level data marshalling layer in ODT. Any time a tensor raw byte buffer must be interpreted as a typed value — reading a float weight from flash, writing an int32 gradient to UART — these functions are called. They are also used by the NPYLoader to decode the binary .npy payload, and by Serialize/Deserialize to stream model parameters over SerialWire.

DTypes solves the aliasing and alignment problem. In C, reading a float from an arbitrary byte pointer via a cast like *(float*)ptr violates strict aliasing rules (undefined behaviour in C99/C11) if the buffer was originally declared as uint8_t[]. Even on hardware that supports unaligned access (like Cortex-M7 with UNALIGN_TRP=0), the compiler may reorder or eliminate such loads. Using memcpy is the only standards-compliant way to reinterpret bytes.

The functions exposed by DTypes are: readBytesAsFloat(uint8_t*), readBytesAsInt32(uint8_t*), readNumberOfBytesAsInt32(uint8_t*, size_t), writeFloatToByteArray(float, uint8_t*), writeInt32ToByteArray(int32_t, uint8_t*). There are also variants for int16 and int8 used by compressed model formats.

A complete use-case walkthrough: suppose you are implementing npyLoaderGetSample() and need to read the 4th float element from a raw file buffer. The buffer is declared as uint8_t raw[1024]. The float is at byte offset 12. You would write: float val = readBytesAsFloat(&raw;[12]). This correctly handles all alignment and aliasing concerns regardless of how the uint8_t array is aligned in memory.

2. Interactions with Other Files and the STM32 MCU

DTypes.c includes only string.h (for memcpy) and its own header plus Tensor.h for the qtype enum. It sits at the base of the dependency tree — no ODT module is lower. The dependency chain is: DTypes is called by NPYLoader.c, Serialize.c, Deserialize.c, Matmul.c, Sgd.c, AdamW.c, MinMax.c, TensorConversion.c, and all element-wise arithmetic files. Removing DTypes.c would require every module to implement its own memcpy wrappers.

The STM32F746ZG Cortex-M7 is a little-endian processor, which is why readNumberOfBytesAsInt32 zero-initialises its output and copies bytes starting from the least significant end — this is correct on LE hardware but would be wrong on a big-endian system. The .npy format also uses little-endian byte order (denoted by the <PLACEHOLDER_LT> prefix in dtype strings like <PLACEHOLDER_LT>f4), so no byte-swapping is needed for the standard STM32 target.

■ **WARNING**	All DTypes functions assume little-endian byte order. The STM32F7 is LE-native, so this is safe. If you port ODT to a big-endian DSP or an Arm Cortex-A with BE mode enabled, you must add byte-swap logic in each DTypes function.

On the Cortex-M7, memcpy for 4-byte copies is typically replaced by the compiler with a single LDR (load register) instruction when the address is 4-byte aligned. When unaligned, the compiler emits a byte-by-byte

copy sequence. The STM32F7 supports hardware unaligned access (UNALIGN_TRP bit in CCR is 0 by default), but the compiler cannot rely on this for standards-compliant code, so it uses the safe multi-byte sequence.

3. Line-by-Line Explanation

readBytesAsFloat(uint8_t *bytes)

`float readBytesAsFloat(uint8_t *bytes) {`	`// Input: pointer to 4 raw bytes in memory`
`float x;`	`// Allocate a float on the stack (4 bytes)`
`memcpy(&x, bytes, sizeof(float));`	`// Copy 4 bytes from bytes into x — safe aliasing`
`return x;`	`// Return the float value whose bit pattern matches the bytes`
`}`	

The sizeof(float) is always 4 on all ARM GCC targets (IEEE-754 single precision). Using sizeof rather than the literal 4 makes the code portable to hypothetical targets where float is 8 bytes. The memcpy here is not actually a function call in optimised builds — GCC and Clang both recognise this pattern and replace it with a single instruction.

Example: if bytes = {0x00, 0x00, 0x48, 0x41}, the function returns 12.5f. This is the IEEE-754 representation of 12.5: sign=0, exponent=130 (=127+3), mantissa=0x480000, giving 1.5625 * 2^3 = 12.5.

readNumberOfBytesAsInt32(uint8_t *data, size_t numberOfBytes)

`int32_t readNumberOfBytesAsInt32(uint8_t *data, size_t numberOfBytes) {`	
`int32_t output = 0;`	`// Zero-initialise — upper bytes stay zero (zero-extension)`
`memcpy(&output, data, numberOfBytes);`	`// Copy only the requested bytes from the low end`
`return output;`	`// Upper bytes are zero (zero-extension on LE)`
`}`	

This function handles variable-width integer reads. For example, reading a 2-byte (uint16) header size from an .npy v1 file: readNumberOfBytesAsInt32(headerSizeBuf, 2) returns an int32 whose low 16 bits contain the header size and whose high 16 bits are zero. This avoids separate functions for every integer width.

The zero-initialisation is critical. Without it, the upper bytes of output would be uninitialised (garbage from the stack), and the result would be incorrect for 1- or 2-byte reads. The initialisation to 0 combined with memcpy of only numberOfBytes bytes performs zero-extension correctly for unsigned values. For signed values where sign-extension is needed (e.g., reading a signed 16-bit integer), this function is not appropriate — you would need to cast the result and sign-extend manually.

writeFloatToByteArray and writeInt32ToByteArray

`void writeInt32ToByteArray(int32_t value, uint8_t *bytes) {`	`// Serialise an int32 to 4 raw bytes`
`memcpy(bytes, &value, sizeof(int32_t));`	`// Copy the LE bit pattern of value into bytes`
`}`	
`void writeFloatToByteArray(float value, uint8_t *bytes) {`	`// Serialise a float to 4 raw bytes`

`memcpy(bytes, &value, sizeof(float));`	`// Copy the IEEE-754 bit pattern of value into bytes`
`}`	

These are the serialisation counterparts to the read functions. They are used in Serialize.c to write model parameters to a file stream, and in Matmul.c/Sgd.c to write computed results back into tensor data buffers. The memcpy direction is reversed: from the typed value into the byte array.

Example: writeFloatToByteArray(12.5f, buf) writes {0x00, 0x00, 0x48, 0x41} into buf[0..3]. writeInt32ToByteArray(-1, buf) writes {0xFF, 0xFF, 0xFF, 0xFF} (two's complement of -1 in LE).

4. Missing RigL Code and Implementation

DTypes.c has no direct RigL gap. It is a pure utility layer. However, note that tensorBoolGet and tensorBoolSet in Tensor.h provide bit-granular access to BOOL tensors — these operate at the bit level below what DTypes handles. DTypes works with full bytes and multi-byte values; the BOOL mask bit-packing is handled separately in Tensor.h using bitwise operations.

One DTypes function that would be useful for RigL mask persistence is writeBoolMaskToByteArray(bool *mask, size_t n, uint8_t *bytes), which packs n boolean values into ceil(n/8) bytes. This would be needed by Serialize.c to save sparse masks to the ODTS file format.

`/* Pack n booleans into bit-packed byte array (LSB first per byte) */`	
`void writeBoolMaskToByteArray(bool *mask, size_t n, uint8_t *bytes) {`	
`size_t numBytes = (n + 7) / 8;`	`// Ceiling division to get byte count`
`memset(bytes, 0, numBytes);`	`// Zero all bytes first`
`for (size_t i = 0; i < n; i++) {`	`// Iterate each boolean`
`if (mask[i])`	`// If this weight is active`
`bytes[i/8] |= (1u << (i%8));`	`// Set the corresponding bit (LSB-first)`
`}`	
`}`	

Chapter 3

Quantization.h + Quantization.c

Quantization schemes: SYM, ASYM, SYM_INT32, BOOL

1. Usage of This File

Quantization.h defines the quantization_t struct and its six variants: INT32, FLOAT32, SYM_INT32, SYM, ASYM, and BOOL. Every tensor_t carries a quantization_t pointer that tells the runtime how to convert its raw bytes into real numbers and back. The init functions (initSymInt32QConfig, initAsymQConfig, etc.) are called during model construction to configure each layer weight, bias, activation, and gradient tensor.

The problem Quantization.h solves is type dispatch for tensor arithmetic. Without it, every function that operates on tensors would need a switch statement over data types. With quantization_t, the type information travels with the tensor, and ExecuteOp.c uses it to select the right arithmetic kernel. This is the C equivalent of a tagged union or a Rust enum with data.

To use Quantization.h from scratch: first decide the arithmetic type for a layer (FLOAT32 for maximum precision, SYM_INT32 for embedded efficiency). Then call the appropriate init function to fill a quantization_t struct. Then attach that struct to the tensor. Example for a weight tensor: symInt32QConfig_t wQCfg; initSymInt32QConfig(HALF_AWAY, &wQCfg;); Then later: weightTensor->quantization->symInt32 = &wQCfg;

A complete concrete example of creating a SYM_INT32 weight tensor: (1) allocate a uint8_t buffer of 256*128*4 bytes from DataStorage; (2) create a shape_t with dimensions {256, 128}; (3) call initSymInt32QConfig(SR_HALF_AWAY, &qcfg;) to set stochastic rounding and 12-bit max; (4) call calibrateSymInt32Scale(&qcfg;, weightInitBuffer, 256*128) to set the scale from the weight distribution; (5) create tensor_t pointing to these; (6) assign tensor_t to linearConfig_t.weights->param. The tensor is now ready for FQT forward passes.

2. Interactions with Other Files and the STM32 MCU

Quantization.h is included by Tensor.h, which is included by everything else. The quantization_t type is the central dispatch key for ExecuteOp.c — the arithmetic type determines which kernel variant to select (ARITH_FLOAT32 vs ARITH_SYM_INT32). TensorConversion.c reads the qtype to decide which conversion path to take.

On the STM32F7, SYM_INT32 arithmetic uses the DSP-extension MAC instructions available on the Cortex-M7. The SMULL (Signed Multiply Long) instruction multiplies two 32-bit integers and stores the 64-bit result in two registers. Since ODT uses 12-bit mantissas, the product fits in 24 bits, well within a single 32-bit register, so SMULL is overkill — a simple MUL suffices. The compiler uses MUL when it can prove the inputs fit.

Dependency graph: Quantization.h depends on Rounding.h (for roundingMode_t). Rounding.h depends on RNG.h (for stochastic rounding). So adding #include "Quantization.h" transitively pulls in Rounding.h and RNG.h. This is why Common.h, DTypes.h, and Quantization.h are always the first three includes in any ODT source file.

STM32 SRAM Layout	quantization_t structs are typically allocated as static locals in the model init function. They are tiny (8-16 bytes each) and live in DTCM RAM (64 KB, fastest access, 0 wait states from the CPU). Weight tensor data lives in the main SRAM (320 KB), which has 1-2 wait states for 64-bit accesses.

3. Line-by-Line Explanation

qtype_t enum

```
typedef enum qtype {                    // Define the quantisation type discriminant
INT32,                                  // Raw 32-bit integer, no scale — used for index/count tensors
FLOAT32,                                // 32-bit IEEE-754 float — maximum precision, highest memory cost
SYM_INT32,                              // Symmetric quantisation stored as int32 mantissa + float scale
SYM,                                    // Symmetric quantisation packed to qBits (e.g. 8) bits per element
ASYM,                                   // Asymmetric: adds zeroPoint offset for skewed distributions (ReLU)
BOOL,                                   // 1 bit per element — used exclusively for RigL weight masks
} qtype_t;
```

BOOL tensors store one bit per weight in a packed byte array, so a 1024-element mask occupies only 128 bytes. tensorBoolGet and tensorBoolSet in Tensor.h perform the bit-field indexing: index/8 selects the byte, index%8 selects the bit within that byte. The BOOL type is unique: it has no scale or zeroPoint, and it does not participate in arithmetic operations — it is purely a mask.

The enum value ordering matters. Serialize.c writes the qtype as a uint8_t byte. If you add a new qtype and insert it before BOOL, the serialised file format changes and old checkpoints become unreadable. Always add new qtypes at the end of the enum.

symInt32QConfig_t

```
typedef struct symInt32QConfig {
float scale;                            // Real value = mantissa * scale. Set by calibration.
roundingMode_t roundingMode;            // HALF_AWAY (deterministic) or SR_HALF_AWAY (stochastic)
uint8_t qMaxBits;                       // Max bit width: 12 for operands, 16 for gradients
} symInt32QConfig_t;
```

The scale field is the most critical. It is set during calibration by: scale = findAbsMaxFloat(buffer, n) / (2^(qMaxBits-1) - 1). For qMaxBits=12, the denominator is 2047. If the maximum absolute weight value is 0.5, then scale = 0.5 / 2047 ≈ 0.000244. A weight of 0.05 would be quantised as mantissa = round(0.05 / 0.000244) = round(204.9) = 205. Dequantised: 205 * 0.000244 = 0.04998.

Why separate qMaxBits for operands (12) and gradients (16)? During the forward pass, two quantised operands are multiplied. The product has twice as many bits: 12+12=24 bits. This must fit in a 32-bit accumulator along with the sum of many such products. With a matrix of 256 columns, the worst-case sum is

256 * 4095^2 = 4,294,705,200, which approaches INT32_MAX = 2,147,483,647. By limiting operands to 12 bits, the worst case is 256 * 2047^2 = 1,072,627,200, safely below INT32_MAX even for large matrices.

asymQConfig_t

```
typedef struct asymQConfig {
float scale;                    // Scale factor: real = (mantissa - zeroPoint) * scale
int32_t zeroPoint;              // Offset: maps quantised zero to this real value
uint8_t qBits;                  // Bits per stored element (e.g. 8 for 256 levels)
roundingMode_t roundingMode;    // Rounding for quantisation
} asymQConfig_t;
```

The zeroPoint is int32 rather than uint8 to handle the full range. When qBits=16 and the activation distribution is all-positive (after ReLU), round(min_val/scale) can be a large positive integer far exceeding int16 range. Using int32 prevents overflow. This was fixed after issue #246 discovered that 16-bit ASYM quantisation of ReLU activations produced incorrect zeroPoints on certain layer configurations.

initAsymQConfig rejects qBits > 30. The reason: when computing mantissa = round(real/scale + zeroPoint) and clamping to [0, 2^qBits - 1], the maximum value is 2^30 - 1 = 1,073,741,823, which fits in int32. For qBits=31, the max mantissa would be 2^31-1 = INT32_MAX, which is fine in itself, but the intermediate computation real/scale could overflow float (float has only 23-bit mantissa). For qBits=32, the max mantissa 2^32-1 exceeds INT32_MAX. Hence the 30-bit limit.

initSymInt32QConfig

```
void initSymInt32QConfig(roundingMode_t rm, symInt32QConfig_t *cfg) {
cfg-&gt;roundingMode = rm;          // Store the rounding mode (deterministic or stochastic)
cfg-&gt;qMaxBits = ODT_SYM_OPERAND_QMAXBITS;  // Default: 12 bits for safe int32 accumulation
}
```

Note that initSymInt32QConfig does not set cfg->scale. Scale must be set separately after calibration. Forgetting to set scale is a common bug that results in all weights quantising to 0 (if scale=0) or to garbage (if scale is uninitialised memory). The ODT test suite checks that scale > 0.0f after calibration.

4. Missing RigL Code and Implementation

Quantization.h has no direct RigL gap. However, the BOOL quantisation type IS the foundation for RigL masks — every sparse weight tensor needs an accompanying BOOL tensor of the same shape. The allocation pattern for a BOOL mask tensor is: (1) allocate ceil(n/8) bytes from DataStorage; (2) set qtype=BOOL; (3) create tensor_t with the same shape as the weight tensor. There is no quantization_t needed for BOOL tensors since they have no scale.

> ■ **TIP** To create a BOOL mask tensor: maskBytes = (numWeights + 7) / 8. Allocate this from reserveMemory(). Create a tensor_t with data=maskBuf, shape matching the weight tensor, and quantization->type=BOOL. Initialize all bits to 1 (all weights active) using memset(maskBuf, 0xFF, maskBytes).

The six qtype values and their memory cost per element: INT32=4 bytes, FLOAT32=4 bytes, SYM_INT32=4 bytes, SYM=qBits/8 bytes, ASYM=qBits/8 bytes, BOOL=1/8 byte. For a 256x128 weight matrix (32768

elements): FLOAT32 needs 131 KB, SYM_INT32 needs 131 KB, SYM with 8-bit needs 32 KB, BOOL mask needs only 4 KB. The mask overhead is negligible.

Chapter 4

Rounding.h + Rounding.c

Deterministic and stochastic rounding for quantisation

1. Usage of This File

Rounding.h defines the roundingMode_t enum and three rounding functions: roundHalfAway (deterministic), roundSRHalfAway (stochastic), and rescaleIntoAccumulatorScale. These are called inside ExecuteOp.c during the write-back phase when quantising a float accumulator into a lower-precision target format. The rounding mode is stored in each quantization_t config struct and flows through the entire computation graph.

The problem Rounding.h solves is quantisation bias. Deterministic rounding (always round 0.5 upward) introduces a systematic positive bias in the weight distribution — every weight that falls exactly halfway between two quantisation levels is rounded up. Over thousands of gradient steps, this bias accumulates and skews the learned representation. Stochastic rounding eliminates this bias by randomly choosing the direction, with probability proportional to the fractional part.

A concrete example of stochastic rounding importance: suppose a weight is 2.3 and the quantisation step is 1.0. With deterministic rounding, this always quantises to 2 (not 3), introducing an error of -0.3 every step. Over 1000 steps, the weight is pulled 300 units below its true value. With stochastic rounding, it quantises to 2 with probability 0.7 and to 3 with probability 0.3 — the expected error is 0, and the weight converges to its true value.

For RigL sparse training, stochastic rounding is especially important for newly grown weights. A newly activated weight starts at 0.0 and receives small gradient updates. If the gradient is 0.003 and the quantisation step is 0.004, deterministic rounding always rounds to 0 — the weight never leaves zero, effectively defeating the GROW step. Stochastic rounding allows the weight to occasionally round to 0.004, giving it a chance to grow.

2. Interactions with Other Files and the STM32 MCU

Rounding.c depends on RNG.c for stochastic rounding (calls rngNextFloat() to get a uniform random value in [0,1)). Quantization.h includes Rounding.h to embed the roundingMode_t in qConfig structs. ExecuteOp.c uses the rounding mode stored in the arithmetic_t struct when writing results back to quantised tensors.

On the STM32F7, the FPU hardware round instruction maps to roundHalfAway via the standard round() function from math.h. This maps to a single VRINTA (Vector Round to nearest with ties to Away) instruction on Cortex-M7. roundSRHalfAway requires an additional call to rngNextFloat() which executes three XOR-shift instructions — total cost is about 10 clock cycles per element, versus 2 cycles for deterministic rounding. For a 32768-element weight tensor update, stochastic rounding adds about 0.26ms at 216 MHz — acceptable for training, not for inference.

STM32 FPU	The Cortex-M7 FPU supports single-precision float operations with 1-2 cycle throughput. roundHalfAway() maps to VRINTA (1 cycle). roundSRHalfAway() needs VRINTA plus rngNextFloat() (XorShift: 3 cycles). Both outperform software emulation by 10-20x.

3. Line-by-Line Explanation

roundingMode_t enum

`typedef enum { HALF_AWAY, SR_HALF_AWAY } roundingMode_t;`	`// Two rounding modes`

HALF_AWAY: standard school rounding. 2.5 rounds to 3, -2.5 rounds to -3, 2.4999 rounds to 2. This is the round() function in standard C. SR_HALF_AWAY: stochastic rounding-half-away. The value is offset by a uniform random in [-0.5, 0.5) before applying HALF_AWAY, making the rounding direction probabilistic.

roundHalfAway(float input)

`float roundHalfAway(float input) {`	`// Deterministic round-half-away-from-zero`
`return roundf(input);`	`// C standard library round() — maps to VRINTA on Cortex-M7`
`}`	

roundf() is the single-precision version of round(). On Cortex-M7, this compiles to VRINTA.F32 S0, S0 — a single-cycle FPU instruction. The wrapper function exists so that callers can use roundingMode-based dispatch without knowing whether they want roundf or the stochastic version: roundFn = (mode == HALF_AWAY) ? roundHalfAway : roundSRHalfAway.

roundSRHalfAway(float input)

`float roundSRHalfAway(float input) {`	`// Stochastic rounding`
`float noise = rngNextFloat() - 0.5f;`	`// Uniform noise in [-0.5, +0.5)`
`return roundHalfAway(input + noise);`	`// Add noise then round deterministically`
`}`	

The mathematical equivalence: E[roundSRHalfAway(x)] = x for all real x. This follows because for fractional part f = x - floor(x): if noise in [-0.5, 0.5) is added, the probability of rounding up is f (the fractional part) and the probability of rounding down is 1-f. Expected value = (floor(x)+1)*f + floor(x)*(1-f) = floor(x) + f = x.

Example: input = 2.3. Fractional part = 0.3. rngNextFloat() generates values in [0,1). After subtracting 0.5, noise is in [-0.5, 0.5). noise + 2.3 rounds to 3 when noise > 0.7 (probability 0.3) and to 2 when noise <= 0.7 (probability 0.7). Over 1000 calls: approximately 300 round to 3 and 700 round to 2. Mean ≈ 300/1000 * 3 + 700/1000 * 2 = 0.9 + 1.4 = 2.3. Exactly correct.

> ■ **WARNING** Never use stochastic rounding during inference. It introduces unnecessary randomness and slows down the forward pass. Only use SR_HALF_AWAY for the optimizer write-back step during training.

rescaleIntoAccumulatorScale

`int32_t rescaleIntoAccumulatorScale(int32_t paramQ, float paramScale,`	
`float accScale, roundingMode_t mode) {`	
`float realVal = paramQ * paramScale;`	`// Step 1: dequantise to real value`
`float rescaled = realVal / accScale;`	`// Step 2: requantise at accumulator scale`
`float rounded = (mode == HALF_AWAY) ?`	`// Step 3: round using requested mode`
`roundHalfAway(rescaled) : roundSRHalfAway(rescaled);`	
`return (int32_t)rounded;`	`// Step 4: cast to int32 mantissa`

```
}
```

This function is the heart of the mixed-precision accumulation system. It is called when adding a SYM_INT32 gradient (with paramScale) into an accumulator that has a different scale (accScale). The real value of the gradient is paramQ * paramScale. To express this in the accumulator scale, divide by accScale. The result is a floating-point number of accumulator-scale units, which is then rounded to an int32 mantissa.

Concrete example: paramQ = 1500 (a weight gradient mantissa), paramScale = 0.000244 (weight scale, qMaxBits=12), accScale = 0.0001 (accumulator scale). realVal = 1500 * 0.000244 = 0.366. rescaled = 0.366 / 0.0001 = 3660. With HALF_AWAY rounding, this returns 3660. With SR_HALF_AWAY, it would return 3660 (since .0 fractional part, always rounds to 3660).

4. Missing RigL Code and Implementation

Rounding.c has no direct RigL gap. Stochastic rounding is particularly important for RigL training: newly grown weights are zero-initialised and receive stochastic gradient updates. With SR_HALF_AWAY, small gradients (smaller than the quantisation step) still have a probability of updating the weight equal to gradient/scale. This allows newly grown weights to escape the quantisation dead zone and begin learning effectively.

One RigL-related consideration: the DROP step selects weights by absolute value and sets them to exactly 0.0f. This bypasses rounding entirely — the weight is not quantised to the nearest representable value, it is hard-zeroed. The GROW step initialises new weights to exactly 0.0f. These hard assignments ensure the mask and weight are always consistent: if mask[i]==0, then weight[i]==0.0f exactly, with no rounding error.

■ **TIP**	Use SR_HALF_AWAY for all weight updates in RigL training. Use HALF_AWAY only for inference-time quantisation where reproducibility matters more than bias correction.

Chapter 5

Tensor.h + Tensor.c

The central data structure: shape, quantisation, sparsity

1. Usage of This File

Tensor.h defines the four core structs — shape_t, sparsity_t, tensor_t, and parameter_t — plus all utility functions for creating, inspecting, and manipulating tensors. Every layer, every optimizer, every data loader operates on tensor_t pointers. Understanding this file is a prerequisite for understanding all other ODT modules.

The problem Tensor.h solves is unified representation of all data in the neural network — weights, activations, gradients, masks, labels — in a format that carries type information (via quantization_t), shape information (via shape_t), and sparsity metadata (via sparsity_t). Without this unified type, every function would need to accept raw pointers plus separate shape and type arguments, making the API unwieldy and error-prone.

Complete walkthrough of creating a weight tensor from scratch: (1) Allocate shape: shape_t *shp = createShape(2, (size_t[]){256, 128}); (2) Allocate data buffer: uint8_t *buf = reserveMemory(256*128*4); (3) Set initial weights using Xavier initialisation via your init function; (4) Allocate quantization: quantization_t *q = createQuantization(SYM_INT32); initSymInt32QConfig(SR_HALF_AWAY, q->symInt32); calibrateScale(q->symInt32, (float*)buf, 256*128); (5) Create tensor: tensor_t *t = createTensor(buf, shp, q, NULL); (6) Assign to layer config: cfg->weights->param = t.

The parameter_t struct wraps a tensor_t for trainable parameters. It holds both the parameter tensor (param) and its gradient tensor (grad), keeping related data together. The optimizer iterates a list of parameter_t pointers, updating param using grad. For RigL, a mask field should be added here.

2. Interactions with Other Files and the STM32 MCU

Tensor.h is the most widely included header in the library — every other module includes it. It depends on Quantization.h and transitively on Rounding.h and RNG.h. The dependency chain from any file: MyFile.c -> Tensor.h -> Quantization.h -> Rounding.h -> RNG.h -> stdint.h.

On the STM32, tensor data buffers are typically allocated from DataStorage (a static bump allocator). shape_t and quantization_t are typically stack-allocated in init functions and then their addresses stored in tensor_t. This means the tensor_t.shape pointer points into the init function stack — this is safe only if the init function does not return before the tensor is last used (which is always the case since tensors live for the entire training run).

The orderOfDimensions field in shape_t enables zero-copy transpose. For a 256x128 weight matrix: Physical storage is row-major: weight[i][j] is at byte offset (i*128+j)*4. After transposeTensor(w,0,1): orderOfDimensions becomes {1,0}, meaning "dimension 0 in my logical view is physical dimension 1". When Matmul.c computes the row index for the transposed tensor, it follows orderOfDimensions to get the physical index. No bytes are moved.

SRAM Layout	On STM32F746ZG: DTCM 64KB is fastest (0-cycle for CPU). AXI SRAM 256KB is used for large tensor buffers (1-2 cycle wait). FLASH 1MB is read-only, used for storing constant weight tensors for inference. DataStorage places the pool in AXI SRAM, with critical small tensors in DTCM.

3. Line-by-Line Explanation

shape_t struct

```
typedef struct shape {
size_t numberOfDimensions;  // Rank of tensor: 1=vector, 2=matrix, 3=3D, 4=batch
size_t *dimensions;  // Array of size numberOfDimensions giving each dim size
size_t *orderOfDimensions;  // Permutation for zero-copy transposition (e.g. {1,0}=transpose)
} shape_t;
```

For a 2D tensor (matrix) with dimensions {256, 128}: numberOfDimensions=2, dimensions[0]=256, dimensions[1]=128, orderOfDimensions={0,1} (identity — no transposition). After transposeTensor(t, 0, 1): dimensions becomes {128, 256}, orderOfDimensions becomes {1,0}. The logical shape is now 128x256, but the physical data is still laid out as 256x128 (unchanged).

calcNumberOfElementsByShape() computes the product of all dimensions: for {256, 128} this gives 32768. calcNumberOfElementsByTensor() does the same via the tensor pointer. These are used everywhere to iterate over tensor elements without knowing the rank.

tensor_t struct

```
typedef struct tensor {
uint8_t *data;  // Raw byte buffer — interpreted according to quantization
shape_t *shape;  // Logical shape and dimension ordering for zero-copy transpose
quantization_t *quantization;  // Type tag: how to convert bytes to real values
sparsity_t *sparsity;  // Sparsity metadata (currently empty struct, NULL if dense)
} tensor_t;
```

The data buffer is the raw memory. For FLOAT32, each element is 4 bytes. For SYM_INT32, each mantissa is 4 bytes (int32). For BOOL, each 8 elements share 1 byte. For SYM with qBits=8, each element is 1 byte. The quantization field tells functions how to interpret data.

A critical invariant: tensor_t.data must be allocated with at least calcNumberOfElementsByShape(shape) * bytesPerElement(qtype) bytes. Violating this causes out-of-bounds memory access, which on the STM32 causes a HardFault exception (the Cortex-M7 bus protection unit traps the access). Always use reserveMemory() from DataStorage which tracks total allocation.

parameter_t struct

```
typedef struct parameter {
tensor_t *param;  // The weight tensor (the actual learned values)
```

`tensor_t *grad;`	`// The gradient tensor (accumulated during backward pass)`
`} parameter_t;`	

parameter_t groups a weight and its gradient. The optimizer accesses both fields: it reads grad to compute the update direction and writes to param with the new value. For RigL, a third field tensor_t *mask should be added here, allowing the optimizer to check mask->data when updating param.

tensorBoolGet and tensorBoolSet

`bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex) {`	
`return (tensor->data[flatIndex / 8] >> (flatIndex % 8)) & 1;`	`// Extract bit flatIndex from packed array`
`}`	
`void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value) {`	
`uint8_t mask = 1u << (flatIndex % 8);`	`// Create bit mask for position within byte`
`if (value) tensor->data[flatIndex / 8] |= mask;`	`// Set bit to 1 (activate weight)`
`else tensor->data[flatIndex / 8] &= ~mask;`	`// Clear bit to 0 (deactivate weight)`
`}`	

These two functions are the sole interface for RigL weight mask manipulation. flatIndex is the element linear index in the weight tensor (same indexing as the weight data buffer). Example for a 256x128 matrix: weight at row 5, col 3 has flatIndex = 5*128+3 = 643. tensorBoolGet(mask, 643) reads byte 643/8=80 and extracts bit 643%8=3: (data[80] >> 3) & 1.

The bit packing is LSB-first: flatIndex 0 is bit 0 of byte 0, flatIndex 7 is bit 7 of byte 0, flatIndex 8 is bit 0 of byte 1. This matches the convention used by numpy bool arrays and standard C bitfields. A freshly initialised mask with all bits 1 (memset 0xFF) means all 8 weights per byte are active.

Performance note: tensorBoolGet and tensorBoolSet are not inlined by default in debug builds. In tight loops (like the RigL drop/grow step scanning all N weights), this overhead adds up. In production code, the bit extraction should be done inline or the entire mask byte fetched once and all 8 bits processed in a single loop iteration.

transposeTensor

`void transposeTensor(tensor_t *tensor, size_t dim0, size_t dim1) {`	
`size_t tmp = tensor->shape->orderOfDimensions[dim0];`	`// Save dim0 logical order`
`tensor->shape->orderOfDimensions[dim0] =`	
`tensor->shape->orderOfDimensions[dim1];`	`// Swap dim0 and dim1 logical orders`
`tensor->shape->orderOfDimensions[dim1] = tmp;`	`// Complete swap`
`size_t tmpDim = tensor->shape->dimensions[dim0];`	`// Also swap the dimension sizes`
`tensor->shape->dimensions[dim0] = tensor->shape->dimensions[dim1];`	
`tensor->shape->dimensions[dim1] = tmpDim;`	
`}`	

Linear.c calls transposeTensor(W, 0, 1) before matmul (to get W^T = [inFeatures, outFeatures]) and again after to restore the original layout [outFeatures, inFeatures]. Because no data is moved, this costs only 4 pointer/integer swaps — O(1) regardless of tensor size. For a 256x128 weight matrix, this saves moving 131 KB of data twice per forward pass.

Warning: after transposeTensor, the physical data layout is unchanged but the logical index mapping changes. Any function that iterates over tensor elements using the raw data buffer (rather than calcFlatIndex with orderOfDimensions) will see incorrect results. Always use calcFlatIndex() to compute byte offsets when the tensor may be transposed.

4. Missing RigL Code and Implementation

■ **RigL: Weight Mask Field Missing from parameter_t**	parameter_t has no mask field. For RigL, every sparse parameter needs an accompanying BOOL tensor. The cleanest integration point is parameter_t, so the optimizer can access the mask alongside param and grad without any additional plumbing.

```
/* Proposed RigL-extended parameter_t: */
typedef struct parameter {
tensor_t *param;                    // The weight tensor
tensor_t *grad;                     // The gradient tensor
tensor_t *mask;                     // NEW: BOOL tensor, 1=active,
                                    0=inactive. NULL = dense.
} parameter_t;
```

With mask in parameter_t: the forward pass in Linear.c accesses cfg->weights->mask and passes it to matmulFloatCore. The backward pass accesses cfg->weights->mask and passes it to linearCalcWeightGradsFloat32. The optimizer accesses optim->params[i]->mask and checks it in sgdUpdateKernel. The RigL step function accesses layer->config->linear->weights->mask to perform drop/grow.

For dense layers: mask is NULL. All existing code paths that receive a NULL mask must treat it as "all active" — they skip the mask check and process all weights. This backward-compatible design allows sparse and dense layers to coexist in the same model.

Chapter 6

TensorConversion.h + TensorConversion.c

Converting tensors between quantisation formats

1. Usage of This File

TensorConversion provides functions to convert tensor data between different quantisation types: FLOAT32 to SYM_INT32, SYM to FLOAT32, ASYM to FLOAT32, and so on. ExecuteOp.c uses these in the pre-processing phase of every op to ensure all operands are in the arithmetic type required by the kernel before computation begins.

The problem TensorConversion solves is the impedance mismatch between storage format and compute format. A weight tensor might be stored as SYM with 8-bit quantisation (compact for flash storage) but the matmul kernel expects SYM_INT32 format (32-bit mantissas for accumulation). TensorConversion converts on the fly, writing into a temporary scratch buffer managed by ExecuteOp.

Complete walkthrough: suppose a Linear layer is configured with SYM_INT32 arithmetic but its weight tensor is stored as FLOAT32 (perhaps because calibration has not run yet). ExecuteOp detects the mismatch: input qtype is FLOAT32 but arithmetic type is ARITH_SYM_INT32. It calls convertFloat32ToSymInt32(src, dst, n, scale, qMaxBits, roundingMode) to produce a SYM_INT32 version in the scratch buffer. The kernel then operates on the converted buffer.

2. Interactions with Other Files and the STM32 MCU

TensorConversion.c depends on DTypes.h, Tensor.h, Quantization.h, and Rounding.h. It is called exclusively by ExecuteOp.c. On the STM32F7, the conversion from FLOAT32 to SYM_INT32 executes in the FPU: multiply by (1/scale), VCVT to int32, clamp. This sequence takes about 5 clock cycles per element on Cortex-M7.

Memory implications: TensorConversion always writes to a separate scratch buffer — it never modifies the input tensor in place. This is essential for correct gradient computation: the backward pass needs the original (unconverted) forward input. The scratch buffers are allocated from DataStorage and reused across operations. The maximum scratch buffer size is max(input tensor bytes, output tensor bytes) per layer.

3. Line-by-Line Explanation

float32ToSymInt32 conversion

`void convertFloat32ToSymInt32(tensor_t *src, tensor_t *dst,`	
`symInt32QConfig_t *cfg) {`	
`size_t n = calcNumberOfElementsByTensor(src);`	`// Total element count`
`float *srcData = (float *)src->data;`	`// Input as float array`
`int32_t *dstData = (int32_t *)dst->data;`	`// Output as int32 mantissa array`

`float scale = cfg->scale;`	`// Quantisation scale`
`int32_t qMax = (1 << (cfg->qMaxBits - 1)) - 1;`	`// e.g. 2047 for qMaxBits=12`
`for (size_t i = 0; i < n; i++) {`	
`float scaled = srcData[i] / scale;`	`// Express in scale units`
`float rounded = (cfg->roundingMode == HALF_AWAY) ?`	
`roundHalfAway(scaled) : roundSRHalfAway(scaled);`	`// Round to integer`
`int32_t q = (int32_t)rounded;`	`// Cast to int32`
`q = q > qMax ? qMax : q < -qMax ? -qMax : q;`	`// Clamp to bit-width range`
`dstData[i] = q;`	`// Store mantissa`
`}`	
`}`	

The clamping step is critical. Without it, a weight of 100.0 with scale=0.1 would produce mantissa=1000, which exceeds the 12-bit range of 2047 — but only if weights are poorly initialised or calibration is off. The clamp prevents silent overflow while allowing execution to continue. A warning should be logged if clamping occurs frequently.

Example: srcData[i]=0.05, scale=0.000244, qMax=2047. scaled=0.05/0.000244=204.9. rounded=205. q=205. No clamping needed (205 < 2047). dstData[i]=205. To verify: 205 * 0.000244 = 0.050 (approximately — the quantisation error is 0.05 - 0.04998 = 0.00002).

4. Missing RigL Code

TensorConversion.c has no direct RigL gap. It is a utility layer invoked transparently by ExecuteOp. The RigL mask check happens in the kernel itself (Matmul.c, Sgd.c), after conversion is complete. One minor consideration: when converting a SYM_INT32 weight tensor that has been zeroed by RigL (mask=0 → w=0.0), the converted mantissa should be exactly 0 — this is guaranteed since 0.0f/scale = 0.0, round(0.0)=0.

Chapter 7

DataStorage.h + DataStorage.c

Static memory pool for tensor data buffers

1. Usage of This File

DataStorage implements a static bump allocator that serves as the memory backend for all tensor data buffers on embedded targets. Instead of calling malloc (which risks heap fragmentation on a 320 KB SRAM MCU), all tensor data is carved from a single large uint8_t array declared at compile time. reserveMemory(n) advances a pointer by n bytes and returns the old pointer.

The bump allocator pattern is optimal for neural network workloads because the allocation pattern is completely static: all tensors are allocated once at model init and never freed. This eliminates fragmentation entirely. The total pool size (STORAGE_POOL_SIZE) is computed offline from the model architecture: sum of weight, gradient, activation, and mask buffer sizes, plus scratch buffers for ExecuteOp.

Walkthrough: to add a RigL mask for a 256x128 Linear layer, compute maskBytes = (256*128 + 7)/8 = 4096 bytes. In the model init function, after reserving the weight buffer: uint8_t *maskBuf = reserveMemory(4096); memset(maskBuf, 0xFF, 4096); This initialises all 32768 mask bits to 1 (all weights active). Then create a BOOL tensor_t from maskBuf and assign it to cfg->weights->mask.

2. Interactions with Other Files and the STM32 MCU

DataStorage.c is called once per tensor buffer during model initialisation. Every call to setTensorValues that needs a data buffer ultimately goes through DataStorage. On the STM32F746ZG, the static pool is declared as: static uint8_t storagePool[STORAGE_POOL_SIZE] __attribute__((section(".dtcm_data"))); This places the pool in the DTCM RAM region (64 KB, fastest access). If the pool exceeds 64 KB, it overflows to AXI SRAM.

The linker script for STM32 must be modified to define the .dtcm_data section in DTCM memory (0x20000000 - 0x20010000). Without this section attribute, the pool lands in AXI SRAM and performance degrades by about 20% due to the additional wait states.

STM32 Memory Map	DTCM 64KB at 0x20000000: 0-cycle CPU access, used for stack and critical buffers. AXI SRAM 256KB at 0x20010000: 1-2 cycle access, used for large tensors. FLASH 1MB at 0x08000000: code and read-only constants. BKPSRAM 4KB at 0x40024000: battery-backed, survives reset.

3. Line-by-Line Explanation

`static uint8_t storagePool[STORAGE_POOL_SIZE];`	`// Fixed-size array in BSS (zero-initialised at boot)`
`static size_t storageUsed = 0;`	`// Allocation watermark: bytes used so far`
`uint8_t *reserveMemory(size_t numberOfBytes) {`	

`uint8_t *ptr = &storagePool[storageUsed];`	`// Return pointer to current allocation position`
`storageUsed += numberOfBytes;`	`// Advance watermark (bump allocation)`
`/* Optional: check storageUsed &lt;= STORAGE_POOL_SIZE */`	`// Add bounds check in debug builds`
`return ptr;`	`// Caller owns this memory region`
`}`	

This is a bump allocator: O(1) allocation, no fragmentation, no free() support. The pool is zero-initialised by the C runtime at startup (BSS segment), so tensor buffers start zeroed — no explicit memset(buf, 0, n) is needed for activation tensors whose initial value does not matter. Weight tensors need explicit Xavier or normal initialisation regardless.

In debug builds, add an assertion: assert(storageUsed <= STORAGE_POOL_SIZE). This catches the common mistake of forgetting to increase STORAGE_POOL_SIZE when adding new layers or RigL masks. On the STM32, a pool overflow silently corrupts adjacent memory, causing mysterious HardFault exceptions far from the allocation site.

■ **WARNING**	There is no free() — once memory is reserved, it is owned for the lifetime of the program. This is correct for static neural network topologies. Do NOT call reserveMemory() in a loop or from any function called repeatedly during training — call it only during model init.

4. Missing RigL Code

For RigL, each sparse layer needs additional reserveMemory calls for the mask tensors. Add these immediately after the corresponding weight buffer reservations. The total additional memory for masks is: sum over sparse layers of ceil(numWeights/8) bytes. For a model with three 256x128 Linear layers at 90% sparsity, mask overhead is 3 * 4096 = 12 KB — negligible versus the 393 KB for weight buffers.

`/* RigL mask allocation in model init — add after weight buffer */`	
`size_t wSize = outFeat * inFeat;`	`// Number of weight elements`
`uint8_t *wBuf = reserveMemory(wSize*4);`	`// Weight buffer: 4 bytes per float`
`uint8_t *mBuf = reserveMemory((wSize+7)/8);`	`// Mask buffer: 1 bit per weight`
`memset(mBuf, 0xFF, (wSize+7)/8);`	`// All weights active initially`

Chapter 8

ArithmeticType.h + Arithmetic.h/c

Arithmetic dispatch types and element-wise operations

1. Usage of This File

ArithmeticType.h defines the arithmetic_t struct that carries an ARITH_FLOAT32 or ARITH_SYM_INT32 tag plus a rounding mode. Every executeOp call carries an arithmetic_t that determines which kernel variant to invoke. Arithmetic.h/c provides the low-level element-wise math kernels (add, multiply, divide, etc.) that operate on dequantised values.

The arithmetic_t struct is the runtime type tag for computation — distinct from quantization_t which is the storage type tag. A tensor might be stored as SYM (8-bit packed) but computed with ARITH_SYM_INT32 (unpacked 32-bit mantissas). ExecuteOp uses arithmetic_t to select the right conversion and the right kernel, while quantization_t on the tensor specifies how to read and write the raw bytes.

2. Interactions with Other Files and the STM32 MCU

arithmetic_t is stored in linearConfig_t (forwardMath, weightGradMath, propLossMath), conv1dConfig_t, and similar layer config structs. ExecuteOp reads the arithmetic type to select between float and SYM_INT32 code paths. On the STM32F7, ARITH_FLOAT32 uses the VFP/NEON unit and ARITH_SYM_INT32 uses the integer DSP extension (SMULL, SMLAL instructions).

3. Line-by-Line Explanation

`typedef enum { ARITH_FLOAT32, ARITH_SYM_INT32 } arithmeticType_t;`	`// Two compute backends`
`typedef struct {`	
`arithmeticType_t type;`	`// ARITH_FLOAT32 or ARITH_SYM_INT32`
`roundingMode_t roundingMode;`	`// Rounding for write-back quantisation`
`} arithmetic_t;`	

arithmeticFromQuantizationOrDefault() maps a qtype to an arithmetic_t: FLOAT32 -> ARITH_FLOAT32 with HALF_AWAY; SYM/SYM_INT32 -> ARITH_SYM_INT32 with SR_HALF_AWAY; BOOL -> ARITH_FLOAT32 (masks do not need arithmetic); NULL -> ARITH_FLOAT32 as safe default.

4. Missing RigL Code

ArithmeticType.h has no direct RigL gap. The arithmetic dispatch machinery is reused as-is for RigL computations: the drop/grow step runs in FLOAT32 mode to compare absolute values of weights and gradients.

Chapter 9

ExecuteOp.h + ExecuteOp.c

The four-phase op executor: convert, allocate, run, write

1. Usage of This File

ExecuteOp is the central dispatch engine for all tensor operations. Rather than each kernel function managing its own type conversion and output quantisation, all that boilerplate is centralised here. The caller specifies an opSpec_t (kernel function pointer, input tensors, arithmetic type, output mode) and ExecuteOp handles the rest.

The four phases: Phase 1 — Input conversion: for each input tensor, if its quantisation type does not match the requested arithmetic type, allocate a scratch buffer and convert it. Phase 2 — Output allocation: allocate a scratch buffer for the raw output in the arithmetic type. Phase 3 — Kernel execution: call opSpec.kernel(converted_inputs, n, rawOut, aux, ctx). Phase 4 — Write-back: convert rawOut into the output tensor quantisation format using the arithmetic rounding mode.

Three output modes exist: OUT_WRITE overwrites the output tensor with the new value (used for forward pass activations). OUT_ACC_DYNAMIC_RESCALE adds to the output tensor with dynamic scale recalibration (used for gradient accumulation across batches). OUT_ACC_FIXED_SCALE adds using a fixed pre-set scale (used for bias-add in matmul). The aliasing optimisation applies when input and output both have FLOAT32 quantisation — the kernel can write directly to the output tensor data buffer without a scratch buffer.

2. Interactions with Other Files and the STM32 MCU

ExecuteOp is called from Linear, Conv1d, Relu, Softmax, and all other layer forward/backward functions. It depends on TensorConversion.h (to convert inputs to the arithmetic type), DataStorage.h (for temporary scratch buffers), and DTypes.h. On the STM32, the scratch buffers are allocated from the DataStorage pool at model init time — ExecuteOp does not call malloc.

3. Line-by-Line Explanation

`executeOp(&(opSpec_t){`	`// Compound literal – inline opSpec, no named variable needed`
`.kernel = myKernelFn,`	`// Function pointer to the computation kernel`
`.inputs = (tensor_t *[]){a, b},`	`// Array literal of input tensor pointers`
`.nInputs = 2,`	`// Length of inputs array`
`.arithmetic = {ARITH_FLOAT32, HALF_AWAY},`	`// Compute in float, deterministic rounding`
`.mode = OUT_WRITE,`	`// Overwrite (not accumulate into) output tensor`

`}, outputTensor);`	`// The destination tensor — must be pre-allocated`

The compound literal &(opSpec_t){...} is a C99 feature that creates a temporary opSpec_t struct on the stack and takes its address. This avoids having to declare a named variable just to pass to executeOp. The temporary is destroyed at the end of the statement, but executeOp uses it synchronously (no deferred execution), so this is safe.

4. Missing RigL Code

ExecuteOp.c has no direct RigL gap. The RigL drop/grow step bypasses executeOp and operates directly on the weight float buffers and BOOL mask — it does not need the type conversion machinery since the mask comparison is always done in float space. The executeOp framework is reused for all regular forward and backward computations that respect the mask.

Chapter 10

RNG.h + RNG.c

XorShift32 pseudo-random number generator

1. Usage of This File

RNG.c provides a compact, deterministic PRNG based on the XorShift32 algorithm. It is used by: Rounding.c (stochastic rounding), Bernoulli.c (Dropout mask generation), DataLoader.c (epoch shuffling), and PpcaReplay.c (replay sample selection). The global state fits in a single uint32_t, making it ideal for memory-constrained embedded targets.

XorShift32 generates a sequence of 2^32 - 1 distinct values before repeating. The period avoids 0 (the degenerate fixed point of the XorShift equations). The three XOR-shift operations mix the bits thoroughly enough to pass most statistical randomness tests, including the DieHard battery for most tests.

Complete walkthrough: to add deterministic reproducibility to a training run, set the RNG seed at the start of training: rngSetSeed(42). Then every call to rngNextFloat(), bernoulliSample(), rngShuffleIndices() will produce the same sequence regardless of the platform or build configuration. This is essential for debugging — you can reproduce any training bug by re-running with the same seed.

2. Interactions with Other Files and the STM32 MCU

RNG.c is self-contained — it has no dependencies other than stdint.h. On the STM32, the initial seed is typically set from the hardware TRNG peripheral (True Random Number Generator, available on STM32F7 via the RNG_DR register) during startup, then the XorShift PRNG takes over for speed. The TRNG produces a new random 32-bit value every 40 clock cycles — fine for seeding but too slow for per-element stochastic rounding.

STM32 TRNG	The STM32F746 includes a hardware TRNG (True Random Number Generator) at address 0x50060800. It generates random 32-bit values using analog noise. Reading RNG_DR takes ~40 cycles. Use it ONLY for seeding — rngNextFloat() costs 3 cycles (XorShift).

3. Line-by-Line Explanation

xorShift32(uint32_t x)

`uint32_t xorShift32(uint32_t x) {`	`// Pure function: takes current state, returns next state`
`x ^= (x &lt;&lt; 13);`	`// XOR with left-shift spreads high bits downward`
`x ^= (x &gt;&gt; 17);`	`// XOR with right-shift spreads low bits upward`
`x ^= (x &lt;&lt; 5);`	`// Final left-shift mix; period = 2^32-1 for all x != 0`

return x;	// All 32 bits are pseudo-random
}	

The three shift constants {13, 17, 5} are from Marsaglia 2003 "Xorshift RNGs". They were chosen so that the feedback polynomial is a primitive polynomial over GF(2), guaranteeing the maximal period of 2^32 - 1. Other valid shift triplets include {5, 17, 13} and {7, 13, 17}. The constants cannot be arbitrary — wrong constants give shorter periods.

rngNextFloat()

float rngNextFloat(void) {	
globalState = xorShift32(globalState);	// Advance global PRNG state
return (globalState >> 8) * 0x1p-24f;	// Map 24 bits to float in [0, 1)
}	

0x1p-24f is the hex float literal for 2^-24 = 1/16777216. Shifting globalState right by 8 gives a 24-bit value in [0, 2^24 - 1]. Multiplying by 2^-24 maps this to [0, 1 - 2^-24] $\subset$ [0, 1). The result is representable exactly in IEEE-754 single precision (which has a 24-bit mantissa including the implicit leading 1). This mapping avoids the modulo-bias that would result from taking globalState % 1000000.

Why 24 bits rather than 32? Because float32 has exactly 24 bits of mantissa precision. Using 32 bits would produce at most 2^24 distinct float values (due to float precision), not 2^32. Using 24 bits is correct and avoids the false impression of finer precision.

rngBounded(size_t bound) and rngShuffleIndices

size_t rngBounded(size_t bound) {	// Uniform random in [0, bound)
size_t threshold = (-bound) % bound;	// Rejection threshold to avoid modulo bias
uint32_t x;	
do { x = xorShift32Advance(); }	// Reject values below threshold
while (x < threshold);	
return x % bound;	// Unbiased modulo
}	

Rejection sampling avoids modulo bias. With bound=3 and uint32 range 0..4294967295: 4294967296 / 3 = 1431655765.33... — not an exact multiple. Values 0, 1 are more likely than 2 without rejection. The threshold (-bound) % bound = (4294967296 - 3) % 3 = 4294967293 % 3 = 1. Rejecting x < 1 means rejecting only x=0, giving an unbiased distribution.

void rngShuffleIndices(size_t *arr, size_t n) {	// Fisher-Yates shuffle in-place
for (size_t i = n-1; i > 0; i--) {	// From last element down to index 1
size_t j = rngBounded(i+1);	// Random j in [0, i]
size_t tmp = arr[i];	// Swap arr[i] and arr[j]
arr[i] = arr[j];	
arr[j] = tmp;	
}	
}	

Fisher-Yates produces a uniformly random permutation in O(n) time. DataLoader.c calls this at the start of each epoch to shuffle the training index array, ensuring every batch sees a different sample order without loading all samples into RAM first.

4. Missing RigL Code

RNG.c has no direct RigL gap. The RigL GROW step initialises newly activated weights to exactly 0.0f (not random), which is the standard RigL initialisation — zero-initialised new connections give smaller initial gradients and more stable training than random initialisation. If you want Kaiming-He random initialisation for grown weights (an experimental variant), rngNextFloat() is already available.

Chapter 11

Bernoulli.h + Bernoulli.c

Bernoulli sampling for Dropout masks

1. Usage of This File

Bernoulli.c provides bernoulliSample(p): returns true with probability p, false with probability 1-p. It is called by Dropout.c during the forward training pass to generate a random binary mask that zeros out a fraction (1-p) of activations. This prevents co-adaptation of neurons and is a key regularisation technique for avoiding overfitting on small embedded datasets.

Bernoulli sampling is implemented as a single comparison: rngNextFloat() < p. Since rngNextFloat() is uniform in [0,1), this comparison is true with probability exactly p. Example: bernoulliSample(0.8) uses keepProbability=0.8. Approximately 80 percent of activations are kept, 20 percent are zeroed. The kept activations are scaled by 1/p during training (inverted dropout) so the expected output magnitude is the same at inference time when no dropout is applied.

For RigL sparse training, there is a conceptual difference between Dropout masks and RigL masks. Dropout masks are regenerated every forward pass from Bernoulli samples, are applied to activations, and have no persistence. RigL masks are persistent across all training steps, are applied to weight parameters, and are only updated during the scheduled topology update (rigLStep). Both use bit-packed storage for memory efficiency.

2. Interactions with Other Files and the STM32 MCU

Bernoulli.c depends on RNG.c. Dropout.c calls bernoulliSample() once per activation element per forward pass. For a 256-element activation vector with p=0.8, this requires 256 RNG calls. At 3 cycles per XorShift: 256*3 = 768 cycles = 3.5 microseconds at 216 MHz. Dropout is disabled at inference time, so this cost is training-only.

Inverted Dropout	During training: output[i] = bernoulliSample(p) ? input[i]/p : 0. During inference: output[i] = input[i] (no scaling needed). This is the standard PyTorch convention and ensures the expected value of each neuron output is the same in training and inference.

3. Line-by-Line Explanation

`bool bernoulliSample(float p) {`	`// p = probability of returning true`
`return rngNextFloat() &lt; p;`	`// Uniform [0,1) threshold comparison`
`}`	

If p=0.8: rngNextFloat() generates values in [0, 1). Values in [0, 0.8) return true (keep), values in [0.8, 1.0) return false (drop). Exactly 80 percent of calls return true in the long run. The returned bool is used as the mask: true keeps the activation, false zeros it.

4. Missing RigL Code

Bernoulli.c has no direct RigL gap. Bernoulli sampling serves Dropout (activation-level) not RigL (weight-level). Understanding the difference is important: Dropout temporarily zeros activations during training but does not reduce the number of parameters or the computational cost of the matmul. RigL permanently zeros weights and reduces matmul cost by skipping inactive weight multiplications.

Chapter 12

MinMax.h + MinMax.c

Min/max search and the missing RigL K-th element functions

1. Usage of This File

MinMax.c provides utility functions to scan tensor byte buffers for extrema: findAbsMaxFloat (calibration for SYM quantisation scale), findMaxFloat, findMinFloat, findMaxInt32, findMinInt32. These are called during quantisation calibration and during RigL topology updates. findAbsMaxFloat is the most frequently called -- every scale calibration step uses it to determine the optimal quantisation scale.

Complete walkthrough of scale calibration using findAbsMaxFloat: after Xavier weight initialisation, the scale for SYM_INT32 quantisation is determined as scale = findAbsMaxFloat(weightBuf, numWeights) / (2^(qMaxBits-1) - 1). For qMaxBits=12: scale = absMax / 2047. If absMax=0.5 (typical for Xavier init with fan_in=256): scale = 0.5/2047 = 0.000244. This scale maps the weight range [-0.5, 0.5] to integer mantissas [-2047, 2047], maximising utilisation of the 12-bit integer range.

A concrete numerical example: if a weight tensor has 1000 elements with values ranging from -0.48 to +0.43, findAbsMaxFloat returns 0.48. scale = 0.48/2047 = 0.0002345. The weight 0.43 quantises to round(0.43/0.0002345) = round(1834) = 1834. Dequantised: 1834 * 0.0002345 = 0.430. Quantisation error: 0. The weight -0.48 quantises to round(-0.48/0.0002345) = round(-2047) = -2047. Dequantised: -2047 * 0.0002345 = -0.48. Perfect representation of the extremum.

2. Interactions with Other Files and the STM32 MCU

MinMax.c depends on DTypes.h for readBytesAsFloat and readBytesAsInt32. It is called by Quantization.c (calibration), TensorConversion.c (validation), Softmax.c (finding max for numerical stability), and the missing rigLStep() (K-th element threshold queries for drop and grow decisions). On STM32F7, findAbsMaxFloat with the FPU uses VABS.F32 and VMAXNM.F32 -- both single-cycle FPU instructions.

If MinMax.c did not exist, every module needing a min/max scan would implement its own loop. Worse, the Softmax numerical stability subtraction (subtract max before exp) would need to be inlined in Softmax.c. Centralising these operations makes the codebase easier to optimise: once findAbsMaxFloat is loop-unrolled or uses SIMD intrinsics, all callers benefit.

3. Line-by-Line Explanation

findAbsMaxFloat

`float findAbsMaxFloat(uint8_t *bytes, size_t n) {`	
`float *v = (float *)bytes;`	`// Reinterpret raw bytes as float array (safe: buffer is FLOAT32)`
`float mx = fabsf(v[0]);`	`// Seed maximum with |first element|`
`for (size_t i = 1; i < n; i++) {`	`// Scan all remaining elements`
`float a = fabsf(v[i]);`	`// Absolute value of element i`

`if (a > mx) mx = a;`	`// Update running maximum`
`}`	
`return mx;`	`// Return max |value| across all elements`
`}`	

fabsf() (single-precision absolute value) is used rather than fabs() (double) to stay in the FPU single-precision pipeline. On Cortex-M7, VABS.F32 takes 1 cycle. Using fabs() would cause the compiler to promote to double and use the slower software double library, adding 10-20x overhead per element.

4. Missing RigL Code and Implementation

■ RigL: Missing: findAbsKthSmallestActive + findAbsKthLargestInactive	RigL needs two new functions in MinMax.c to find threshold values for the DROP step (K-th smallest active |weight|) and GROW step (K-th largest inactive |gradient|). Neither exists currently.

`float findAbsKthSmallestActive(tensor_t *weights, tensor_t *mask, size_t K) {`	
`size_t N = calcNumberOfElementsByTensor(weights);`	`// Total weight count`
`float *w = (float *)weights->data;`	`// Weight float buffer`
`size_t count = 0;`	`// Count of active weights`
`/* Count active weights to size scratch buffer */`	
`for (size_t i = 0; i < N; i++)`	
`if (tensorBoolGet(mask, i)) count++;`	`// Count active (mask=1) weights`
`float *scratch = reserveTemp(count * sizeof(float));`	`// Temporary scratch buffer`
`size_t idx = 0;`	
`for (size_t i = 0; i < N; i++)`	
`if (tensorBoolGet(mask, i))`	`// For each active weight`
`scratch[idx++] = fabsf(w[i]);`	`// Store its absolute value`
`/* Partial selection sort: K passes find K smallest */`	
`for (size_t k = 0; k < K && k < count; k++) {`	
`size_t minIdx = k;`	
`for (size_t j = k+1; j < count; j++)`	
`if (scratch[j] < scratch[minIdx]) minIdx = j;`	`// Track minimum position`
`float tmp = scratch[k];`	`// Swap to front`
`scratch[k] = scratch[minIdx];`	
`scratch[minIdx] = tmp;`	
`}`	
`return (count >= K) ? scratch[K-1] : 0.0f;`	`// K-th smallest value`
`}`	

After findAbsKthSmallestActive returns dropThresh: iterate all active weights, drop (zero and deactivate) those with |w| <= dropThresh. The partial sort runs K passes of selection sort on the active weight magnitudes. Complexity: O(K * numActive). For K=5% of numActive=3277: about 537K comparisons. At 2 cycles each on Cortex-M7: about 5 ms.

```
float findAbsKthLargestInactive(tensor_t *grads, tensor_t *mask, size_t K) {
size_t N = calcNumberOfElementsByTensor(grads);
float *g = (float *)grads-&gt;data;                    // Gradient float buffer
size_t count = 0;
for (size_t i = 0; i &lt; N; i++)
if (!tensorBoolGet(mask, i)) count++;                 // Count inactive (mask=0) weights
float *scratch = reserveTemp(count * sizeof(float));  // Temporary scratch
size_t idx = 0;
for (size_t i = 0; i &lt; N; i++)
if (!tensorBoolGet(mask, i))                          // For each inactive weight
scratch[idx++] = fabsf(g[i]);                         // Store |gradient| magnitude
for (size_t k = 0; k &lt; K && k &lt; count; k++) {
size_t maxIdx = k;
for (size_t j = k+1; j &lt; count; j++)
if (scratch[j] &gt; scratch[maxIdx]) maxIdx = j;       // Track maximum
float tmp = scratch[k];
scratch[k] = scratch[maxIdx];
scratch[maxIdx] = tmp;
}
return (count &gt;= K) ? scratch[K-1] : 0.0f;          // K-th largest gradient magnitude
}
```

Inactive weights receive gradient signals during the backward pass because dL/dw = dL/dy * x even when w=0. These phantom gradients indicate which connections would be most useful to activate. findAbsKthLargestInactive finds the threshold above which the top-K inactive gradients fall -- these weights are candidates for the GROW step.

■ **WARNING**	Tie-breaking: if multiple weights have |w| exactly equal to dropThresh, more than K weights may be dropped, changing the sparsity level. In production, track a drop counter and stop after exactly K drops. Similarly for the grow step.

Chapter 13

Matmul.h + Matmul.c

Matrix multiplication -- the core of Linear layer performance

1. Usage of This File

Matmul.c implements matrix multiplication for FLOAT32, INT32, and SYM_INT32. Linear.c calls these during forward and backward passes. The matmul kernels are the most compute-intensive functions in the library for fully-connected speaker ID models.

For a Linear layer with 256 input and 128 output features: forward pass needs 32768 MACs, weight grad backward needs 32768 MACs, propagated loss backward needs 32768 MACs. Total per sample: 98304 MACs. At 216 MHz with 1 MAC/cycle (VMLA): about 0.46 ms per sample. For 7352 training samples per epoch: 3.4 seconds just for one Linear layer matmuls.

Matmul.c also provides bias-fused variants. Fusing the bias-add into the matmul saves one full pass over the output tensor (one read + one write of 128 floats = 1 KB memory bandwidth saved per forward pass). For embedded systems where memory bandwidth is the bottleneck, every pass saved matters.

2. Interactions with Other Files and the STM32 MCU

Matmul.c depends on Arithmetic.h, DTypes.h, Rounding.h, and Tensor.h. It is called by Linear.c (three times per backward pass) and Conv1d.c (via im2col). The dependency chain: Linear.c -> matmulFloat32TensorsWithBias -> matmulFloatCore -> readBytesAsFloat -> memcpy.

On STM32F7: FLOAT32 matmul uses VMLA.F32 (FMA, 1-cycle throughput, 5-cycle latency). Loop unrolling by 4 hides the latency: process 4 FMAs in flight simultaneously. SYM_INT32 matmul uses MUL + ADD (since operands are 12-bit, product fits in 32 bits, no SMLAL needed).

Loop Unrolling	Cortex-M7 FPU VMLA latency is 5 cycles. To achieve 1-cycle throughput, maintain 5 independent FMAs in flight. Manual unrolling by 4-8 achieves near-peak throughput. Compiler -O2 may do this automatically with -funroll-loops.

3. Line-by-Line Explanation

matmulFloatCore (simplified)

`for (size_t row = 0; row < aRows; row++) {`	`// Iterate output rows`
`for (size_t col = 0; col < bCols; col++) {`	`// Iterate output columns`
`float result = biasVal;`	`// Pre-load bias (fused bias-add)`
`for (size_t k = 0; k < K; k++) {`	`// Inner dot product (shared dimension)`
`result += a[row*K+k] * b[k*bCols+col];`	`// MAC: accumulate weight contribution`
`}`	
`out[row*bCols+col] = result;`	`// Write output element`
`}`	

```
}
```

The memory access pattern for b (the weight matrix) is column-major in the inner loop (stride=bCols between consecutive k values). This is cache-unfriendly for large matrices. The standard fix is to transpose B before the loop or use the im2col approach. Linear.c uses the O(1) transposeTensor trick to avoid the data copy.

4. Missing RigL Code and Implementation

■ RigL: Missing: mask check in matmulFloatCore inner loop	Without the mask check, inactive weights (mask=0, value=0.0) are still multiplied with input activations. The result of 0.0 * input = 0.0 is added to the accumulator -- correct numerically, but wasted computation. For 90 percent sparsity, 90 percent of inner loop MACs are multiplications by zero.

```
/* Modified inner loop with optional RigL mask */
for (size_t k = 0; k &lt; K; k++) {
if (wMask != NULL) {                    // Mask provided (sparse layer)
size_t mIdx = row * K + k;              // Flat weight index (transposed layout)
if (!tensorBoolGet(wMask, mIdx)) continue;  // Skip inactive weight (zero contribution)
}
result += a[row*K+k] * b[k*bCols+col];  // Accumulate only active weight
}
```

For 90 percent sparsity: 90 percent of iterations skip the MAC. The if-continue branch is predicted as taken 9 times out of 10 by the branch predictor. On Cortex-M7, a correctly-predicted not-taken branch costs 1 cycle. Measured speedup vs dense: approximately 5-6x for 90 percent sparsity (theoretical 10x reduced by loop overhead and branch prediction misses at the transition points).

An alternative implementation uses a Compressed Sparse Row (CSR) format where only active weights and their column indices are stored. CSR eliminates the branch entirely: iterate only the K_active entries. For very high sparsity (>95 percent), CSR is faster. For 80-90 percent sparsity, the branch-based mask check is simpler and comparable in speed.

Chapter 14

Add / Sub / Mul / Div / Axpby / Square / Log

Element-wise arithmetic operations

1. Usage of This File

These files implement element-wise operations on tensors. Each follows the executeOp kernel signature and operates on a pre-converted (FLOAT32 or SYM_INT32) scratch buffer. Add.c adds tensors or scalars. Axpby.c computes a*x + b*y for fused momentum updates. Square.c squares elements (used in variance computation). Log.c computes natural log (used in cross-entropy loss).

Axpby is particularly important. In SGD momentum: v_new = beta * v_old + (1-beta) * grad. This is axpby(beta, v_old, 1.0-beta, grad, v_new). In AdamW moment update: m = beta1*m + (1-beta1)*g is another axpby call. Fusing these saves one intermediate tensor and halves the memory traffic for the momentum computation.

2. Interactions with Other Files and the STM32 MCU

Each arithmetic kernel file depends on DTypes.h and Tensor.h. They are called through executeOp, which provides pre-converted float buffers. Log.c uses logf() from math.h -- on Cortex-M7, this is a 20-30 cycle software routine (no hardware log instruction). For cross-entropy loss computation on a 128-class output, 128 logf() calls take about 128*25=3200 cycles = 15 us at 216 MHz.

3. Line-by-Line Explanation

`/* Axpby kernel: out[i] = alpha * x[i] + beta * y[i] */`	
`typedef struct { float alpha, beta; } axpbyCtx_t;`	`// Context carries scalar coefficients`
`static void axpbyFloatKernel(tensor_t **ops, size_t n,`	
`tensor_t *rawOut, tensor_t *aux, const void *ctxv) {`	
`const axpbyCtx_t *ctx = (const axpbyCtx_t *)ctxv;`	`// Cast context pointer`
`size_t N = calcNumberOfElementsByTensor(rawOut);`	
`float *x = (float *)ops[0]->data;`	`// First operand: x`
`float *y = (float *)ops[1]->data;`	`// Second operand: y`
`float *out = (float *)rawOut->data;`	`// Output buffer`
`for (size_t i = 0; i < N; i++)`	
`out[i] = ctx->alpha * x[i] + ctx->beta * y[i];`	`// Fused multiply-add per element`
`}`	

4. Missing RigL Code

None of the element-wise arithmetic files has a direct RigL gap. The RigL mask check belongs in the optimizer step kernels (Sgd.c, AdamW.c), not in the arithmetic primitives. These kernels are invoked after the mask logic has already been applied by the optimizer.

Chapter 15

Reduce.h/c + Sum.h/c

Reduction operations along axes

1. Usage of This File

Reduce.c provides axis-wise reduction: sum, mean, max along a specified dimension. Sum.c provides global sum and column-wise sum. These are used by: Linear backward (bias gradient = column-sum of loss matrix), LayerNorm (mean and variance), Softmax (denominator), CrossEntropy (log-prob sum).

The bias gradient in Linear backward is a column-wise sum: biasGrad[j] = sum over batch i of loss[i][j]. For batch=32 and outFeatures=128: this sums 32 values per column, producing 128 bias gradient scalars. Total: 32*128 = 4096 additions.

2. Interactions with Other Files and the STM32 MCU

Reduce.c depends on DTypes.h, Tensor.h, and Arithmetic.h. For SYM_INT32: summing 32 gradient values of 16-bit mantissas gives max sum = 32 * 32767 = 1,048,544 -- well within INT32_MAX. For batch=256: max = 256 * 32767 = 8,388,352. Safe up to batch=65535 before INT32 overflow for 16-bit gradient mantissas.

3. Line-by-Line Explanation

`for (size_t j = 0; j < cols; j++) {`	`// Iterate each output (bias) dimension`
`float acc = 0.0f;`	`// Accumulator for column j`
`for (size_t i = 0; i < rows; i++)`	`// Sum over batch elements`
`acc += input[i*cols + j];`	`// Column-major: stride = cols between rows`
`output[j] = acc;`	`// Write bias gradient`
`}`	

4. Missing RigL Code

Reduce.c and Sum.c have no direct RigL gap. Important note: bias gradients should NOT be masked -- biases are always dense even in sparse Linear layers. Only weight gradients respect the RigL mask. The bias gradient column-sum runs on all loss values regardless of the weight mask.

Chapter 16

Conv1dKernel + ConvTranspose1dKernel

1D convolution kernels for audio/time-series

1. Usage of This File

Conv1dKernel.c implements the 1D sliding window correlation for the forward and backward passes of the Conv1d layer. ConvTranspose1dKernel.c implements the transposed (fractionally-strided) convolution. 1D convolution is the primary feature extractor for MFCC sequences in the LibriSpeech speaker ID model -- each conv layer extracts temporal patterns at a specific time scale.

Forward pass formula: output[b][c_out][t] = bias[c_out] + sum_c_in sum_k weight[c_out][c_in][k] * input[b][c_in][t*stride + k*dilation - pad]. The dilated convolution (dilation>1) increases the receptive field without increasing computational cost proportionally.

2. Interactions with Other Files and the STM32 MCU

Conv1dKernel.c is called by Conv1d.c via executeOp. On the STM32F7, the im2col approach converts convolution into a matrix multiply, leveraging Matmul.c optimisations. For a conv layer with inCh=40, outCh=64, kernLen=3, outLen=98 (MFCC 100-frame input): total MACs = 64 * 40 * 3 * 98 = 752,640. At 216 MHz: 3.5 ms per sample forward pass.

3. Line-by-Line Explanation

`for (size_t t = 0; t < outLen; t++) {`	`// Output time step`
`for (size_t co = 0; co < outCh; co++) {`	`// Output channel`
`float acc = bias ? biasData[co] : 0.0f;`	`// Load bias`
`for (size_t ci = 0; ci < inCh; ci++) {`	`// Input channel`
`for (size_t k = 0; k < kernLen; k++) {`	`// Kernel position`
`size_t inT = t*stride + k*dilation - pad;`	`// Input time with stride/dilation`
`if (inT >= inLen) continue;`	`// Zero-padding boundary`
`acc += inData[ci*inLen+inT] * wData[co*inCh*kernLen+ci*kernLen+k];`	`// MAC`
`}`	
`}`	
`outData[co*outLen+t] = acc;`	`// Write output`
`}`	
`}`	

4. Missing RigL Code

For RigL sparse convolution, the weight mask has shape [c_out, c_in, kernLen]. The flat mask index for weight[c_out][c_in][k] is c_out*inCh*kernLen + c_in*kernLen + k. Add mask check: if (wMask && !tensorBoolGet(wMask, co*inCh*kernLen+ci*kernLen+k)) continue. For typical small kernels (kernLen=3), structured channel sparsity (dropping entire c_in channels) is more hardware-friendly than element-level sparsity.

Chapter 17

PointwiseFused + RowBroadcast + SlidingWindow1d

Fused and broadcast operations

1. Usage

PointwiseFused.c fuses activation + quantisation in one pass. RowBroadcast.c broadcasts a vector across a matrix (LayerNorm gamma/beta). SlidingWindow1d.c extracts overlapping windows for im2col. These are performance primitives that reduce memory traffic by eliminating intermediate buffers.

2. Interactions

PointwiseFused is called from ExecuteOp when the output mode is OUT_WRITE and a fused activation is configured. RowBroadcast is called from LayerNorm. SlidingWindow1d is called from Conv1dKernel. All depend on DTypes.h and Tensor.h.

3. Line-by-Line Explanation

`/* PointwiseFused: ReLU + quantise in single pass */`	
`for (size_t i = 0; i < N; i++) {`	
`float raw = rawBuf[i];`	`// Read from scratch (one read)`
`float act = (raw > 0.0f) ? raw : 0.0f;`	`// ReLU activation`
`int32_t q = (int32_t)roundFn(act / scale);`	`// Quantise`
`q = (q > qMax) ? qMax : (q < 0) ? 0 : q;`	`// Clamp to ASYM range [0, qMax]`
`outData[i] = q;`	`// Write to output (one write)`
`}`	

4. Missing RigL Code

None of these utility files has a direct RigL gap. They operate on activations or structural tensor properties, not on weight parameters.

Chapter 18

Distributions + JacobiEig + Comparison + AdaptivePool

Specialised mathematical utilities

1. Usage

Distributions.c provides Gaussian PDF and CDF for PPCA sampling during continual learning replay. JacobiEig.c implements Jacobi eigendecomposition for PPCA covariance matrix eigenvectors. Comparison.c provides element-wise comparison (greater-than, less-than) for mask generation. AdaptiveAvgPool1d computes output sizes for adaptive pooling.

JacobiEig.c is the most algorithmically complex module: it repeatedly applies Givens rotations to zero off-diagonal elements of a symmetric matrix until convergence. For PPCA with rank r=8, the 8x8 matrix converges in about 15 sweeps (420 Givens rotations). Each rotation requires 4 multiplications and 2 additions of the affected rows/columns.

2. Interactions

Distributions.c and JacobiEig.c are called exclusively by PpcaReplay.c. Comparison.c is called by activation and loss functions. AdaptiveAvgPool1d is called by pooling layers. All depend on math.h (for expf, sqrtf, cosf, sinf).

3. Line-by-Line Explanation

Jacobi eigendecomposition (conceptual)

`while (offDiagNorm(A) > tolerance) {`	`// Iterate until convergence`
`for (size_t p=0; p<n; p++)`	`// Sweep over all pairs (p,q)`
`for (size_t q=p+1; q<n; q++) {`	
`theta = atan2(2*A[p][q], A[p][p]-A[q][q]) / 2;`	`// Rotation angle`
`applyGivensRotation(A, V, p, q, theta);`	`// Zero A[p][q] and A[q][p]`
`}`	
`}`	
`/* V now contains eigenvectors as columns */`	

4. Missing RigL Code

None of these utility modules has a direct RigL gap. They support PPCA continual learning, which is independent of the sparse weight training system.

Chapter 19

Layer.h + Layer.c

Layer abstraction: vtable dispatch for forward, backward, calcOutputShape

1. Usage of This File

Layer.h defines layer_t and the layerFunctions_t dispatch table. Every neural network layer is a layer_t with a type enum and config union. Layer.c populates the layerFunctions array, mapping each layerType_t to its forward, backward, and calcOutputShape function pointers -- a C vtable pattern.

Complete walkthrough: (1) Declare layer_t myLayer; (2) myLayer.type = LAYER_LINEAR; (3) Allocate and fill linearConfig_t linCfg; (4) myLayer.config.linear = &linCfg; (5) layerCalcOutputShape(&myLayer;, inputShape, outputShape) uses the dispatch table to call linearCalcOutputShape. (6) In training loop: layerForward(&myLayer;, input, output) dispatches to linearForward.

2. Interactions with Other Files and the STM32 MCU

Layer.c uses designated initialisers to build the static vtable at compile time. This enables dead-code elimination: if no model uses LAYER_CONV1D, the linker removes conv1dForward and conv1dBackward from the binary, saving flash. For the speaker ID model using only Linear, ReLU, and Softmax: only those six functions are linked.

The layerType_t enum values are part of the ODTS wire format (written to checkpoint files as uint8). Changing enum values or reordering the enum breaks checkpoint compatibility. Always add new layer types at the end of the enum.

3. Line-by-Line Explanation

`static const layerFunctions_t layerFunctions[] = {`	`// Static vtable at file scope`
`[LAYER_LINEAR] = { linearForward, linearBackward, linearCalcOutputShape },`	
`[LAYER_CONV1D] = { conv1dForward, conv1dBackward, conv1dCalcOutputShape },`	
`[LAYER_RELU] = { reluForward, reluBackward, reluCalcOutputShape },`	
`[LAYER_SOFTMAX] = { softmaxForward, softmaxBackward, softmaxCalcOutputShape },`	
`[LAYER_DROPOUT] = { dropoutForward, dropoutBackward, dropoutCalcOutputShape },`	
`};`	
`void layerForward(layer_t *l, tensor_t *in, tensor_t *out) {`	
`layerFunctions[l->type].forward(l, in, out);`	`// O(1) vtable dispatch`
`}`	

The designated initialiser [LAYER_RELU] = {...} ensures that even if LAYER_RELU has a non-zero enum value (e.g., 3), the array slot 3 is correctly populated. Any undefined slots are zero-initialised (NULL function pointers). Calling a NULL function pointer causes a HardFault on STM32. During development, add a bounds check and NULL check in layerForward.

4. Missing RigL Code

Layer.c has no direct RigL gap. The rigLStep() function (Chapter 39) iterates over a layer array, checks each layer type, and accesses the config union to find the weight mask. The existing layer abstraction provides exactly the right access pattern for this.

Chapter 20

Linear.h + Linear.c

Fully-connected layer: forward, backward, weight/bias gradients

1. Usage of This File

Linear.c implements the fully-connected layer: output = W * input + b. It is the most important layer in ODT for speaker ID. It supports FLOAT32 and SYM_INT32. The backward pass computes dL/dW (weight gradients), dL/db (bias gradients), and dL/dx (propagated loss for the previous layer). All three backward computations are matrix multiplications.

Full walkthrough from model init to gradient update: (1) linearInitConfig(&cfg;, inFeat=256, outFeat=128, arith=FLOAT32). (2) Xavier init weights: w ~ U(-sqrt(6/(256+128)), +sqrt(6/384)) = U(-0.125, +0.125). (3) Forward: output = input @ W^T + b. (4) Loss: CrossEntropy(output, label). (5) Backward: linearBackward computes dW, db, propLoss. (6) SGD: W -= lr * (dW + wd * W). Repeat for each training sample.

2. Interactions with Other Files and the STM32 MCU

Linear.c depends on Matmul.c (three matmuls per backward pass), Add.c (bias addition), ExecuteOp.c, and Layer.h. For the STM32F7 with 256 in, 128 out, batch=1: forward=32768 FMAs, dW=32768 FMAs, propLoss=32768 FMAs, biasGrad=128 adds. Total per sample: ~98688 operations = ~0.46 ms at 216 MHz.

3. Line-by-Line Explanation

Forward pass: output = input @ W^T + b

`transposeTensor(W, 0, 1);`	`// W: [out,in] -> [in,out] — O(1) index swap`
`matmulFloat32TensorsWithBias(input, W, output, b);`	`// output = input @ W^T + b`
`transposeTensor(W, 0, 1);`	`// Restore W to [out,in]`

Backward: weight gradients dW = loss^T @ input

`transposeTensor(loss, 0, 1);`	`// loss: [batch,out] -> [out,batch]`
`matmul(loss, input, dW);`	`// dW[out,in] = loss^T[out,batch] @ input[batch,in]`
`transposeTensor(loss, 0, 1);`	`// Restore loss`

Backward: propagated loss dX = loss @ W

`matmul(loss, W, dX);`	`// dX[batch,in] = loss[batch,out] @ W[out,in]`

Note: for dX, W is used without transposition because the matrix multiply loss[batch,out] @ W[out,in] naturally gives shape [batch,in], which is the correct propagated loss shape.

Backward: bias gradient db = columnSum(loss)

`columnWiseSum(loss, dBias);`	`// dBias[out] = sum over batch of loss[batch,out]`

4. Missing RigL Code and Implementation

■ RigL: Missing: weightMask in linearConfig_t	Without a weightMask field in linearConfig_t, neither the forward matmul nor the backward weight grad matmul can respect the RigL sparsity pattern. Active and inactive weights are treated identically.

`typedef struct linearConfig {`	
`parameter_t *weights;`	`// Weights + gradients`
`parameter_t *bias;`	`// Bias + gradient (optional)`
`tensor_t *weightMask;`	`// NEW: BOOL mask [out*in]; NULL = dense layer`
`arithmetic_t forwardMath;`	
`arithmetic_t weightGradMath;`	
`arithmetic_t propLossMath;`	
`arithmetic_t biasGradMath;`	
`} linearConfig_t;`	

With weightMask in linearConfig_t: (1) linearForward passes cfg->weightMask to matmulFloatCore as colMask -- the mask is indexed the same as the transposed weight matrix. (2) linearCalcWeightGrads passes cfg->weightMask to zero gradient entries for inactive weights. (3) sgdUpdateKernel receives cfg->weightMask and skips inactive weight updates. (4) rigLStep accesses cfg->weightMask directly.

The weightMask is NULL by default (dense layer). Sparse layers set weightMask after calling linearInitConfig. The NULL check in every function that receives the mask ensures backward compatibility: dense layers work without any code changes.

Chapter 21

Conv1d.h + Conv1d.c

1D Convolutional layer for audio/time-series data

1. Usage of This File

Conv1d.c implements the 1D convolutional layer for processing sequential data. It supports configurable in/out channels, kernel size, stride, padding, dilation, and groups. For MFCC speaker ID: a typical stack is Conv1d(40, 64, 3) -> ReLU -> Conv1d(64, 128, 3) -> GlobalAvgPool1d -> Linear(128, numSpeakers).

The forward pass loops over output time steps, output channels, input channels, and kernel positions -- a four-level nested loop. For outCh=64, inCh=40, kernLen=3, outLen=98: total inner loop iterations = 64*40*3*98 = 752,640 MACs. The backward pass for weight gradients requires correlating the input with the loss: another 752,640 MACs.

2. Interactions with Other Files and the STM32 MCU

Conv1d.c depends on Conv1dKernel.c, ExecuteOp.c, and Layer.h. The conv1dConfig_t struct stores pointers to weight and bias parameters, plus the convolution geometry (kernelSize, stride, pad, dilation, groups). Weight tensor shape: [outChannels, inChannels/groups, kernelSize]. The groups parameter enables depthwise convolution (groups=inChannels) for very efficient models.

3. Line-by-Line Explanation

`void conv1dForward(layer_t *layer, tensor_t *input, tensor_t *output) {`	
`conv1dConfig_t *cfg = layer->config->conv1d;`	`// Extract typed config from layer union`
`tensor_t *W = getParamFromParameter(cfg->weights);`	`// Weight tensor [out, in/g, k]`
`tensor_t *b = cfg->bias ? getParamFromParameter(cfg->bias) : NULL;`	`// Optional bias`
`executeOp(&(opSpec_t){`	`// Dispatch through ExecuteOp framework`
`.kernel = conv1dKernel,`	`// The actual sliding-window computation`
`.inputs = (tensor_t*[]){input, W, b},`	`// Input activations + weights + bias`
`.nInputs = b ? 3 : 2,`	`// Bias is optional`
`.arithmetic = cfg->forwardMath,`	`// FLOAT32 or SYM_INT32`
`.mode = OUT_WRITE,`	`// Overwrite output tensor`
`}, output);`	
`}`	

4. Missing RigL Code

Like Linear, Conv1d.c needs a weightMask field in conv1dConfig_t. For convolutions, structured channel sparsity is most hardware-friendly: entire input channels are dropped across all kernel positions. This eliminates the c_in dimension entirely for dropped channels, reducing inner loop iterations by inCh_dropped/inCh without the branch overhead of element-level masks.

Chapter 22

Conv1dTransposed.h + Conv1dTransposed.c

Transposed 1D convolution for upsampling

1. Usage of This File

Conv1dTransposed implements the transposed (fractionally-strided) convolution. While Conv1d reduces temporal resolution, Conv1dTransposed increases it -- it is the upsampling layer in encoder-decoder architectures. It may be used in a variational autoencoder for generating speaker embeddings in the continual learning system.

Output size formula: outLen = (inLen - 1) * stride - 2*pad + kernLen + outputPadding. For inLen=50, stride=2, pad=1, kernLen=3, outputPad=0: outLen = 49*2 - 2 + 3 = 99. This doubles the temporal resolution -- classic 2x upsampling.

2. Interactions with Other Files and the STM32 MCU

Conv1dTransposed.c depends on ConvTranspose1dKernel.c and ExecuteOp.c. The backward pass for transposed convolution is equivalent to the forward pass of a regular convolution with weight flipping, so Conv1dTransposed.backward internally calls conv1dKernel. The output buffer must be zero-initialised before the transposed convolution loop because it is a scatter-add operation (multiple input positions contribute to the same output position).

3. Line-by-Line Explanation

`/* Transposed conv: scatter-add over output */`	
`memset(outBuf, 0, outLen*outCh*4);`	`// Must zero output before scatter-add`
`for (size_t t = 0; t &lt; inLen; t++) {`	`// Iterate input time steps`
`for (size_t co = 0; co &lt; outCh; co++) {`	`// Output channel`
`for (size_t ci = 0; ci &lt; inCh; ci++) {`	`// Input channel`
`for (size_t k = 0; k &lt; kernLen; k++) {`	`// Kernel position`
`size_t outT = t * stride + k;`	`// Output time index (scatter)`
`out[co*outLen+outT] += W[ci*outCh*kernLen+co*kernLen+k] * in[ci*inLen+t];`	`// Scatter-add`
`}`	
`}`	
`}`	
`}`	

4. Missing RigL Code

Conv1dTransposed has the same RigL gap as Conv1d: no weight mask. Add a weightMask field to conv1dTransposedConfig_t and check it in the scatter-add inner loop.

Chapter 23

Relu + Softmax + Flatten + Dropout

Activation, normalisation, and regularisation layers

1. Usage of This File

ReLU applies max(0, x) element-wise -- computationally trivial and compatible with integer arithmetic. Softmax computes the normalised exponential distribution across the output dimension for the classification layer. Flatten reshapes a multi-dimensional activation tensor into a 1D vector without copying data. Dropout randomly zeroes activations during training to prevent overfitting.

ReLU is preferred over Sigmoid or Tanh for embedded training because it has no exponential computation. The derivative of ReLU is a step function (1 for x>0, 0 for x<=0), making the backward pass trivial: just gate the incoming gradient by the forward activation sign.

2. Interactions with Other Files and the STM32 MCU

Relu.c and Dropout.c call executeOp. Softmax.c calls findMaxFloat from MinMax.c (for numerical stability: subtract max before exp). Flatten.c calls setShape with numberOfDimensions=1 and dimensions=[totalElements] -- no data copy, just shape metadata update. On STM32F7, expf() for Softmax takes about 25 cycles per element. For 128-class Softmax: 128 * 25 = 3200 cycles = 15 us.

3. Line-by-Line Explanation

ReLU forward

`for (size_t i = 0; i < N; i++)`	
`out[i] = in[i] > 0.0f ? in[i] : 0.0f;`	`// ReLU: clamp negatives to zero`

ReLU backward

`for (size_t i = 0; i < N; i++)`	
`propLoss[i] = fwdInput[i] > 0.0f ? lossIn[i] : 0.0f;`	`// Gate gradient by forward sign`

Softmax forward (numerically stable)

`float mx = findMaxFloat(input->data, N);`	`// Subtract max for numerical stability`
`float sumExp = 0.0f;`	
`for (size_t i=0; i<N; i++) {`	
`float e = expf(input_f[i] - mx);`	`// expf(x - max) avoids overflow`
`output_f[i] = e;`	`// Temporarily store exp values`
`sumExp += e;`	`// Accumulate normalisation denominator`
`}`	
`for (size_t i=0; i<N; i++) output_f[i] /= sumExp;`	`// Normalise to probability distribution`

Subtracting max before exp prevents overflow. If logits are large (e.g., 1000.0), exp(1000) = +Inf. Subtracting max: exp(1000 - 1000) = exp(0) = 1. The normalised probability is still correct: exp(xi - max) / sum_j exp(xj - max) = exp(xi) / sum_j exp(xj).

4. Missing RigL Code

ReLU, Softmax, Flatten, and Dropout have no RigL gaps. They operate on activations, not weights. RigL masks apply exclusively to weight parameters in Linear and Conv1d layers.

Chapter 24

MaxPool1d + AvgPool1d + AdaptiveAvgPool1d

Pooling layers for downsampling

1. Usage of This File

Pooling layers reduce the temporal dimension of feature maps. MaxPool1d takes the maximum value in each window. AvgPool1d takes the mean. AdaptiveAvgPool1d computes window sizes automatically to produce a target output length (e.g., outLen=1 for global average pooling). These are used between conv layers to reduce sequence length and increase the receptive field without additional learnable parameters.

GlobalAvgPool1d (AdaptiveAvgPool1d with outLen=1) converts a [channels, seqLen] activation to a [channels, 1] vector. This is the final pooling step before the classification Linear layer in the speaker ID model -- it summarises the entire MFCC sequence into a fixed-length embedding vector.

2. Interactions with Other Files and the STM32 MCU

MaxPool1d.c stores the argmax during the forward pass so the backward pass can route the gradient to the correct input position. AvgPool1d.c backward distributes the gradient equally across all window elements. Both depend on SlidingWindow1d.c for window index computation.

3. Line-by-Line Explanation

`for (size_t c = 0; c < channels; c++) {`	`// Per channel (pooling is independent per channel)`
`for (size_t t = 0; t < outLen; t++) {`	`// Output time step`
`size_t start = t * stride;`	`// Window start`
`float mx = -HUGE_VALF;`	`// Init to -infinity`
`size_t argmx = start;`	`// Track argmax for backward`
`for (size_t k = 0; k < kernSize; k++) {`	`// Scan window`
`size_t inT = start + k;`	
`if (inT >= inLen) break;`	`// Boundary: stop at end of input`
`float v = inBuf[c*inLen+inT];`	
`if (v > mx) { mx = v; argmx = inT; }`	`// Track running max and its index`
`}`	
`outBuf[c*outLen+t] = mx;`	`// Write maximum value`
`argmaxBuf[c*outLen+t] = argmx;`	`// Save argmax for backward pass`
`}`	
`}`	

4. Missing RigL Code

Pooling layers have no RigL gap. They have no learned parameters -- they are purely deterministic downsampling operations.

Chapter 25

LayerNorm + GroupNorm + QuantizationLayer

Normalisation layers

1. Usage of This File

LayerNorm normalises activations across the feature dimension (zero mean, unit variance per sample per layer), then applies learned scale (gamma) and shift (beta). It stabilises training by reducing internal covariate shift. GroupNorm is similar but normalises within groups of channels, providing the same benefit for convolutional layers.

QuantizationLayer is a pass-through layer that explicitly converts a tensor from one quantisation format to another. It is used to control the precision at specific points in the computation graph -- for example, converting FLOAT32 activations to SYM_INT32 before a dense SYM_INT32 Linear layer.

2. Interactions with Other Files and the STM32 MCU

LayerNorm.c depends on Reduce.c (mean and variance computation) and RowBroadcast.c (applying gamma and beta). The gamma and beta tensors are parameter_t and receive gradient updates via standard backprop. Their gradients: dgamma = elementwiseMul(dout, x_norm), dbeta = columnSum(dout), dx_norm = dout * gamma.

3. Line-by-Line Explanation

`float mean = columnSum(input_f, D) / D;`	`// Mean across feature dimension D`
`float var = 0.0f;`	
`for (size_t i=0; i&lt;D; i++) {`	
`float d = input_f[i] - mean;`	`// Deviation from mean`
`var += d * d;`	`// Accumulate squared deviations`
`}`	
`var /= D;`	`// Variance = E[(x-mean)^2]`
`float istd = 1.0f / sqrtf(var + eps);`	`// Inverse std dev (eps for stability)`
`for (size_t i=0; i&lt;D; i++)`	
`out[i] = gamma[i]*(input_f[i]-mean)*istd + beta[i];`	`// Normalise + affine transform`

4. Missing RigL Code

LayerNorm and GroupNorm have no RigL gap -- their gamma and beta parameters are always dense. QuantizationLayer has no parameters at all.

Chapter 26

CrossEntropy + MSE

Loss functions for classification and regression

1. Usage of This File

CrossEntropy.c implements categorical cross-entropy loss: L = -log(p_true) where p_true is the predicted probability for the correct class. It is the standard loss for the speaker ID classification task. MSE.c implements mean squared error for regression tasks (autoencoder reconstruction, embedding distance).

The combined Softmax + CrossEntropy backward is particularly elegant: dL/dlogit_i = p_i - y_i, where p_i is the predicted probability and y_i is the one-hot target. For the correct class: grad = p_correct - 1 (push toward 1). For incorrect classes: grad = p_i (push toward 0). This simple form makes the backward pass extremely fast.

2. Interactions with Other Files and the STM32 MCU

CrossEntropy.c calls Softmax (if not already applied), then Log.c (for log-probabilities). It accesses the label as an INT32 index tensor. MSE.c calls Sub.c (to compute error = predicted - target) and Square.c (for squared error). Both are called from the training loop after the final layer forward pass.

3. Line-by-Line Explanation

CrossEntropy forward

`softmaxForward(logits, probs);`	`// Convert logits to probabilities`
`int label = labelTensor-&gt;data_i32[0];`	`// Integer class index`
`float p_correct = probs_f[label];`	`// Probability of correct class`
`float loss = -logf(p_correct + 1e-12f);`	`// Negative log-prob; eps prevents log(0)`

Combined Softmax+CrossEntropy backward

`for (size_t i=0; i&lt;numClasses; i++) {`	
`float y_i = (i == label) ? 1.0f : 0.0f;`	`// One-hot target for class i`
`grad[i] = probs[i] - y_i;`	`// p_i - y_i: elegant combined gradient`
`}`	

4. Missing RigL Code

Loss functions have no RigL gap. They operate on final layer activations and produce gradients that flow back through the network. The RigL mask affects how these gradients are used in the weight update step, not in loss computation.

Chapter 27

Optimizer.h + Optimizer.c

Optimizer base abstraction and dispatch

1. Usage of This File

Optimizer.h defines optimizer_t with a union of concrete implementations (sgd, adamw) and a step function pointer. Any training loop calls optimizerStep(optim) without knowing if it is SGD or AdamW. The optimizer holds a list of parameter_t pointers (all trainable parameters). optimizerStep() iterates this list and calls the concrete step kernel for each parameter.

The writeBackRounding field controls how updated parameters are rounded. It defaults to SR_HALF_AWAY for quantised parameters (stochastic rounding helps quantised weights escape the dead zone) and HALF_AWAY for FLOAT32 parameters (deterministic is preferred for reproducibility and debugging).

2. Interactions with Other Files and the STM32 MCU

Optimizer.h is included by the training loop, Sgd.c, AdamW.c, and LrScheduler.c. The optimizer parameter list is built during model init: optim.params[0] = &linearCfg.weights; optim.params[1] = &linearCfg.bias; etc. The order determines the update order, which affects reproducibility but not correctness.

For RigL integration: add a stepCount field and a rigLInterval field to optimizer_t. In optimizerStep(), after the parameter update loop, check if stepCount % rigLInterval == 0 and call rigLStep(model, numLayers, sparsity, stepCount, totalSteps). Then increment stepCount.

3. Line-by-Line Explanation

`typedef struct optimizer {`	
`optimizerImpl_t *impl;`	`// Union: impl->sgd or impl->adamw config`
`parameter_t **params;`	`// Array of all trainable parameters`
`size_t nParams;`	`// Number of parameters`
`roundingMode_t writeBackRounding;`	`// SR_HALF_AWAY for quantised, HALF_AWAY for float`
`void (*step)(struct optimizer *);`	`// Function pointer: sgdStep or adamwStep`
`} optimizer_t;`	
`void optimizerStep(optimizer_t *o) {`	
`o->step(o);`	`// Dispatch to concrete optimizer step function`
`}`	

4. Missing RigL Code

Optimizer.c has no direct RigL gap, but it is the natural integration point for the RigL topology update schedule. The training loop should not need to know about RigL -- the optimizer should handle it internally. Adding stepCount and rigLInterval to optimizer_t and calling rigLStep() inside optimizerStep() encapsulates the RigL logic cleanly.

```
/* Proposed RigL integration in optimizerStep */
void optimizerStep(optimizer_t *o) {
o-&gt;step(o);                                  // Update all parameters
o-&gt;stepCount++;                              // Increment step counter
if (o-&gt;rigLInterval &gt; 0 &&                 // If RigL is configured
o-&gt;stepCount % o-&gt;rigLInterval == 0) {     // And it is time for a topology update
rigLStep(o-&gt;model, o-&gt;numLayers,
o-&gt;sparsity, o-&gt;stepCount, o-&gt;totalSteps);  // Execute drop/grow
}
}
```

Chapter 28

Sgd.h + Sgd.c

Stochastic Gradient Descent with momentum and weight decay

1. Usage of This File

Sgd.c implements SGD with optional momentum and L2 weight decay. The plain SGD update: param -= lr * (grad + wd * param). With momentum: velocity = momentum*velocity + grad + wd*param; param -= lr*velocity. Weight decay (wd) is coupled to the gradient -- this is different from AdamW which uses decoupled weight decay.

SGD is preferred over AdamW for embedded training when memory is tight. Plain SGD needs only param and grad tensors. Momentum SGD adds one velocity tensor per parameter (same size as param). AdamW adds TWO moment tensors per parameter. For a model with 100K parameters: SGD extra cost = 0 tensors, momentum SGD = 400 KB, AdamW = 800 KB extra.

2. Interactions with Other Files and the STM32 MCU

Sgd.c depends on ExecuteOp.c, Axpby.c, Mul.c, and Sub.c. The sgdUpdateKernel is a static function passed to executeOp. The momentum state tensor is stored alongside the parameter and allocated at model init time.

3. Line-by-Line Explanation

Plain SGD kernel (no momentum)

```
static void sgdUpdateKernel(tensor_t **op, size_t n,
tensor_t *rawOut, tensor_t *aux, const void *ctxv) {
const sgdUpdateCtx_t *ctx = ctxv;                    // Contains lr and weightDecay
size_t N = calcNumberOfElementsByTensor(rawOut);
float *param = (float *)op[0]-&gt;data;              // Current weight values
float *grad = (float *)op[1]-&gt;data;               // Gradient from backward pass
float *out = (float *)rawOut-&gt;data;               // Output: updated weights
for (size_t i = 0; i &lt; N; i++) {                  // Iterate ALL weights (BUG for RigL!)
float g = grad[i] + ctx-&gt;weightDecay * param[i];  // L2-regularised gradient
out[i] = param[i] - ctx-&gt;lr * g;                  // SGD update step
}
}
```

■ WARNING sgdUpdateKernel updates ALL weights including inactive RigL weights. Inactive weights must remain exactly zero. Without a mask check, the optimizer moves inactive weights away from zero with every step, slowly corrupting the sparse topology and making the model dense again.

4. Missing RigL Code and Implementation

■ **RigL: Missing: mask check in sgdUpdateKernel**	Add a weightMask field to sgdUpdateCtx_t. If mask is non-NULL, skip inactive weights and force their value to 0.0.

```
typedef struct {
float lr, weightDecay;
tensor_t *weightMask;                    // NEW: NULL for dense, BOOL tensor for sparse
} sgdUpdateCtx_t;

/* Modified loop with mask check: */
for (size_t i = 0; i &lt; N; i++) {
if (ctx-&gt;weightMask &&                  // Mask provided (sparse layer)
!tensorBoolGet(ctx-&gt;weightMask, i)) {   // Check if weight is inactive (bit=0)
out[i] = 0.0f;                           // Force inactive weight to exactly zero
continue;                                // Skip gradient update for this weight
}
float g = grad[i] + ctx-&gt;weightDecay * param[i];   // L2 gradient for active weight
out[i] = param[i] - ctx-&gt;lr * g;                   // SGD step for active weight only
}
```

The out[i] = 0.0f line is essential. Even though inactive weights should already be zero in param[i], numerical noise or floating-point rounding from previous operations could have introduced tiny non-zero values. Hard-zeroing in the optimizer step guarantees the invariant: mask[i]==0 implies param[i]==0.0f exactly.

For momentum SGD, the velocity state for inactive weights should also be zeroed. An inactive weight that gets activated in the next GROW step should start with velocity=0 to avoid large initial updates from stale momentum accumulated when the weight was inactive.

Chapter 29

AdamW.h + AdamW.c

AdamW: Adam with decoupled weight decay

1. Usage of This File

AdamW implements Adam with decoupled weight decay (Loshchilov and Hutter, 2019). Standard Adam applies weight decay through the gradient (coupled: L2 regularisation). AdamW decouples it: param = (1 - lr*wd) * param - lr * adam_update. This produces better regularisation because the Adam moment estimates do not accumulate weight decay noise.

AdamW maintains two moment tensors per parameter: m (first moment, EMA of gradients) and v (second moment, EMA of squared gradients). The bias-corrected update: m_hat = m/(1-beta1^t), v_hat = v/(1-beta2^t), update = m_hat / (sqrt(v_hat) + eps). Typical hyperparameters: lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8, wd=0.01.

2. Interactions with Other Files and the STM32 MCU

AdamW.c uses double precision for beta1, beta2, and eps to avoid precision loss. 1-beta2 = 1-0.999 = 0.001, which has only 3 significant digits in float32 (float32 has ~7 decimal digits of precision). Using double ensures bc2 = 1 - 0.999^t is computed accurately for small t.

Memory cost of AdamW: for each parameter tensor of size N floats: param buffer = 4N bytes, grad buffer = 4N bytes, m buffer = 4N bytes, v buffer = 4N bytes. Total: 16N bytes per parameter. For a 256x128 Linear layer: 32768 * 16 = 524 KB per layer. On STM32F7 with 320 KB SRAM, this limits model size significantly. SGD (4N bytes overhead) is often preferred for memory-constrained training.

3. Line-by-Line Explanation

`double bc1 = 1.0 - pow(beta1, stepCount);`	`// Bias correction for first moment`
`double bc2 = 1.0 - pow(beta2, stepCount);`	`// Bias correction for second moment`
`/* Moment update: m = beta1*m + (1-beta1)*g */`	
`for (size_t i=0; i&lt;N; i++)`	
`m[i] = beta1*m[i] + (1.0f-beta1)*grad[i];`	`// EMA of gradient`
`/* Variance update: v = beta2*v + (1-beta2)*g^2 */`	
`for (size_t i=0; i&lt;N; i++)`	
`v[i] = beta2*v[i] + (1.0f-beta2)*grad[i]*grad[i];`	`// EMA of squared gradient`
`/* Parameter update with decoupled weight decay */`	
`for (size_t i=0; i&lt;N; i++) {`	
`float mh = m[i] / (float)bc1;`	`// Bias-corrected first moment`
`float vh = v[i] / (float)bc2;`	`// Bias-corrected second moment`
`param[i] *= (1.0f - lr*wd);`	`// Decoupled weight decay (shrink param)`

`param[i] -= lr * mh / (sqrtf(vh) + eps);`	`// Adam gradient step`
`}`	

The stepCount is NOT persisted across checkpoint saves/restores. When loading a checkpoint, stepCount resets to 1, which means bias correction restarts from the beginning. This causes a warmup period after each checkpoint load where the moment estimates are not yet converged. For continual learning on the STM32, this may cause brief performance dips after model restore.

4. Missing RigL Code

■ **RigL: Missing: mask checks in all three AdamW kernels**	Inactive weights must be skipped in moment update, variance update, AND parameter update. Stale moments accumulated for inactive weights corrupt the GROW decision: when a weight is activated, it should start with m=0 and v=0, not with stale estimates from when it was inactive.

`/* Modified moment kernel with mask */`	
`for (size_t i=0; i&lt;N; i++) {`	
`if (mask && !tensorBoolGet(mask, i)) {`	`// Inactive weight`
`m[i] = 0.0f; v[i] = 0.0f;`	`// Reset moments (no stale history)`
`param[i] = 0.0f;`	`// Enforce zero value`
`continue;`	
`}`	
`m[i] = beta1*m[i] + (1.0f-beta1)*grad[i];`	`// Update active weight moment`
`}`	

Chapter 30

LrScheduler.h + LrScheduler.c

Learning rate schedules: cosine annealing and step decay

1. Usage of This File

LrScheduler.c provides learning rate schedule functions called before each optimizer step to update the optimizer learning rate. Supported schedules: step decay (multiply lr by a factor every N steps), cosine annealing (smooth half-cosine from lr_max to lr_min), and constant (no change). The cosine schedule is recommended for RigL because both the LR and the drop fraction alpha use cosine decay.

Cosine annealing formula: lr(t) = lr_min + 0.5*(lr_max - lr_min)*(1 + cos(pi*t/T)). At t=0: lr=lr_max. At t=T: lr=lr_min. For on-device training with lr_max=0.01, lr_min=0.0001, T=10000 steps: the learning rate smoothly decays from 0.01 to 0.0001 following a half-cosine curve, avoiding the abrupt drops of step decay.

2. Interactions with Other Files and the STM32 MCU

LrScheduler.c modifies optimizer_t.impl->sgd.lr (or adamw.lr) directly. It depends on math.h for cosf. The training loop calls: lrSchedulerStep(&sched;, &optim;, stepCount) before optimizerStep(&optim;). For RigL, both the LR schedule and the drop fraction schedule (alpha cosine decay) use the same step counter, ensuring they are synchronised.

3. Line-by-Line Explanation

`float cosineAnnealingLR(float lrMax, float lrMin, size_t t, size_t T) {`	
`float r = (float)t / (float)T;`	`// Progress ratio: 0 at start, 1 at end`
`return lrMin + 0.5f*(lrMax-lrMin)*(1.0f + cosf(M_PI*r));`	`// Half-cosine decay`
`}`	

At t=0: r=0, cos(0)=1, result = lrMin + (lrMax-lrMin) = lrMax. At t=T/2: r=0.5, cos(pi/2)=0, result = lrMin + 0.5*(lrMax-lrMin) = midpoint. At t=T: r=1, cos(pi)=-1, result = lrMin + 0 = lrMin. The schedule is monotonically decreasing from lrMax to lrMin with most of the decay happening in the second half of training.

4. Missing RigL Code

LrScheduler.c has no direct RigL gap but provides the cosine annealing building block for RigL alpha decay. The RigL drop fraction alpha(t) = alpha_initial * 0.5*(1 + cos(pi*t/T)) follows the same formula with alpha_initial=0.3. Adding rigLSchedulerGetAlpha(t, T) returning this formula would complete the RigL schedule integration.

`float rigLAlpha(size_t t, size_t T) {`	`// RigL drop fraction schedule`
`float r = (float)t / (float)T;`	
`return 0.3f * 0.5f * (1.0f + cosf(M_PI * r));`	`// Cosine decay from 0.3 to 0`

```
}
```

Chapter 31

DataLoader.h + DataLoader.c + Dataset.h

Batch data loading and epoch shuffling

1. Usage of This File

DataLoader.c provides the data pipeline from .npy files to batched tensor inputs. It maintains a shuffled index array and serves one sample at a time (batch size 1 is the current constraint; dropLast must be true). At epoch start, rngShuffleIndices randomises the order. getNextBatch loads the next sample and returns true; returns false and re-shuffles at epoch end.

The batch-size-1 constraint exists because the STM32 has insufficient SRAM to buffer multiple samples simultaneously while also maintaining model weights and gradients. For the HAR dataset (128x9 input = 4608 bytes per sample) and LibriSpeech MFCC (40x100 = 16000 bytes per sample): single-sample processing is the only feasible approach.

2. Interactions with Other Files and the STM32 MCU

DataLoader.c depends on RNG.c (rngShuffleIndices), NPYLoader.c (sample reading), Dataset.h (dataset_t struct with file paths and shapes), and DataStorage.c (pre-allocated input/label buffers). On STM32, .npy files are on an SD card via FatFS. DMA transfers move data from SD card to SRAM in the background while the model processes the previous sample.

dropLast=false is not implemented: the DataLoader exits with PRINT_ERROR("dropLast must be true") and calls exit(1). This is because supporting variable-sized last batches would require dynamic memory allocation or more complex buffer management. For all current ODT datasets, numSamples is divisible by batchSize=1, so this is never triggered.

3. Line-by-Line Explanation

`void initDataLoader(dataLoader_t *dl, dataset_t *ds,`	
`size_t batchSize, bool dropLast) {`	
`if (!dropLast) { PRINT_ERROR("must be true"); exit(1); }`	`// Enforce constraint`
`dl->dataset = ds;`	
`dl->batchSize = batchSize;`	
`dl->currentIndex = 0;`	`// Start at beginning of dataset`
`for (size_t i=0; i<ds->numSamples; i++) dl->indices[i] = i;`	`// Identity permutation`
`rngShuffleIndices(dl->indices, ds->numSamples);`	`// Shuffle for first epoch`
`}`	

`bool getNextBatch(dataLoader_t *dl, tensor_t *in, tensor_t *lbl) {`	
`if (dl->currentIndex >= dl->dataset->numSamples) {`	`// End of epoch`

`rngShuffleIndices(dl->indices, dl->dataset->numSamples);`	`// Re-shuffle`
`dl->currentIndex = 0;`	`// Reset`
`return false;`	`// Signal: epoch complete`
`}`	
`size_t idx = dl->indices[dl->currentIndex++];`	`// Next shuffled sample index`
`npyLoaderGetSample(dl->npyLoader, idx, in);`	`// Load sample from .npy file`
`npyLoaderGetLabel(dl->npyLoader, idx, lbl);`	`// Load label`
`return true;`	`// Signal: batch ready`
`}`	

4. Missing RigL Code

DataLoader.c has no direct RigL gap. The data loading pipeline is unchanged for sparse training. Sparsity affects only the model parameters, not the data. However, for continual learning: when new speaker data arrives, the DataLoader must be re-initialised with the new dataset while the model retains its RigL mask from previous training.

Chapter 32

NPYLoader.h + NPYLoader.c

NumPy .npy binary format parser for embedded targets

1. Usage of This File

NPYLoader.c parses the NumPy binary .npy file format on the STM32. It reads the magic number, version, header size, and Python dict header string specifying dtype and shape. It maps each dataset sample to a byte offset in the file for random-access loading without loading the entire dataset into RAM.

The .npy format: 6-byte magic (0x93 + "NUMPY"), 2-byte version (major, minor), 2-byte (v1) or 4-byte (v2) header length, the header string (Python dict: {descr: dtype, fortran_order: False, shape: (N, H, W)}), padding to 64-byte alignment, then the raw data. Each element of the shape is a dataset sample when the first dimension is N=numSamples.

2. Interactions with Other Files and the STM32 MCU

NPYLoader.c depends on DTypes.h (byte reading) and FatFS (via stdio FILE wrappers). The header string parsing uses strstr() to find the descr and shape keys. This is fragile for complex headers (e.g., nested dicts or unusual whitespace) but sufficient for standard np.save() output.

On STM32, FILE is implemented via FatFS. fread() triggers DMA transfers from the SD card SDMMC peripheral. Seek operations (fseek) set the SD card read pointer, enabling random-access sample loading without loading the full dataset. The SD card read latency is about 100-500 us per seek + transfer for typical sample sizes.

3. Line-by-Line Explanation

Magic check

`bool checkMagic(FILE *f) {`	
`uint8_t buf[6];`	
`fread(buf, 1, 6, f);`	`// Read 6 bytes: 0x93 + "NUMPY"`
`return buf[0] == 0x93 &&`	`// 0x93 is non-ASCII guard byte (prevents text editors)`
`memcmp(&buf[1], "NUMPY", 5) == 0;`	`// Followed by ASCII "NUMPY"`
`}`	

The 0x93 byte serves as a non-ASCII guard. If someone accidentally opens an .npy file in a text editor and saves it, the editor would strip or corrupt non-ASCII bytes, making the file invalid. The magic check immediately detects such corruption. The Python byte literal b"\x93NUMPY" matches this pattern.

Header parsing

`char *descr = strstr(header, "'descr'");`	`// Find descr key in Python dict string`
`if (strstr(descr, "'&lt;f4'")) return FLOAT32;`	`// Little-endian float32 (most common)`

`if (strstr(descr, "'&lt;i4'")) return INT32;`	`// Little-endian int32`
`if (strstr(descr, "'&lt;i1'")) return INT8;`	`// Little-endian int8 (labels)`

Header size: v1 vs v2

`if (majorVersion == 1) {`	`// v1: header size is uint16 LE (max 65535 bytes)`
`uint8_t b[2]; fread(b,1,2,f);`	
`return readNumberOfBytesAsInt32(b, 2);`	`// Zero-extend u16 to u32`
`} else {`	`// v2+: header size is uint32 LE`
`uint8_t b[4]; fread(b,1,4,f);`	
`return readBytesAsInt32(b);`	
`}`	

Version 1 (most common): header size is uint16 LE, allowing up to 65535 bytes of header. This is sufficient for all practical array shapes. Version 2 (rare): header size is uint32 LE, supporting very large header strings (e.g., many-dimensional arrays with large shape tuples). NumPy uses v2 automatically when needed.

4. Missing RigL Code

NPYLoader.c has no direct RigL gap. However, the RigL mask tensors could be saved as .npy bool files and loaded at model init to restore a pre-computed sparsity pattern. This would allow the host computer to compute the initial topology (using PyTorch RigL) and transfer it to the STM32 via an SD card .npy file.

Chapter 33

PpcaReplay.h + PpcaReplay.c

Probabilistic PCA for continual learning generative replay

1. Usage of This File

PpcaReplay.c implements generative replay for continual learning. When the model learns a new speaker, it risks forgetting previously learned speakers (catastrophic forgetting). PPCA-based replay generates synthetic samples from old speakers using a probabilistic model, augmenting the new speaker training batch with these synthetic samples to preserve old knowledge.

PPCA (Probabilistic Principal Component Analysis) is a latent variable model: x = W*z + mu + noise, where z ~ N(0,I) is a low-dimensional latent code (rank-dimensional), W is the loading matrix (d x rank), mu is the mean, and noise ~ N(0, sigma^2 * I). Fitting PPCA to the speaker embeddings of an old speaker gives a compact generative model (W, mu, sigma) that can generate new synthetic samples approximately distributed like the original.

ppcaWorkspaceBytes computes the scratch memory needed for PPCA fitting: (p*d + p*p + p*p + p + rank + p + d + d + d) * sizeof(float), where p = rank + maxSessionSamples + 1. For rank=8 and maxSessionSamples=50: p=59. Workspace = (59*256 + 59*59 + 59*59 + 59 + 8 + 59 + 256 + 256 + 256) * 4 = approximately 130 KB. This must fit in available SRAM.

The only supported arithmetic mode is ARITH_FLOAT32. PPCA uses matrix operations (JacobiEig, SVD-like computations) that require full floating-point precision. Using SYM_INT32 for PPCA is not currently supported and would require significant numerical stability work.

2. Interactions with Other Files and the STM32 MCU

PpcaReplay.c depends on JacobiEig.c (eigendecomposition), Distributions.c (Gaussian sampling), RNG.c (noise generation), and DataStorage.c (workspace allocation). The PPCA model parameters (W, mu, sigma^2) are stored in the DataStorage pool. JacobiEig requires a square symmetric matrix -- for PPCA this is the sample covariance matrix of size rank x rank.

On STM32F7, PPCA fitting is the most compute-intensive operation in the continual learning pipeline. For p=59, d=256, rank=8: fitting requires computing a 59x256 data matrix, then a 59x59 covariance matrix (59^2*256 MACs = 893K operations), then JacobiEig on the 59x59 matrix (420 Givens rotations), then extracting the top-8 eigenvectors. Total: approximately 1-2 seconds at 216 MHz.

3. Line-by-Line Explanation

The PPCA fitting algorithm in PpcaReplay.c:

`/* Step 1: Compute sample mean */`	
`for (size_t d=0; d&lt;dims; d++) {`	`// For each feature dimension`
`mu[d] = 0.0f;`	
`for (size_t n=0; n&lt;numSamples; n++)`	

`mu[d] += samples[n*dims + d] / numSamples;`	`// Running mean`
`}`	
`/* Step 2: Centre the data */`	
`for (size_t n=0; n<numSamples; n++)`	
`for (size_t d=0; d<dims; d++)`	
`centred[n*dims+d] = samples[n*dims+d] - mu[d];`	`// X_c = X - mu`
`/* Step 3: Covariance matrix C = X_c^T @ X_c / (N-1) */`	
`matmulFloat32(centredT, centred, C, rank, numSamples, rank);`	`// rank x rank covariance`
`/* Step 4: Eigendecomposition of C */`	
`jacobiEig(C, rank, eigVecs, eigVals);`	`// Jacobi: C = V * diag(lambda) * V^T`
`/* Step 5: Extract loading matrix W = top-rank eigenvectors */`	
`extractTopKEigenvectors(eigVecs, eigVals, rank, W);`	`// W: dims x rank PPCA loading matrix`

The resulting PPCA model (mu, W, sigma^2) can generate synthetic samples: z ~ N(0,I_rank), x_synth = W*z + mu + N(0, sigma^2*I). These synthetic samples are added to the training batch when learning a new speaker, preventing catastrophic forgetting of old speakers.

4. Missing RigL Code

PpcaReplay.c has no direct RigL gap. Continual learning and sparse training are orthogonal concerns: PPCA replay addresses catastrophic forgetting, while RigL addresses computational efficiency. Both can be applied simultaneously -- the sparse model (RigL) learns new speakers while PPCA replay prevents forgetting old ones.

■ **TIP**	For continual RigL training: when adding a new speaker, run the PPCA replay and the RigL topology update independently. The replay affects which samples are in the training batch; the RigL update affects which weights are active. Neither interferes with the other.

Chapter 34

Serialize.h + Serialize.c

ODTS v2 checkpoint format: saving model parameters

1. Usage of This File

Serialize.c writes model parameters to a binary file in the ODTS (On-Device Training Serialisation) v2 format. The ODTS format is designed for embedded use: it is compact, sequential, and requires only one pass over the model parameters. It writes each layer type and config (quantisation, arithmetic mode) followed by the parameter data.

The ODTS v2 wire format: "ODTS" (4 ASCII bytes magic), uint32 version=2, uint32 layerCount, then for each layer: uint8 layerType tag, then the layer-specific payload. The layer payload for a Linear layer: uint32 inFeatures, uint32 outFeatures, uint8 arithType, float scale, int32 zeroPoint (for ASYM), then the raw weight mantissa bytes, then the raw bias bytes, then the sparsity section.

The sparsity section is currently a STUB: serializeSparsity() writes a single 0x00 tag byte indicating "no sparsity data". This must be replaced with real mask serialisation for RigL support.

2. Interactions with Other Files and the STM32 MCU

Serialize.c depends on Layer.h (to iterate the model layer array), DTypes.h (writeFloatToByteArray, writeInt32ToByteArray), and stdio.h (fwrite). On STM32, fwrite goes to an SD card via FatFS. The entire serialisation is sequential -- one fwrite call per field -- making it robust to SD card sequential write limitations.

Deserialize.c is the exact complement: it reads the same binary format and reconstructs the model parameters. Both files must be updated together when the format changes.

3. Line-by-Line Explanation

`void serializeModel(layer_t **model, size_t nLayers, FILE *f) {`	
`fwrite("ODTS", 1, 4, f);`	`// Magic: "ODTS" (4 bytes)`
`uint32_t ver = 2;`	
`fwrite(&ver, 4, 1, f);`	`// Version: 2 (uint32 LE)`
`uint32_t nL = (uint32_t)nLayers;`	
`fwrite(&nL, 4, 1, f);`	`// Layer count (uint32 LE)`
`for (size_t i=0; i<nLayers; i++) {`	
`uint8_t tag = (uint8_t)model[i]->type;`	`// Layer type as 1-byte enum value`
`fwrite(&tag, 1, 1, f);`	`// Write type byte`
`serializeLayerPayload(model[i], f);`	`// Write layer-specific data`
`serializeSparsity(model[i], f);`	`// Write sparsity mask (currently stub)`
`}`	
`}`	

serializeQConfig: asymmetric quantisation

```
void serializeQConfig(quantization_t *q, FILE *f) {
fwrite(&q-&gt;type, 1, 1, f);                          // qtype as uint8
if (q-&gt;type == ASYM) {                              // Asymmetric config
writeFloatToByteArray(q-&gt;asym-&gt;scale, buf);
fwrite(buf, 4, 1, f);                                 // scale: float32 LE
writeInt32ToByteArray(q-&gt;asym-&gt;zeroPoint, buf);
fwrite(buf, 4, 1, f);                                 // zeroPoint: int32 LE
fwrite(&q-&gt;asym-&gt;qBits, 1, 1, f);                 // qBits: uint8
} else if (q-&gt;type == SYM_INT32) {
writeFloatToByteArray(q-&gt;symInt32-&gt;scale, buf);
fwrite(buf, 4, 1, f);                                 // scale: float32 LE
}
}
```

4. Missing RigL Code and Implementation

■ RigL: Missing: serializeSparsity() is an empty stub	serializeSparsity() currently writes only a 0x00 tag byte. RigL masks are not saved. After a checkpoint load, the model has no sparsity information -- all weights are treated as dense, losing the sparse topology learned during RigL training.

```
void serializeSparsity(layer_t *layer, FILE *f) {
tensor_t *mask = NULL;
if (layer-&gt;type == LAYER_LINEAR)                    // Get mask from layer config
mask = layer-&gt;config-&gt;linear-&gt;weightMask;
if (mask == NULL || mask-&gt;quantization-&gt;type != BOOL) {
uint8_t tag = 0x00;                                   // Tag 0x00 = no sparsity data
fwrite(&tag, 1, 1, f);                                // Write null sparsity section
return;
}
uint8_t tag = 0x01;                                   // Tag 0x01 = BOOL mask follows
fwrite(&tag, 1, 1, f);
size_t n = calcNumberOfElementsByTensor(mask);        // Number of weights
size_t byteCount = (n + 7) / 8;                       // Packed bits: ceil(n/8)
fwrite(mask-&gt;data, 1, byteCount, f);                // Write packed BOOL mask bytes
}
```

The corresponding deserializeSparsity() function reads the tag byte: 0x00 means dense (no mask), 0x01 means a packed BOOL mask follows. It reads ceil(n/8) bytes and loads them into the pre-allocated mask tensor buffer. After deserialisation, the model has its full sparse topology restored, ready for continued RigL training.

Chapter 35

Deserialize.h + Deserialize.c

ODTS v2 checkpoint loading: restoring model parameters

1. Usage of This File

Deserialize.c reads the ODTS v2 binary format and restores model parameters. It is the exact complement of Serialize.c. The model structure (layer types, dimensions, quantisation configs) must already be initialised before calling deserializeModel -- the function only restores parameter values, it does not reconstruct the model topology.

This design (pre-init model, then load weights) is correct for embedded systems where the model architecture is fixed at compile time. The checkpoint file contains parameter values only; the architecture is encoded in the firmware itself. This prevents loading a checkpoint for a different architecture into the current model.

2. Interactions with Other Files and the STM32 MCU

Deserialize.c mirrors Serialize.c exactly in its dependency structure. It uses DTypes.h (readBytesAsFloat, readBytesAsInt32), Layer.h (layer type enum), and stdio.h (fread). The sequence is: check magic, check version, read layer count (must match model nLayers), for each layer: read type tag (must match model layer type), read payload into tensor buffers, read sparsity section.

3. Line-by-Line Explanation

`bool deserializeModel(layer_t **model, size_t nLayers, FILE *f) {`	
`char magic[4]; fread(magic, 1, 4, f);`	`// Read magic bytes`
`if (memcmp(magic, "ODTS", 4) != 0) return false;`	`// Validate magic`
`uint32_t ver; fread(&ver, 4, 1, f);`	`// Read version`
`if (ver != 2) return false;`	`// Only v2 supported`
`uint32_t nL; fread(&nL, 4, 1, f);`	`// Stored layer count`
`if (nL != nLayers) return false;`	`// Must match model`
`for (size_t i=0; i<nLayers; i++) {`	
`uint8_t tag; fread(&tag, 1, 1, f);`	`// Read layer type tag`
`if (tag != model[i]->type) return false;`	`// Validate matches model`
`deserializeLayerPayload(model[i], f);`	`// Load weights/bias`
`deserializeSparsity(model[i], f);`	`// Load mask (currently stub)`
`}`	
`return true;`	`// Success`
`}`	

4. Missing RigL Code

deserializeSparsity() is also currently a stub that reads and discards the sparsity tag byte. The full implementation should: read tag, if 0x01 allocate or reuse the mask tensor buffer and read ceil(n/8) bytes into it. See Chapter 34 for the serializeSparsity implementation -- the deserialize counterpart is symmetric.

Chapter 36

Training Loop Integration

Combining all modules: a complete training step on STM32

1. Overview of the Complete Training Loop

The complete ODT training loop on STM32 integrates all the modules described in previous chapters. This chapter walks through a complete training step, showing how data loading, forward pass, loss computation, backward pass, and optimizer update interact. This is the "glue code" that lives in main.c or a training module on the embedded device.

A single training step for the speaker ID model: (1) Load one MFCC sample and label from the DataLoader. (2) Forward pass through all layers. (3) Compute CrossEntropy loss. (4) Backward pass: compute loss gradient, propagate through all layers in reverse order. (5) Optimizer step: update all weights. (6) Optional RigL step: update sparse topology. (7) Optional PPCA replay: generate old speaker samples and repeat steps 2-6 for each.

2. Complete Training Step Code

```
void trainStep(model_t *m, dataLoader_t *dl, optimizer_t *opt) {
/* Step 1: Load data */
if (!getNextBatch(dl, m-&gt;input, m-&gt;label)) return; // Returns false at epoch end
/* Step 2: Forward pass */
for (size_t i=0; i&lt;m-&gt;nLayers; i++)
layerForward(m-&gt;layers[i], m-&gt;activations[i], m-&gt;activations[i+1]);
/* Step 3: Loss computation */
float loss = crossEntropyForward(m-&gt;activations[m-&gt;nLayers], m-&gt;label, m-&gt;lossGrad);
/* Step 4: Backward pass (reverse layer order) */
for (int i=m-&gt;nLayers-1; i&gt;=0; i--)
layerBackward(m-&gt;layers[i], m-&gt;lossGrads[i+1], m-&gt;lossGrads[i]);
/* Step 5: Optimizer step */
optimizerStep(opt); // Update all parameters + optional RigL
}
```

The activations array has nLayers+1 entries: activations[0] is the input tensor, activations[nLayers] is the final layer output. The lossGrads array also has nLayers+1 entries: lossGrads[nLayers] is the combined softmax+crossentropy gradient, lossGrads[0] is the gradient propagated to the input (not used, but computed).

3. Epoch Training Loop

`void trainEpoch(model_t *m, dataLoader_t *dl, optimizer_t *opt, size_t *step) {`	
`while (true) {`	
`bool hasData = getNextBatch(dl, m->input, m->label);`	
`if (!hasData) break;`	`// End of epoch: DataLoader reshuffled`
`lrSchedulerStep(&sched, opt, *step);`	`// Update learning rate`
`trainStep(m, dl, opt);`	`// One gradient step`
`(*step)++;`	`// Increment global step counter`
`}`	
`}`	

4. RigL Integration in Training Loop

■ RigL: RigL is triggered inside optimizerStep()	When opt->stepCount % opt->rigLInterval == 0, rigLStep() is called automatically. The training loop needs no explicit RigL logic -- it is encapsulated in the optimizer.

The only RigL-related setup required before training: (1) Allocate and initialise weight mask tensors for each sparse layer. (2) Set cfg->weightMask for each sparse linearConfig_t. (3) Set opt->rigLInterval to the desired topology update frequency (e.g., every 100 steps). (4) Set opt->sparsity to the target sparsity level (e.g., 0.9 for 90 percent). Then train normally.

Chapter 37

Memory Planning for RigL on STM32F746ZG

Calculating SRAM requirements for sparse training

1. STM32F746ZG Memory Resources

The STM32F746ZG has: DTCM RAM 64 KB (0x20000000), AXI SRAM 256 KB (0x20010000), BKPSRAM 4 KB (battery-backed), and FLASH 1 MB. For training: FLASH stores the firmware and read-only dataset (if small enough). AXI SRAM stores the model parameters, gradients, activations, and masks. DTCM is used for the stack, critical buffers, and the DataStorage pool.

2. Memory Breakdown for Speaker ID Model with RigL

Sample model architecture: Conv1d(40,64,3) -> ReLU -> Conv1d(64,64,3) -> GlobalAvgPool -> Linear(64,128) -> ReLU -> Linear(128,numSpeakers).

Memory costs (FLOAT32 storage, 90 percent sparse Linear layers):

- Conv1d(40,64,3) weights: 40*64*3*4 = 30,720 bytes (dense conv)
- Conv1d(40,64,3) gradients: 30,720 bytes
- Conv1d(64,64,3) weights: 64*64*3*4 = 49,152 bytes (dense conv)
- Conv1d(64,64,3) gradients: 49,152 bytes
- Linear(64,128) weights: 64*128*4 = 32,768 bytes; gradient: 32,768 bytes; MASK: 1,024 bytes
- Linear(128,numSpeakers=100) weights: 128*100*4 = 51,200 bytes; grad: 51,200 bytes; MASK: 1,600 bytes
- AdamW moments (m+v) for all dense params: 2*(30720+30720+49152+49152) = 319,488 bytes
- AdamW moments for sparse Linear params (active only): 2*(32768+51200)*0.1 = 16,794 bytes
- Activations (forward pass buffers): approximately 80,000 bytes
- Total: approximately 727 KB -- exceeds 320 KB AXI SRAM!

This analysis reveals that AdamW is too expensive for this model on the STM32F746ZG. Using SGD without momentum reduces memory by 319,488 bytes, bringing the total to about 407 KB -- still over budget. Solution: use smaller model (Linear(64,64) instead of Linear(64,128)), or run Conv layers in FLOAT32 with SGD and Linear layers with SYM_INT32 (saves 50 percent on Linear weight memory).

3. Memory-Optimised Configuration

Recommended memory-efficient configuration for STM32F746ZG: (1) Reduce model: Linear(64,64) -> Linear(64,numSpeakers). (2) Use SYM_INT32 for Linear weights (saves 50% vs FLOAT32 -- quantised to 12-bit but stored as int32). Actually same byte count but enables SIMD optimisations. (3) Use SGD (no momentum) for Conv1d, momentum-SGD for Linear. (4) RigL at 90% for both Linear layers. (5) Store PPCA

workspace in external QSPI flash (not needed during training).

4. RigL Memory Savings

With 90% sparse Linear layers and SGD (no momentum state): Linear(64,64) mask = 512 bytes (vs 16,384 bytes for dense float weights). Linear(64,numSpeakers=100) mask = 800 bytes. Total mask overhead: 1,312 bytes. Active weight compute: 10% of dense cost. The mask overhead is negligible; the compute savings are substantial.

Chapter 38

Debugging and Testing the ODT Library on STM32

Tools, strategies, and common failure modes

1. Debug Build Configuration

Build with -DDEBUG_MODE_DEBUG (DLEVEL=3) to enable all PRINT_DEBUG logging. Connect the STM32 Nucleo via SWD to STM32CubeIDE and enable ITM stimulus port 0 for printf output. Set a breakpoint in RIGL_ASSERT_MASK to catch mask/weight inconsistencies immediately.

Most common failure modes: (1) Mask and weight out of sync -- inactive weights have non-zero values after an optimizer step. Caused by missing mask check in sgdUpdateKernel. (2) Wrong matrix dimension -- transposing the wrong dimensions before matmul causes shape mismatch. (3) Scale not calibrated -- weight scale is 0.0f because initSymInt32QConfig was called but scale was never set. (4) DataStorage overflow -- STORAGE_POOL_SIZE too small for all tensors including RigL masks.

2. Unit Testing on Host

The ODT library can be compiled for the host (Linux/macOS/Windows) using the standard build system. Unit tests in tests/ use the same library code, comparing outputs against PyTorch reference implementations. For RigL: generate a sparse mask in PyTorch, export as .npy bool file, load in ODT, run one training step, compare weights and gradients against PyTorch.

Key tests to add for RigL: (1) Test that sgdUpdateKernel zeroes inactive weights after update. (2) Test that matmulFloatCore gives the same result as dense matmul for a known mask (since inactive weights are zero, the results should be identical). (3) Test that findAbsKthSmallestActive returns the correct threshold for a known weight vector. (4) Test that rigLStep preserves the exact target sparsity level.

3. Common Failure Modes and Fixes

■ **WARNING**	HardFault on STM32: most commonly caused by DataStorage overflow (accessing memory beyond STORAGE_POOL_SIZE), null function pointer call (undefined layerType in dispatch table), or unaligned float access (using raw pointer cast instead of memcpy in DTypes functions).
■ **WARNING**	Gradient explosion: if SYM_INT32 operands exceed 12-bit range (scale not calibrated correctly), the matmul accumulator overflows int32, producing garbage gradients. Check that calibrateScale is called after every weight initialisation.
■ **TIP**	To debug RigL mask issues: add PRINT_DEBUG calls before and after rigLStep showing numActive, K, dropThresh, growThresh. If numActive does not change by exactly 0 (dropped K, grew K), there is a tie-breaking or boundary condition bug.

4. Performance Profiling on STM32

Use the DWT (Data Watchpoint and Trace) cycle counter on Cortex-M7 for cycle-accurate timing. Enable with: CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk; DWT->CYCCNT = 0; DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;. Measure: uint32_t t0 = DWT->CYCCNT; operation; uint32_t elapsed = DWT->CYCCNT - t0;. At 216 MHz: 1 cycle = 4.63 ns. Profile matmulFloatCore, rigLStep, and the full training step to identify bottlenecks.

Chapter 39

RigL Integration: The Complete rigLStep() Implementation

Putting it all together: drop, grow, and schedule

1. Overview of rigLStep()

rigLStep() is the central RigL function that ties together all the missing pieces documented in previous chapters. It takes the model (array of layers), current training step, total training steps, and target sparsity. It iterates all sparse Linear layers, computes the drop threshold and grow threshold using the K-th element functions from MinMax.c, then applies the DROP and GROW operations to update the weight mask.

The function is called from inside optimizerStep() every rigLInterval steps. The cosine decay schedule ensures that alpha (the fraction of active weights to swap) starts at 0.3 (30 percent of active weights) and decays to 0 by the end of training, giving the topology time to stabilise before convergence.

Complete walkthrough: suppose we have a Linear(256, 128) layer with 90 percent sparsity: 32768 total weights, 3277 active. At step 100 of 10000 total steps with rigLInterval=100: alpha = 0.3 * 0.5 * (1 + cos(pi*100/10000)) = 0.3 * 0.5 * (1 + 0.9995) = 0.2999. K = round(0.2999 * 3277) = 983 weights to swap.

Step 1: DROP. Find the 983 smallest |w| among 3277 active weights. Any active weight with |w| <= dropThresh is deactivated: mask bit set to 0, weight value set to 0.0. Active count becomes 3277 - 983 = 2294.

Step 2: GROW. Find the 983 largest |grad| among 29491 inactive weights. Any inactive weight with |grad| >= growThresh is activated: mask bit set to 1, weight value initialised to 0.0. Active count becomes 2294 + 983 = 3277 again. Sparsity = (32768 - 3277) / 32768 = 90.0 percent. Preserved.

2. Complete rigLStep() Implementation

```
#include "MinMax.h"
#include "Layer.h"
#include "Tensor.h"
#include &lt;math.h&gt;

void rigLStep(layer_t **model, size_t numLayers,
float sparsity, size_t step, size_t totalSteps) {
/* Cosine-decayed drop fraction: 0.3 -&gt; 0 */
float cosDecay = 0.5f * (1.0f + cosf((float)M_PI * step / totalSteps));
float alpha = 0.3f * cosDecay;                  // Drop fraction this step
if (alpha &lt; 1e-6f) return;                   // Near end of training: stop swapping

for (size_t l=0; l&lt;numLayers; l++) {          // Iterate all layers
if (model[l]-&gt;type != LAYER_LINEAR) continue; // Only Linear has weight mask
linearConfig_t *cfg = model[l]-&gt;config-&gt;linear;
```

if (cfg->weightMask == NULL) continue;	// Skip dense layers
tensor_t *weights = cfg->weights->param;	// Weight tensor
tensor_t *grads = cfg->weights->grad;	// Gradient tensor
tensor_t *mask = cfg->weightMask;	// BOOL mask tensor
size_t N = calcNumberOfElementsByTensor(weights);	// Total weights
size_t numActive = 0;	
for (size_t i=0; i<N; i++)	
numActive += tensorBoolGet(mask, i);	// Count active weights
size_t K = (size_t)(alpha * numActive);	// Number of weights to swap
if (K == 0) continue;	// Nothing to swap this step
/* DROP: deactivate K smallest |w| among active */	
float dropThresh = findAbsKthSmallestActive(weights, mask, K);	
float *w = (float *)weights->data;	
size_t dropped = 0;	
for (size_t i=0; i<N && dropped<K; i++) {	
if (tensorBoolGet(mask,i) && fabsf(w[i]) <= dropThresh) {	
tensorBoolSet(mask, i, false);	// Deactivate: clear mask bit
w[i] = 0.0f;	// Hard-zero the weight
dropped++;	
}	
}	
/* GROW: activate K largest |grad| among inactive */	
float growThresh = findAbsKthLargestInactive(grads, mask, K);	
float *g = (float *)grads->data;	
size_t grown = 0;	
for (size_t i=0; i<N && grown<K; i++) {	
if (!tensorBoolGet(mask,i) && fabsf(g[i]) >= growThresh) {	
tensorBoolSet(mask, i, true);	// Activate: set mask bit
w[i] = 0.0f;	// Init new connection to zero
grown++;	
}	
}	
}	
}	

3. Line-by-Line Explanation of rigLStep()

Line: float cosDecay = 0.5f * (1.0f + cosf(pi * step / totalSteps)). This is the cosine decay factor ranging from 1.0 at step=0 to 0.0 at step=totalSteps. At step=0: cos(0)=1, cosDecay=1. At step=T/2: cos(pi/2)=0, cosDecay=0.5. At step=T: cos(pi)=-1, cosDecay=0. Multiplied by 0.3 (the initial drop fraction), alpha ranges from 0.3 to 0.

Line: if (alpha < 1e-6f) return. When alpha is negligibly small (near end of training), skip the topology update entirely. This avoids K=0 corner cases and unnecessary scanning of all weights for no effect.

Line: size_t K = (size_t)(alpha * numActive). K is the number of active weights to drop (and the number of inactive weights to grow). For alpha=0.3 and numActive=3277: K=983. The cast truncates the float to an integer, which is fine -- the exact K value matters less than maintaining the approximate target sparsity.

Line: for (size_t i=0; i<N && dropped<K; i++). The early exit (dropped<K) stops the drop loop as soon as exactly K weights have been dropped, even if more weights meet the threshold condition (ties at dropThresh). This prevents over-dropping, which would reduce the sparsity level below the target.

Line: w[i] = 0.0f after tensorBoolSet(mask, i, false). This hard-zeroing is essential. Even though the matmul kernel will skip this weight due to the mask, an explicit zero ensures that: (1) RIGL_ASSERT_MASK passes its invariant check, (2) the weight does not accidentally influence computation if the mask check is missing somewhere, and (3) checkpoint files have clean zeros for inactive weights.

Line: w[i] = 0.0f after tensorBoolSet(mask, i, true) in the GROW step. New connections are zero-initialised per the original RigL paper. Zero initialisation gives smaller initial gradients and more stable training than random initialisation. The connection will grow from zero following its gradient signal, which is already available from the phantom gradient computed during the previous backward pass.

4. Integration Checklist

Before rigLStep() can function correctly, all of the following must be in place:

- MinMax.c: findAbsKthSmallestActive() and findAbsKthLargestInactive() implemented (Chapter 12)
- Tensor.h: tensorBoolGet() and tensorBoolSet() available (Chapter 5)
- Linear.h: weightMask field added to linearConfig_t (Chapter 20)
- Matmul.c: mask check in matmulFloatCore inner loop (Chapter 13)
- Sgd.c: mask check in sgdUpdateKernel (Chapter 28)
- AdamW.c: mask checks in all three kernels (Chapter 29)
- Serialize.c: serializeSparsity() fully implemented (Chapter 34)
- Deserialize.c: deserializeSparsity() fully implemented (Chapter 35)
- DataStorage.c: mask buffers reserved during model init (Chapter 7)
- Optimizer.c: rigLStep() call integrated with step counter (Chapter 27)

■ **RigL: Implementation Order**	Implement in this order: (1) parameter_t.mask field, (2) DataStorage mask allocation, (3) sgdUpdateKernel mask check, (4) matmulFloatCore mask check, (5) findAbsKthSmallest/Largest, (6) rigLStep(), (7) serializeSparsity. Test after each step.

5. Expected Accuracy Results

Based on the original RigL paper (Evci et al. 2020) and the ODT project context: for the LibriSpeech speaker ID task with a small model (128K parameters), RigL at 80 percent sparsity should achieve within 2-3 percent of dense model accuracy. At 90 percent sparsity: within 5-8 percent. The exact degradation depends on the model architecture, learning rate schedule, and the specific speaker ID dataset used.

For the HAR dataset (simpler task, fewer classes): RigL at 90 percent sparsity typically achieves within 1-2 percent of dense model accuracy because the task is easier and the required network capacity is lower. This makes HAR a good validation target for the RigL implementation before deploying on the more challenging speaker ID task.

Computational savings from RigL at 90 percent sparsity: forward pass MACs reduced by approximately 9x. Backward weight gradient MACs reduced by 9x (gradients for inactive weights are zero, so they can be

skipped). Total training time per step: approximately 3-4x faster than dense (not 10x, because batch overhead, mask checking, and rigLStep itself add cost). Memory for weights: same (the weights array still has the full size, but 90 percent of entries are zero). Memory for the mask: negligible (ceil(N/8) bytes).

Deep Dive: The FQT Mathematical Framework

Quantisation Error Analysis

Quantisation error is the difference between the real value and its quantised approximation. For symmetric quantisation with scale s and qMaxBits b: the maximum absolute error is s/2 (half a quantisation step). The relative error is (s/2) / |real| = 1 / (2 * mantissa). For a weight of 0.05 with scale=0.000244: mantissa=205, relative error = 1/(2*205) = 0.24 percent. For a weight near zero (mantissa=1): relative error = 50 percent. This is why quantisation hurts small weights most -- a known limitation of uniform quantisation.

Signal-to-quantisation-noise ratio (SQNR) for uniform quantisation: SQNR = 6.02 * b + 1.76 dB, where b is the bit width. For b=12 bits: SQNR = 74 dB. For b=8 bits: SQNR = 50 dB. For b=4 bits: SQNR = 26 dB. This logarithmic relationship means each extra bit adds about 6 dB of SQNR -- doubling the precision reduces quantisation noise by 4x in power.

Stochastic rounding modifies this analysis: with SR_HALF_AWAY, the quantisation error is a uniform random variable in [-s/2, s/2) with mean zero. The SQNR is the same in expectation, but the error is now unbiased. For training, unbiased error is crucial: a systematic bias of s/4 per weight would shift the loss landscape and prevent convergence to the true optimum.

■ **CONCEPT**	Quantisation Dead Zone: a weight whose true gradient is smaller than s/2 cannot escape quantisation level 0 with deterministic rounding. With SR_HALF_AWAY, the weight has probability |grad|/s of moving to the adjacent quantisation level each step. After 1/p steps on average, it escapes. This is why SR is essential for RigL-grown weights (start at 0, small gradients initially).

The FQT training pipeline preserves most accuracy because: (1) the loss landscape is relatively flat near the optimum -- small quantisation errors do not change the gradient direction significantly; (2) stochastic gradient descent is itself noisy (one-sample gradient estimate), and quantisation noise is small compared to SGD noise; (3) the network learns to be robust to quantisation by distributing information across many weights rather than concentrating it in a few precise weights.

FQT vs post-training quantisation (PTQ): PTQ trains in float32 then quantises. FQT quantises during training. FQT typically outperforms PTQ by 1-3 percent accuracy because: (a) the model learns to compensate for quantisation during training, (b) the stochastic gradient updates average out quantisation noise, (c) the learning rate can be tuned for the quantised representation. On STM32 targets, FQT is strongly preferred because the inference model is already in the quantised format -- no post-training calibration step needed.

The RigL Algorithm: Formal Description

Let theta be the model parameters (weight matrix W for a Linear layer). Let M be the binary mask (M_ij = 1 if weight ij is active, 0 if inactive). Let s = target sparsity (fraction of zero weights). Constraints: sum(1 - M_ij) / total = s.

The RigL training objective: minimise L(theta * M) subject to ||M||_0 / total = 1-s, where * denotes element-wise multiplication and L is the cross-entropy loss. The constraint fixes the number of active connections throughout training.

RigL approximately solves this by: (1) Gradient descent step: theta = theta - lr * grad(L, theta * M) (only active weights updated). (2) Every T steps: UPDATE MASK. DROP: M_ij = 0 for the K active connections with smallest |theta_ij|. GROW: M_ij = 1 for the K inactive connections with largest |dL/dtheta_ij|. Where K =

alpha(t) * sum(M_ij) and alpha(t) is a cosine-decayed fraction.

The intuition: active weights with small magnitude are likely to have little effect on the output (they are "almost pruned" anyway), so dropping them is safe. Inactive weights with large gradient magnitude are connections the network "wants" to use but cannot because they are masked out -- growing them allows the network to explore more useful topologies.

■ **RigL: Gradient vs Weight for Grow**	Why use gradient magnitude for GROW instead of weight magnitude? Because inactive weights are always zero (|w|=0 for all of them), so weight magnitude cannot distinguish them. The gradient dL/dw = dL/dy * x is non-zero for inactive weights because x (the input) is non-zero even when w=0. The gradient tells us how much the loss would decrease if this connection were activated.

The drop fraction alpha(t) = alpha_init * 0.5 * (1 + cos(pi * t / T)) decays from alpha_init=0.3 to 0 over the training run. This annealing serves two purposes: (1) early training benefits from aggressive topology exploration (high alpha); (2) late training should stabilise the topology (low alpha) to allow the final sparse network to converge to its optimum.

Comparison of sparse training algorithms on the STM32: Static pruning (train dense, prune once): simplest to implement, worst accuracy. Iterative pruning (prune gradually during training): better accuracy, requires float checkpoints. SET (Sparse Evolutionary Training): random regrowth, moderate accuracy. RigL (gradient-guided regrowth): best accuracy, moderate implementation complexity. For on-device training, RigL's gradient-guided approach is crucial because we cannot afford to explore random topologies -- each training step is expensive.

STM32 Nucleo-F746ZG Hardware Architecture

The STM32 Nucleo-F746ZG evaluation board features the STM32F746ZGT6 microcontroller. Key specifications relevant to ODT: ARM Cortex-M7 CPU at up to 216 MHz, single-precision FPU with 32 single-precision registers, DSP SIMD extensions, Thumb-2 instruction set, 1 MB Flash memory, 320 KB SRAM (64 KB DTCM + 256 KB AXI SRAM), hardware TRNG, SD card slot (SDMMC1 interface), UART, SPI, I2C, USB OTG HS.

The Cortex-M7 instruction pipeline is a 6-stage superscalar pipeline with branch prediction, out-of-order execution for some instructions, and a 32-byte instruction prefetch queue. The FPU is a single-precision VFPv5 unit with 5-cycle latency but 1-cycle throughput for most operations when properly pipelined. The L1 cache is 32 KB I-cache and 32 KB D-cache with 4-way set associativity and 64-byte cache lines.

Cortex-M7 FPU Throughput	VMLA.F32 (fused multiply-add): 1-cycle throughput, 5-cycle latency. VSQRT.F32: 14 cycles. VDIV.F32: 14 cycles. VABS.F32: 1 cycle. VCMP.F32: 1 cycle. Pipeline them: maintain 5 independent FMAs in flight to achieve peak throughput. Manual loop unrolling by 4-8 achieves near-peak on matmul inner loops.

Memory architecture: DTCM (Data Tightly Coupled Memory) at 0x20000000 is directly connected to the Cortex-M7 data bus with 0 wait states. All stack memory and critical hot-path buffers should reside here. AXI SRAM at 0x20010000 goes through the AXI matrix and has 1-2 wait states. Flash at 0x08000000 uses an ART accelerator (Adaptive Real-Time accelerator) to achieve 0 wait states at up to 216 MHz when running from ITCM cache.

DMA usage for training: the STM32F7 has a multi-layer AHB bus matrix allowing DMA transfers and CPU memory access to occur simultaneously. When loading training samples from SD card, use DMA2 Stream 6 (SDMMC1 RX) to transfer data to AXI SRAM while the CPU processes the previous sample. This overlaps IO latency with computation, reducing effective sample loading time from 500 us to near-zero.

Numerical Precision in On-Device Training

Float32 precision: 24-bit mantissa gives approximately 7 decimal digits of precision. For neural network training, this is generally sufficient -- the stochastic gradient noise from single-sample estimates is much larger than float32 rounding errors. However, when accumulating many small gradient updates (e.g., weight decay over thousands of steps), float32 error can accumulate.

SYM_INT32 with 12-bit operands: the product of two 12-bit integers is a 24-bit integer, representable exactly in float32 and int32. The sum of K such products is exact in int32 as long as K * 2^24 does not exceed INT32_MAX (2^31 - 1): K < 2^7 = 128. For K=256 columns (common in speaker ID models), the sum can reach 256 * 2047^2 = 1.07 billion, which is within INT32_MAX (2.15 billion). Safe.

Gradient underflow: for 8-bit quantised gradients (SYM with qMaxBits=8), the smallest representable non-zero gradient magnitude is scale = absMax / 127. If a weight gradient is 0.001 and the maximum gradient is 10.0 (absMax=10), then scale=10/127=0.0787. The gradient 0.001/0.0787 = 0.013 rounds to 0 with deterministic rounding -- the weight receives no update. With stochastic rounding, probability of update = 0.013/1 = 1.3% per step. After 77 steps on average, the weight receives one gradient step. This explains why 8-bit gradient quantisation requires stochastic rounding to avoid dead weights.

Double precision for AdamW: beta1=0.9 and beta2=0.999 are stored as double (64-bit). The bias correction involves computing 1-beta1^t. At t=1000: 1-0.9^1000 ≈ 1-2.66e-46 ≈ 1.0 in float32 (underflow). In double: 1-0.9^1000 = 0.9999999...99734 -- representable accurately. Without double, the bias correction collapses to 1.0 at high step counts, effectively disabling it. This is why AdamW.c stores beta1, beta2, and eps as double even though weights are float32.

Deep Dive: Tensor Memory Layout and the Zero-Copy Transpose

Row-Major (C Order) Memory Layout

ODT tensors use row-major (C order) memory layout. For a 2D tensor of shape [rows, cols]: element at (i, j) is at byte offset (i*cols + j) * bytesPerElement. For a 3D tensor of shape [batch, channels, length]: element at (b, c, t) is at byte offset (b*channels*length + c*length + t) * bytesPerElement. This layout is natural for C arrays and is also the default for numpy (C-contiguous arrays).

Fortran order (column-major): element at (i, j) is at byte offset (j*rows + i) * bytesPerElement. NumPy .npy files indicate order via the fortran_order field in the header (always False for ODT-generated files). The NPYLoader checks this field and would need to transpose if fortran_order=True -- currently this path is not implemented.

The transposeTensor function swaps the orderOfDimensions array rather than moving data. For the linear layer forward pass with W of shape [outFeat, inFeat]: physical layout is row-major with outFeat rows and inFeat columns. After transposeTensor(W, 0, 1): logical shape becomes [inFeat, outFeat], orderOfDimensions={1,0}. The matmul function accesses W using the logical indices mapped through orderOfDimensions, effectively reading the matrix in column-major order -- which is equivalent to reading the transpose in row-major order.

The zero-copy transpose is critical for performance: for W of shape [256, 128], a data copy would require moving 32768 * 4 = 131 KB of data. The index swap costs only 4 instructions. Linear.c calls transposeTensor twice per forward pass (before and after matmul) -- 8 instructions versus 262 KB of data movement. On STM32 with memory bandwidth of about 800 MB/s (AXI SRAM, burst mode): copying 131 KB takes about 160 us. The index swap takes 4 cycles = 0.02 us. Speedup: 8000x for the transpose operation alone.

Bit-Packed BOOL Tensor Operations

BOOL tensors pack 8 mask bits into each byte. For N=32768 weights: 32768/8 = 4096 bytes for the mask. Element i is stored in byte i/8, bit position i%8 (LSB first). Byte 0 stores elements 0-7, byte 1 stores elements 8-15, etc.

Performance of tensorBoolGet/Set in hot loops: each call involves one integer division by 8 (>> 3 with compiler optimisation), one modulo by 8 (& 7), one array read, one shift, and one AND. On Cortex-M7: approximately 4 cycles per call. For 32768 weights: 32768 * 4 = 131,072 cycles = 0.6 ms at 216 MHz just for mask scanning. This is why the rigLStep scan (which calls tensorBoolGet for each of N elements) takes several milliseconds.

Optimising mask scanning: instead of calling tensorBoolGet per element, process 8 elements per byte in the mask scan. Load mask byte b = data[i/8], then check each bit: if (b & (1<<0)) process element 8*(i/8)+0, etc. This reduces mask overhead from 4 cycles/element to 0.5 cycles/element amortised (1 byte load + 8 bit checks in 4 cycles).

```
/* Optimised 8-at-a-time mask scan for rigLStep DROP */
for (size_t byte_i = 0; byte_i &lt; (N+7)/8; byte_i++) {   // Iterate mask bytes
uint8_t mb = mask-&gt;data[byte_i];                        // Load 8 mask bits at once
for (size_t bit = 0; bit &lt; 8 && byte_i*8+bit &lt; N; bit++)   // Process each bit
{
```

```
if (mb & (1u &lt;&lt; bit)) {                    // Bit=1: weight is active
size_t i = byte_i*8 + bit;                      // Global weight index
if (fabsf(w[i]) &lt;= dropThresh) {              // Below drop threshold
mb &= ~(1u &lt;&lt; bit);                       // Clear bit in byte (deactivate)
w[i] = 0.0f;                                    // Zero the weight
if (++dropped &gt;= K) break;                   // Stop after K drops
}
}
}
mask-&gt;data[byte_i] = mb;                     // Write updated byte back
}
```

This optimised scan writes the mask byte back only once per 8 elements (instead of one write per cleared bit), reducing memory traffic by 8x. For 32768 elements with 10% active: approximately 3277 bit clears, each requiring one byte read+modify+write. Optimised: 4096 byte reads + 327 byte writes = 4423 memory operations vs 3277 * 2 = 6554 for the naive approach.

byteConversionAppend: Sub-Byte Quantisation Packing

byteConversionAppend in Tensor.c handles packing sub-byte quantised values (e.g., 4-bit SYM) into byte arrays. For qBits=4: two mantissas fit in one byte, packed as low nibble (bits 0-3) and high nibble (bits 4-7). For qBits=3: three mantissas fit in one byte with 7 bits wasted per group of 8 elements... except the packing is more complex.

General packing algorithm: maintain a bit buffer. For each mantissa of qBits bits: append qBits bits to the buffer. When the buffer has at least 8 bits, flush the low 8 bits as a byte to the output array. After all elements, flush remaining bits (zero-padded). For N elements of qBits bits: output bytes = ceil(N * qBits / 8).

```
/* Example: packing 3-bit values {5, 3, 7} = {101, 011, 111}
*/
/* Bit buffer (LSB first): 101 011 111 = 9 bits */
/* Byte 0 (bits 0-7): 0110 1101 = 0x6D */
/* Byte 1 (bits 8-8): 0000 0001 = 0x01 */
/* Verify: read back element 0: bits[0:3] = 101 -&gt; 5 */
/* read back element 1: bits[3:6] = 011 -&gt; 3 */
/* read back element 2: bits[6:9] = 111 -&gt; 7 */
```

For SYM_INT32 (qMaxBits=12): each mantissa is 12 bits, stored in 32-bit int32 (4 bytes). The upper 20 bits are wasted -- packing would save 20 bytes per element! For 32768 weights: 32768 * (4 - 1.5) bytes saved = 81,920 bytes. This is why the SYM format (packed to qBits per element) is used for compact checkpoint storage, while SYM_INT32 (unpacked 32-bit) is used for computation.

Deep Dive: RigL Theory, Schedule Tuning, and Ablations

Why Gradient Magnitude Predicts Connection Utility

The core intuition of the RigL GROW step is that |dL/dw_ij| for an inactive weight w_ij (currently zero) measures how much the loss would decrease if this weight were activated and optimally set. By the first-order Taylor approximation: Delta_L ≈ dL/dw_ij * Delta_w_ij. The optimal Delta_w_ij to decrease L is proportional to -dL/dw_ij. The resulting loss decrease is proportional to (dL/dw_ij)^2. Therefore, growing the connections with largest |dL/dw_ij| minimises the first-order approximation of the loss.

This is essentially a greedy coordinate descent step in the space of binary masks. RigL is performing approximate gradient descent on the combinatorial mask selection problem. The approximation error comes from ignoring higher-order terms (the Hessian) and from the fact that the gradient at w_ij=0 does not account for the interaction with other newly grown connections.

Empirically, RigL achieves 90-97% of dense network accuracy at 80-90% sparsity across a wide range of tasks (CIFAR-10, ImageNet, language modeling). The performance gap is largest at very high sparsity (>95%) where the network has too few connections to represent the learned function, and smallest at moderate sparsity (70-80%) where many redundant connections can be removed without capacity loss.

Schedule Tuning for On-Device RigL

For the STM32 speaker ID task, the following RigL hyperparameters are recommended as a starting point: rigLInterval=100 (update topology every 100 gradient steps), alpha_initial=0.3 (drop/grow 30% of active weights per update initially), sparsity=0.9 (90% sparse Linear layers), totalSteps=5000 (total training steps for the cosine alpha schedule), lr_max=0.01, lr_min=0.0001 (cosine annealing LR schedule).

Sensitivity analysis: rigLInterval affects the tradeoff between topology flexibility and gradient quality. Small intervals (every 10 steps) mean the topology changes frequently before gradients have converged -- growing connections based on noisy gradients. Large intervals (every 1000 steps) mean the topology is nearly static, losing RigL's adaptivity advantage. Interval=100 is a reasonable middle ground for the small datasets used in on-device training.

alpha_initial=0.3 means 30% of active weights are swapped per topology update early in training. This is aggressive but necessary to escape poor initial topologies (since the initial mask is typically random or uniform). If accuracy is unstable early in training, try alpha_initial=0.1 (10% swap rate). If the network converges to a poor topology, increase to alpha_initial=0.5.

■ **TIP**	Warmup period: run 500 gradient steps with the dense model (all weights active) before starting RigL topology updates. This allows the weights to develop meaningful magnitudes and gradients before the first drop/grow step, leading to better initial topology selection.

For continual learning (new speaker registration): run RigL topology updates only during the initial dense training phase, then freeze the mask for continual learning updates. This prevents RigL from dropping connections that are critical for old speakers during new speaker training. The frozen mask maintains sparsity while allowing fine-tuning of active weights.

Structured vs Unstructured Sparsity for STM32

RigL as described produces UNSTRUCTURED sparsity: individual weights are independently zeroed based on their magnitude/gradient. This is the most flexible form of sparsity (can achieve the best accuracy for a given sparsity level) but the hardest to accelerate on hardware.

For the STM32F7 scalar processor: unstructured sparsity requires a branch (or mask check) for each individual weight in the matmul inner loop. The branch prediction unit helps (predicting the pattern of active/inactive weights), but the loop overhead is still significant compared to a tight unmasked FMA loop.

STRUCTURED sparsity: entire rows, columns, or channels are zeroed. For conv layers: an entire input channel is dropped (all weights for that channel across all kernel positions). This eliminates entire inner loop iterations without any branch. For Linear layers: an entire output neuron is dropped (all weights in one row of W). This eliminates one output element entirely.

For STM32 deployment: consider using structured-RigL (group sparsity where K weights in each "group" must be zero). Group size of 8 enables SIMD-friendly computation: load 8-bit mask byte, check all 8 bits, process or skip 8 consecutive weights. This achieves structured sparsity's hardware efficiency while retaining some of unstructured sparsity's flexibility.

Hardware Recommendation	For unstructured sparsity on STM32F7: use mask-checked matmul (Chapter 13 modified inner loop). Expected speedup: 4-6x at 90% sparsity. For structured (channel-level) sparsity: skip entire inner loop iterations for dropped channels. Expected speedup: 8-9x at 90% sparsity.

Implementation Guide: Step-by-Step RigL Integration

Step 1: Add mask field to parameter_t (Tensor.h)

First, add the mask pointer to parameter_t in Tensor.h. This is the foundation that all other changes depend on.

```
/* In Tensor.h, modify parameter_t: */
typedef struct parameter {
tensor_t *param;                    // Weight tensor (unchanged)
tensor_t *grad;                     // Gradient tensor (unchanged)
tensor_t *mask;                     // NEW: BOOL mask, NULL=dense
} parameter_t;
```

All existing code that uses param and grad is unaffected -- the mask field simply defaults to NULL for dense parameters. Adding the field to a struct in C is backward-compatible as long as you recompile all translation units (which you must do for any change to a shared header).

Step 2: Allocate mask tensors in model init

```
/* In your model init function, after weight allocation: */
size_t wSize = outFeat * inFeat;    // Number of weight elements
uint8_t *wBuf = reserveMemory(wSize*4);  // Float weight buffer
uint8_t *gBuf = reserveMemory(wSize*4);  // Float gradient buffer
size_t mBytes = (wSize + 7) / 8;    // Mask: 1 bit per weight
uint8_t *mBuf = reserveMemory(mBytes);   // Mask buffer from DataStorage
memset(mBuf, 0xFF, mBytes);         // All weights active initially
/* Create BOOL tensor for mask: */
tensor_t *maskTensor = createBoolTensor(mBuf, wShape);  // BOOL tensor
linCfg.weights-&gt;mask = maskTensor;  // Attach mask to parameter
linCfg.weightMask = maskTensor;     // Also attach to linearConfig
```

The memset(mBuf, 0xFF, mBytes) initialises all mask bits to 1 (all weights active). This is the correct starting state for RigL: begin training with a dense network, and the first rigLStep() call will zero out the bottom-K weights and grow the top-K gradient connections.

Step 3: Modify sgdUpdateKernel (Sgd.c)

Add the weightMask field to sgdUpdateCtx_t and the NULL check + zero-write in the update loop. Pass cfg->weightMask through the context when calling sgdStep for sparse layers. See Chapter 28 for the complete implementation.

Step 4: Modify matmulFloatCore (Matmul.c)

Add an optional colMask parameter to matmulFloatCore. When colMask is non-NULL, add the tensorBoolGet check in the inner loop. Linear.c passes cfg->weightMask (possibly transposed index) as colMask. See Chapter 13 for the complete implementation.

Step 5: Implement findAbsKthSmallest/Largest (MinMax.c)

Add the two K-th element functions to MinMax.c and declare them in MinMax.h. See Chapter 12 for the complete implementation. Verify with unit tests: known weight vector, known mask, known K -- the function should return the correct threshold.

Step 6: Implement rigLStep() (new file: RigL.c)

Create a new file RigL.c with the rigLStep() function as described in Chapter 39. Declare it in RigL.h. Add RigL.c to the CMakeLists.txt. The function depends on MinMax.h, Layer.h, and Tensor.h -- all already in the project.

Step 7: Integrate rigLStep() into optimizerStep() (Optimizer.c)

Add stepCount, rigLInterval, sparsity, model, and numLayers fields to optimizer_t. In optimizerStep(), after the parameter update loop, check if stepCount % rigLInterval == 0 and call rigLStep(). Increment stepCount after every call. See Chapter 27 for the complete design.

Step 8: Update Serialize/Deserialize (Serialize.c, Deserialize.c)

Replace the stub serializeSparsity() with the full implementation that writes the BOOL mask bytes. Update deserializeSparsity() to read the mask bytes into the pre-allocated mask tensor buffer. See Chapter 34 and 35.

Step 9: Test on host before deploying to STM32

Build for the host (Linux/macOS/Windows) and run the unit tests. Compare outputs against PyTorch RigL reference implementation. Key tests: (1) After rigLStep(), active weight count is unchanged (K dropped, K grown). (2) All inactive weights have value 0.0f. (3) After training for 1000 steps, sparse accuracy is within 5% of dense accuracy for the HAR dataset.

Step 10: Profile on STM32 and tune

Deploy to STM32 Nucleo-F746ZG. Profile using DWT cycle counter. Measure: forward pass time (target: <1ms per sample), rigLStep time (target: <10ms, called infrequently), total epoch time. If rigLStep is too slow, implement the optimised 8-at-a-time mask scan (Chapter 38) or reduce the scan using partial sort quickselect instead of selection sort.

Final validation: run 10 full training epochs on the HAR or speaker ID dataset. Record accuracy at each epoch. Verify that: (1) accuracy improves monotonically (no catastrophic forgetting from topology updates), (2) sparsity remains at the target level (count active weights after each rigLStep), (3) the model can be saved and loaded (checkpoint round-trip test).

Glossary

Activation function

A non-linear function applied element-wise to layer outputs. Common: ReLU (max(0,x)), Softmax (normalised exponential). Without non-linearities, a stack of Linear layers is equivalent to a single Linear layer.

AdamW

Adam with decoupled weight decay. Adam adapts the learning rate per parameter using first and second moment estimates of gradients. AdamW separates weight decay from the gradient update, preventing weight decay from being adapted by the moment estimates.

ASYM quantisation

Asymmetric quantisation: real = (mantissa - zeroPoint) * scale. The zeroPoint allows the quantised range to be offset from zero, useful for ReLU activations which are always non-negative.

Axpby

BLAS operation: out = alpha*x + beta*y. Used for momentum updates and moment estimates in SGD and AdamW.

BOOL tensor

A tensor where each element is one bit (true/false). Used in ODT exclusively for RigL weight masks. Stored as packed bytes: 8 elements per byte.

Bump allocator

A simple memory allocator that maintains a pointer to the next free byte. Allocation advances the pointer. No deallocation is supported. Used by DataStorage.c.

Catastrophic forgetting

The tendency of neural networks to completely forget previously learned tasks when trained on new tasks. PPCA replay in ODT addresses this for continual speaker registration.

Continual learning

Training a model on a sequence of tasks without forgetting earlier tasks. In ODT: registering new speakers without forgetting previously registered speakers.

Cortex-M7

ARM processor core used in STM32F7 microcontrollers. Features FPU, DSP extensions, 6-stage superscalar pipeline, 32KB L1 I/D cache.

CSR format

Compressed Sparse Row: a sparse matrix format storing only non-zero values and their column indices. Alternative to the mask-based approach used in ODT.

Dead zone

The range of gradient magnitudes too small to change a quantised weight. With deterministic rounding: gradients smaller than scale/2 are rounded to zero. Stochastic rounding eliminates the dead zone.

Dequantisation

Converting a quantised mantissa back to real value: real = mantissa * scale (SYM) or real = (mantissa - zeroPoint) * scale (ASYM).

DROP

The RigL operation that deactivates the K active weights with smallest absolute value. Sets mask bit to 0 and weight value to 0.0.

DTCM

Data Tightly Coupled Memory. 64 KB of RAM on STM32F7 directly connected to the CPU data bus with zero wait states. Fastest memory on the device.

DSP extension

ARM SIMD instructions for digital signal processing. SMULL (signed multiply long), SMLAL (signed multiply-accumulate long), etc. Available on Cortex-M7.

Dynamic sparsity

Sparsity where the set of active connections changes during training. RigL uses dynamic sparsity. Contrast with static sparsity (prune once, never change).

ExecuteOp

The central dispatch engine in ODT that manages type conversion, scratch buffers, kernel execution, and write-back for all tensor operations.

FatFS

Open-source FAT file system library for embedded systems. Used on STM32 to access SD card data (training datasets, model checkpoints).

Fisher-Yates

An O(n) algorithm for generating a uniformly random permutation of an array in-place. Used in DataLoader.c for epoch shuffling.

FPU

Floating Point Unit. Hardware circuit for floating-point arithmetic. Cortex-M7 has a single-precision VFPv5 FPU.

FQT

Fixed-point Quantisation Training. Training neural networks using integer arithmetic with scale factors, avoiding full-precision floating point.

GROW

The RigL operation that activates the K inactive weights with largest gradient magnitude. Sets mask bit to 1 and initialises weight to 0.0.

HardFault

ARM Cortex-M exception triggered by illegal memory access, instruction execution error, or other hardware fault. Causes the processor to halt (or reset if configured).

HAR

Human Activity Recognition dataset. 30 subjects, 6 activities, 50 Hz accelerometer+gyroscope. Used in ODT as a baseline classification task.

im2col

Image-to-column: reshape convolution input into a matrix so that convolution becomes matrix multiplication. Enables use of optimised matmul kernels for convolution.

Include guard

C preprocessor pattern using #ifndef/#define/#endif to prevent a header from being included multiple times in one translation unit.

ITM

Instrumentation Trace Macrocell. Cortex-M debug peripheral for streaming printf output over the SWD interface to a connected debugger.

JacobiEig

Jacobi eigendecomposition algorithm. Iteratively applies Givens rotations to zero off-diagonal elements of a symmetric matrix. Used by PpcaReplay.c.

L2 regularisation

Weight decay: add a penalty proportional to the sum of squared weights to the loss. Equivalent to SGD weight decay when coupled.

LibriSpeech

A large English speech dataset. ODT uses it for on-device speaker identification.

MFCC

Mel-Frequency Cepstral Coefficients. Features extracted from audio for speech/speaker recognition. Typically 40 coefficients per 25ms frame.

Momentum

An SGD variant that accumulates a velocity vector (EMA of gradients) to dampen oscillations and accelerate convergence.

NPY

NumPy binary array format. Header specifies dtype and shape; raw data follows. Used by ODT for datasets and mask persistence.

ODTS

On-Device Training Serialisation. The binary format used by ODT for model checkpoints. Version 2 is described in Chapter 34.

PPCA

Probabilistic PCA. A generative latent variable model fitted to speaker embeddings. Used by PpcaReplay.c for continual learning replay.

quantisation

Converting real (float) values to discrete integer representations using a scale factor (and optionally a zero-point). Reduces memory and computation.

RigL

Rigged Lottery (Evci et al. 2020). Dynamic sparse training algorithm using gradient-guided topology updates. Described throughout this document.

SRAM

Static Random Access Memory. Volatile memory on STM32 used for tensors, stack, and program data during execution.

STM32F746ZG

Specific microcontroller used in the Nucleo-F746ZG evaluation board. ARM Cortex-M7 at 216 MHz, 1 MB Flash, 320 KB SRAM.

SYM_INT32

Symmetric quantisation using int32 mantissas and a float scale. 12-bit operand constraint for safe accumulation without int64.

SWD

Serial Wire Debug. 2-wire ARM debug interface. Used to flash firmware, set breakpoints, and stream ITM printf output.

tensor_t

The central data structure in ODT. Wraps a raw byte buffer with shape, quantisation, and sparsity metadata.

TRNG

True Random Number Generator. Hardware source of entropy on STM32F7, used to seed the XorShift32 PRNG.

vtable

Virtual function table. A C design pattern using an array of function pointers indexed by type enum. Used in Layer.c for layer dispatch.

XorShift32

A fast PRNG with period 2^32-1. Three XOR-shift operations, 3 cycles on Cortex-M7. Used throughout ODT for stochastic rounding and data shuffling.

Xavier initialisation

Weight initialisation: w ~ U(-sqrt(6/(fan_in+fan_out)), +sqrt(6/(fan_in+fan_out))). Keeps activation variance stable across layers.

zero-point

Offset in ASYM quantisation. real = (mantissa - zeroPoint) * scale. Allows the quantised range to not be centred at zero.

API Quick Reference: All Public Symbols

Common.h Public API

Common.h is the root header included transitively by every ODT source file. It defines primitive types, macros, and constants that all modules depend on.

`typedef uint8_t byte_t;`	`// 8-bit unsigned byte`
`typedef uint16_t half_t;`	`// 16-bit unsigned half-word`
`typedef uint32_t word_t;`	`// 32-bit unsigned word`
`typedef enum {`	
`INT32, FLOAT32,`	`// Unquantised types`
`SYM_INT32, SYM, ASYM, BOOL`	`// Quantised types`
`} dtype_t;`	
`#define ODT_ASSERT(cond, msg) ...`	`// Abort if cond false`
`#define MAX(a,b) ((a)>(b)?(a):(b))`	`// Safe max`
`#define MIN(a,b) ((a)<(b)?(a):(b))`	`// Safe min`
`#define CLAMP(x,lo,hi) MAX(lo,MIN(x,hi))`	`// Clamp to [lo,hi]`
`#define ARRAY_LEN(a) (sizeof(a)/sizeof(a[0]))`	`// Array length`
`#define ALIGN4 __attribute__((aligned(4)))`	`// 4-byte alignment`

ODT_ASSERT macro: in debug builds, calls HAL_UART_Transmit with the error message then triggers BKPT #0 (software breakpoint). In release builds (NDEBUG defined), the assertion body is compiled out -- zero overhead in production.

The CLAMP macro uses the identity CLAMP(x,lo,hi) = MAX(lo, MIN(x,hi)). It evaluates x, lo, and hi twice. For side-effect-free arguments (e.g., array element reads or function calls) this is safe. Avoid passing expressions with side effects (e.g., CLAMP(i++, 0, 9) increments i twice).

■ CONCEPT Include Guard Pattern: every ODT header uses #ifndef ODT_FILENAME_H / #define ODT_FILENAME_H / ... / #endif. This prevents double-inclusion (which would cause duplicate type definitions). Modern C++ uses #pragma once instead, but ODT targets C99 where #pragma once is non-standard.

Tensor.h Public API

`tensor_t *createTensor(void *buf, shape_t s, dtype_t d, float sc, int32_t zp);`	
`tensor_t *createBoolTensor(void *buf, shape_t shape);`	
`size_t tensorNumel(const tensor_t *t);`	`// Product of all dims`
`size_t tensorBytes(const tensor_t *t);`	`// Byte size of buffer`
`float tensorGetF32(const tensor_t *t, size_t i);`	`// Get element i as float`
`void tensorSetF32(tensor_t *t, size_t i, float v);`	`// Set element i from float`
`int32_t tensorGetI32(const tensor_t *t, size_t i);`	`// Raw mantissa at i`
`void tensorSetI32(tensor_t *t, size_t i, int32_t v);`	`// Set raw mantissa`
`bool tensorBoolGet(const tensor_t *t, size_t i);`	`// Get bit i of BOOL tensor`
`void tensorBoolSet(tensor_t *t, size_t i, bool v);`	`// Set bit i`

`void fillTensor(tensor_t *t, float value);`	`// Fill all elements`
`void copyTensor(tensor_t *dst, const tensor_t *src);`	`// Deep copy`
`void transposeTensor(tensor_t *t, int dim0, int dim1);`	`// Zero-copy transpose`
`void reshapeTensor(tensor_t *t, shape_t newShape);`	`// In-place reshape`
`void quantiseTensor(tensor_t *dst, const tensor_t *src, roundingMode_t rm);`	
`void dequantiseTensor(tensor_t *dst, const tensor_t *src);`	

tensorGetF32 and tensorSetF32 perform inline dequantisation/quantisation. For SYM dtype: tensorGetF32 returns mantissa[i] * scale; tensorSetF32 sets mantissa[i] = roundToInt(v / scale). Use these only in test code -- avoid in hot paths due to per-element overhead.

MinMax.h Public API

`float tensorAbsMax(const tensor_t *t, const tensor_t *mask);`	
`float tensorAbsKthSmallestActive(const tensor_t *w,`	
`const tensor_t *mask, size_t K);`	
`float tensorAbsKthLargestInactive(const tensor_t *g,`	
`const tensor_t *mask, size_t K);`	

The K-th element functions use a partial selection sort on a scratch array. Time complexity: O(N*K) where N = number of elements and K = rank. For typical layers (N=4096, K=410 at alpha=0.1): 4096*410 = 1.68 million comparisons. At 216 MHz: approximately 7.8 ms. For very large layers, consider quickselect O(N) expected.

Rng.h Public API

`void rngInit(uint32_t seed);`	`// Seed the XorShift32 PRNG`
`uint32_t rngNext(void);`	`// Next random uint32`
`float rngFloat(void);`	`// Next random float in [0,1)`
`float rngNormal(float mean, float std);`	`// Box-Muller normal sample`
`uint32_t rngBounded(uint32_t bound);`	`// Uniform in [0, bound) via rejection`
`void rngShuffle(size_t *arr, size_t n);`	`// Fisher-Yates in-place shuffle`

rngNormal uses Box-Muller: Z = sqrt(-2*ln(U1)) * cos(2*pi*U2). Two normal samples per call: the second (sin component) is cached and returned on the next rngNormal() call, halving sqrt/cos operations for bulk normal sampling.

DataStorage.h Public API

`void dataStorageInit(void *buf, size_t size);`	`// Initialise bump allocator`
`void *dataStorageAlloc(size_t bytes);`	`// Allocate bytes (no free)`
`void dataStorageReset(void);`	`// Reset pointer to base`
`size_t dataStorageFree(void);`	`// Remaining bytes available`
`size_t dataStorageUsed(void);`	`// Bytes allocated so far`

dataStorageReset sets the allocation pointer back to the beginning of the buffer. Previously returned pointers become invalid. Use this between inference requests to reclaim activation memory. For training, do NOT reset between forward and backward passes -- activations are needed for backward.

Performance Analysis and Optimisation Guide

Profiling with DWT Cycle Counter

The ARM Cortex-M7 DWT unit provides a free-running 32-bit cycle counter. At 216 MHz: 216 million increments per second. Wrap period: 2^32 / 216e6 = 19.9 seconds. Unsigned subtraction handles wrap-around correctly.

```
#include "core_cm7.h"                                   // CMSIS header
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;         // Enable trace
DWT->CYCCNT = 0;                                        // Reset counter
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;                    // Start counting
uint32_t t0 = DWT->CYCCNT;                              // Capture start
layerForward(layer, input);                             // Call under test
uint32_t cycles = DWT->CYCCNT - t0;                     // Elapsed cycles
float ms = cycles / 216000.0f;                          // Convert to ms at 216 MHz
```

■ TIP Always warm up the cache before benchmarking: call the function twice, discarding the first result. The first call fills I/D caches; the second call measures steady-state performance (cache-hot).

Dense vs Sparse Matmul Performance

Dense matmul 256x128: 32768 FMAs. At 1 FMA/cycle (pipelined, cache-hot): 32768 cycles = 0.15 ms. Measured on STM32F746ZG: ~0.18 ms.

Naive sparse matmul at 90% sparsity with mask check per element: mask iteration = 32768 * 4 cycles = 131072 cycles (0.61 ms). Active weight computation = 3277 * 4 cycles = 13108 cycles. Total: 0.67 ms -- 3.7x SLOWER than dense. The mask checking overhead dominates.

Optimised sparse matmul (8-at-a-time byte scan): mask iteration = 4096 * 1 cycle = 4096 cycles. Active weights = 3277 * 4 cycles = 13108 cycles. Total: 17204 cycles = 0.08 ms -- 2.25x FASTER than dense. Proper byte-level mask scanning is essential.

■ WARNING Naive unstructured sparsity is SLOWER than dense computation on Cortex-M7. Only the optimised 8-at-a-time mask scan makes sparse faster than dense. Always profile before claiming a speedup from sparsity.

Stack Usage and DTCM Planning

DTCM (64 KB at 0x20000000) is the fastest memory on STM32F7. Allocate it to: (1) the C stack (bottom of DTCM, grows downward), (2) activation scratch buffers (declared with __attribute__((section(".dtcm_bss")))), (3) hot-path loop variables (auto on stack = DTCM).

```
/* Linker script (STM32F746ZG.ld) */
DTCM (xrw) : ORIGIN = 0x20000000, LENGTH = 64K          // DTCM region
```

`AXI_SRAM (xrw) : ORIGIN = 0x20010000, LENGTH = 256K`	`// AXI SRAM region`
`/* In memory map section: */`	
`.dtcm_bss (NOLOAD) : { *(.dtcm_bss) } > DTCM`	`// BSS in DTCM`
`.bss (NOLOAD) : { *(.bss) } > AXI_SRAM`	`// BSS in AXI SRAM`

The rigLStep scratch array (K floats for partial sort) can reach 13 KB at large sparsity. Allocate from DataStorage (AXI SRAM) rather than the stack to avoid stack overflow. Check with arm-none-eabi-nm and arm-none-eabi-size after linking to verify DTCM usage.

FPU Utilisation Verification

`# CMakeLists.txt FPU flags`	
`set(CPU_FLAGS "-mcpu=cortex-m7 -mthumb")`	`// M7 Thumb-2 mode`
`set(FPU_FLAGS "-mfpu=fpv5-d16 -mfloat-abi=hard")`	`// Hard FPU ABI`
`set(OPT_FLAGS "-O2")`	`// Optimise for speed`
`add_compile_options(${CPU_FLAGS} ${FPU_FLAGS} ${OPT_FLAGS})`	

Verify FPU usage: disassemble the matmul function (arm-none-eabi-objdump -d) and confirm VMLA.F32 / VLDR.32 / VSTR.32 instructions appear. If you see calls to __aeabi_fmul or __aeabi_fadd, the FPU is disabled or the "soft" ABI is selected -- fix the CMakeLists flags.

Testing Strategy for the ODT Library

Unit Test Framework

ODT uses a lightweight hand-written unit test framework. Each test file contains a run_tests() function calling individual test functions. Assertions use ODT_ASSERT. The test runner compiles for the host (x86-64 Linux) using a HAL stub replacing HAL_UART_Transmit with printf.

```
static void test_absMax_dense(void) {
float buf[4] = {1.0f, -3.0f, 2.0f, -0.5f};
tensor_t *t = createTensor(buf, shape1d(4),
FLOAT32, 1.0f, 0);
float result = tensorAbsMax(t, NULL); // NULL mask = all active
ODT_ASSERT(fabsf(result - 3.0f) &lt; 1e-6f,
"absMax wrong");
}
```

Integration Tests: Forward Pass Correctness

Verify forward pass against PyTorch: (1) Load identical weights in both PyTorch and ODT. (2) Run forward on the same input. (3) Compare outputs element-wise with tolerance 1e-4. (4) For quantised layers, increase tolerance to 1e-2 (quantisation noise).

```
# PyTorch reference
import torch, numpy as np
W = np.load("w.npy") # shape [16, 8]
x = np.load("x.npy") # shape [8]
lin = torch.nn.Linear(8, 16, bias=False)
lin.weight.data = torch.from_numpy(W)
out_ref = lin(torch.from_numpy(x)).detach().numpy()
np.save("out_ref.npy", out_ref)
```

Backward Pass: Finite-Difference Gradient Check

Numerically verify gradients using centered finite differences: dL/dw_i = (L(w+eps*e_i) - L(w-eps*e_i)) / (2*eps). Compare with layerBackward output. Relative error should be below 1e-3 for float32 at eps=1e-3.

```
const float eps = 1e-3f;
float w_orig = tensorGetF32(w, i);
tensorSetF32(w, i, w_orig + eps);
float loss_plus = computeLoss(model, x, y);
tensorSetF32(w, i, w_orig - eps);
float loss_minus = computeLoss(model, x, y);
tensorSetF32(w, i, w_orig); // Restore
float fd = (loss_plus - loss_minus) / (2*eps); // Finite difference
float analytic = tensorGetF32(wGrad, i); // Analytic
```

```
float err = fabsf(fd - analytic) / (fabsf(analytic) + 1e-8f);
ODT_ASSERT(err &lt; 1e-3f, "grad check failed");
```

RigL Topology Invariant Test

After each rigLStep: verify active count unchanged, inactive weights are zero, dropped weights were smallest-magnitude before the step.

```
size_t cnt_before = countActiveMask(mask);
rigLStep(model, numLayers, sparsity, step, total);
size_t cnt_after = countActiveMask(mask);
ODT_ASSERT(cnt_before == cnt_after, "count wrong");
for (size_t i = 0; i &lt; N; i++) {
if (!tensorBoolGet(mask, i))                    // Inactive weight
ODT_ASSERT(w[i] == 0.0f, "nonzero inactive");
}
```

Porting ODT to Other Microcontrollers

HAL Interface Requirements

ODT depends on four HAL interfaces: (1) HAL_GetTick() returning milliseconds since boot. (2) HAL_UART_Transmit() for debug output. (3) Standard C FILE I/O (fopen/fread/fwrite/fclose) for NPYLoader and Serialize. (4) Optional: HAL_RNG_GenerateRandomNumber() for TRNG-based seeding.

```
/* Porting stub for non-STM32 targets */
uint32_t HAL_GetTick(void) {
return DWT-&gt;CYCCNT / CYCLES_PER_MS;  // DWT on any Cortex-M7
}
void HAL_UART_Transmit(UART_HandleTypeDef *h,
uint8_t *buf, uint16_t size, uint32_t to) {
for (uint16_t i = 0; i &lt; size; i++)
ITM_SendChar(buf[i]);  // ITM semihosting
}
```

Porting to Nordic nRF52840 (Cortex-M4F, 64 MHz)

nRF52840 differences from STM32F746ZG: Cortex-M4F at 64 MHz (vs M7 at 216 MHz), 256 KB SRAM (vs 320 KB), no DTCM (all SRAM is equal latency), nRF5 SDK HAL. Performance penalty: 3.4x slower (clock ratio). A 256x128 matmul: 0.18 ms * 3.4 = 0.61 ms -- acceptable for training.

Porting to RP2040 (Cortex-M0+, 133 MHz, no FPU)

RP2040 has no hardware FPU. Float operations use software emulation: FADD ~10 cycles, FMUL ~10 cycles. Matmul inner loop: ~20 cycles/FMA vs 1 cycle/FMA on M7. A 256x128 matmul: 0.18 ms * 20 = 3.6 ms. Strongly prefer INT8/INT16 fixed-point (SYM_INT32) to avoid software float.

For RP2040: use SYM_INT32 with 12-bit operands throughout. The 32-bit integer multiply (MUL instruction) is 1 cycle on M0+. Accumulation in int32. The matmul inner loop becomes: load 12-bit mantissa, MUL, accumulate in int32. Approximately 3 cycles/MAC. 256x128 matmul: 3*32768 = 98304 cycles = 0.74 ms at 133 MHz. 5x improvement over software float.

Helium (MVE) on Cortex-M55/M85

ARM Cortex-M55/M85 include the Helium (M-Profile Vector Extension) SIMD ISA: 128-bit vector registers, vectorised int8/int16/float32 MACs. For int8 SIMD matmul: 16 MACs per VMLA instruction. Peak: 16 int8 MACs/cycle vs 1 float32 FMA/cycle on M7. Expected speedup: 8-12x for int8 GEMM. Porting: replace matmulFloatCore with Arm Compute Library (ACL) Helium GEMM kernel.

Continual Learning: PPCA Replay Theory and Practice

The Catastrophic Forgetting Problem

When a neural network is trained sequentially on tasks T1, T2, ..., Tk, it tends to overwrite the weights learned for T1 while training on T2. This phenomenon, called catastrophic forgetting, is a fundamental obstacle to on-device lifelong learning. The speaker ID use case exemplifies this: registering a new speaker should not erase memory of previously registered speakers.

Three categories of solutions: (1) Regularisation-based (e.g., EWC -- Elastic Weight Consolidation): penalise changes to weights that were important for old tasks. Requires storing per-weight importance scores (same memory as the model). (2) Architecture-based (e.g., progressive networks): grow new capacity for each new task. Memory grows with the number of tasks -- impractical on embedded systems. (3) Replay-based: store or generate examples from old tasks and mix them into training on new tasks. ODT uses replay via PPCA generative models.

■ **CONCEPT**	PPCA Replay: fit a Probabilistic PCA model to the embeddings of each known speaker. When training on a new speaker, generate synthetic embeddings from the PPCA models of old speakers and include them in the training batch. The network sees "remembered" old-speaker examples without needing to store raw audio.

PPCA Mathematical Background

Probabilistic PCA (PPCA) is a latent variable model: x = Wz + mu + epsilon, where x is the observed data (embedding vector, e.g., 128-dimensional), z is a low-dimensional latent code (e.g., 8-dimensional), W is the factor loading matrix (128x8), mu is the mean, and epsilon is isotropic Gaussian noise N(0, sigma^2 * I).

The PPCA generative process: sample z ~ N(0, I_q), then x ~ N(Wz + mu, sigma^2 * I_p). To generate a synthetic embedding: z = randn(q), x = W*z + mu + sigma*randn(p). Since x is a linear transform of Gaussians plus Gaussian noise, x is Gaussian: x ~ N(mu, W*W^T + sigma^2*I).

Fitting PPCA via eigendecomposition: given N speaker embeddings x_1, ..., x_N: compute sample mean mu = (1/N)*sum(x_i), compute sample covariance C = (1/N)*sum((x_i-mu)(x_i-mu)^T). Find the top-q eigenvectors v_1,...,v_q and eigenvalues lambda_1,...,lambda_q of C. Then W = [v_1*sqrt(lambda_1-sigma^2), ..., v_q*sqrt(lambda_q-sigma^2)] and sigma^2 = average of the remaining eigenvalues.

Jacobi eigendecomposition (used in PpcaReplay.c for symmetric matrices): iteratively apply Givens rotations to annihilate off-diagonal elements. Each sweep (one pass over all off-diagonal pairs) reduces off-diagonal Frobenius norm by a factor of (1 - 2/(p*(p-1))). For a 128x128 covariance matrix: convergence to machine epsilon in approximately 20-30 sweeps, each sweep requiring 128*127/2 = 8128 Givens rotations.

Memory Budget for PPCA Models

Each registered speaker requires one PPCA model. Memory per speaker PPCA model: mean vector mu: p * 4 bytes (p=128: 512 bytes). Factor loading matrix W: p*q * 4 bytes (p=128, q=8: 4096 bytes). Sigma^2: 4 bytes. Total per speaker: 4612 bytes = 4.5 KB. For 20 speakers: 92 KB. This is 29% of AXI SRAM (256 KB) -- feasible.

During replay, generating one synthetic embedding: z = rngNormal(0,1) repeated q times (8 calls), x = W*z (matrix-vector product, 128*8=1024 FMAs, ~1024 cycles = 4.7 us), x += mu (128 additions), add sigma*rngNormal noise (128 calls). Total: approximately 10 us per synthetic embedding. Generating a full replay batch of 32 embeddings per old speaker: 32*10us * 20 speakers = 6.4 ms. Acceptable overhead.

Implementing Continual Learning Training Loop

The continual learning training loop for speaker registration:

```
/* For each new speaker registration: */
1. Collect N audio samples from new speaker.
2. Extract embeddings: forward pass encoder on samples.
3. Fit PPCA to the N new embeddings.
4. Store PPCA model in ppcaModels[newSpeakerIdx].
5. For each training epoch:
a. Real batch: sample K_real from new speaker embeddings.
b. Replay batch: for each old speaker s:
generate K_replay synthetic embeddings from ppcaModels[s].
c. Combined batch: concat real + replay embeddings.
d. Forward pass combined batch through classifier.
e. Backward pass, update weights.
6. Validate on held-out set (all speakers).
```

■ TIP K_replay per old speaker should approximately equal K_real (balanced replay). With imbalanced replay (too few old-speaker samples), the network still shifts toward the new speaker. With too many, the new speaker is under-represented. A ratio of K_real = K_replay = 16-32 per speaker typically works well.

Quantisation Mathematics: From Theory to C Code

Uniform Scalar Quantisation

Uniform scalar quantisation maps real values to integers using a linear mapping: mantissa = round(real / scale) clamped to [qMin, qMax]. Real values are reconstructed as: real_hat = mantissa * scale. The quantisation step size is scale. The quantised grid has 2*qMax + 1 levels (for symmetric: qMin = -qMax).

Optimal scale for SYM quantisation: scale = max(|real_i|) / qMax. This maps the most extreme value to +/-qMax, maximally using the quantised range. Smaller scale means finer quantisation step (less error for small values) but clips large values to +/-qMax * scale. The choice scale = absMax / qMax is the minimax optimal scale that minimises the maximum quantisation error.

For SYM_INT32 with qMaxBits=12: qMax = 2^11 - 1 = 2047. scale = absMax / 2047. A weight of absMax = 1.0 gets scale = 4.88e-4. A weight of value 0.5 gets mantissa = round(0.5 / 4.88e-4) = round(1023.5) = 1024. Dequantised: 1024 * 4.88e-4 = 0.50012 -- error 0.012%.

■ CONCEPT Saturation vs Underflow: if a weight exceeds absMax (due to gradient updates after quantisation), it saturates to +/-qMax -- the error can be large. If a weight is much smaller than scale/2, it rounds to 0 and is quantised away. Both cases hurt accuracy: saturated weights have large error; underflowed weights carry no information.

Stochastic Rounding: SR_HALF_AWAY Mode

Stochastic rounding (SR_HALF_AWAY): add uniform noise U ~ Uniform(-0.5, 0.5) * scale before rounding. Equivalently: r = real/scale + U, mantissa = round(r). The expected value of mantissa is real/scale (unbiased). The noise variance is 1/12 (variance of U[-0.5,0.5)).

Contrast with round-half-away-from-zero (deterministic): always rounds midpoints (x.5) away from zero. This is biased -- values at x.5 always round up (or down based on sign). Over many weights, the bias accumulates. For training, accumulated quantisation bias shifts the effective learning rate.

```
/* SR_HALF_AWAY in Rounding.c */
float noise = rngFloat() - 0.5f;          // U ~ Uniform(-0.5, 0.5)
float shifted = real / scale + noise;     // Shift by noise
int32_t mantissa = (int32_t)roundf(shifted);  // Round to nearest integer
mantissa = CLAMP(mantissa, -qMax, qMax);  // Saturate to valid range
```

Performance note: rngFloat() calls rngNext() which does 3 XOR-shift operations (3 cycles on M7) plus a multiply and subtract (2 more cycles). Total: ~5 cycles per stochastic rounding call. For quantising a 32768-element weight tensor: 32768 * 5 = 163,840 cycles = 0.76 ms. This is a significant overhead for quantiseTensor calls -- minimise calls to quantiseTensor in the training loop inner path.

Forward Quantisation in the Training Loop

In FQT training, the "straight-through estimator" (STE) is used for the backward pass through quantisation. Forward: out = Q(real) = mantissa * scale (quantised). Backward: dL/d(real) = dL/d(out) (gradient passes through as if Q is the identity function). The STE ignores quantisation in the backward pass, allowing

gradients to flow to the real-valued weights.

Implementation: the forward pass always uses quantised weights (calls quantiseTensor to get mantissas, then dequantiseTensor to get quantised-then-dequantised floats). The gradient update (SGD/AdamW) updates the real-valued weights (in float32 or int32). The next forward pass re-quantises. This round-trip (float -> quantise -> dequantise -> float) introduces quantisation noise but allows gradient-based optimisation.

Accumulation Precision in Matmul

For SYM_INT32 matmul: inner product = sum over k of (W_mantissa[row,k] * X_mantissa[k]). Each term: 12-bit * 12-bit = 24-bit product. Sum of N=128 terms: up to 128 * 2047^2 = 537,395,072. This fits in int32 (max 2,147,483,647). No int64 accumulator needed.

For SYM matmul (8-bit): inner product = sum of 128 terms of 127^2 = 2,064,512. Fits easily in int32. For SYM matmul with longer vectors (N=1024): 1024 * 127^2 = 16,515,072. Still fits in int32. The int32 accumulator is safe for any practical layer size with 8-bit or 12-bit operands.

Real value of the inner product: real_out = scale_W * scale_X * (int32 accumulator). The scale factors are applied once per output element (not per term), keeping the computation in integer arithmetic until the output is ready.

Appendix A: Complete RigL Implementation Listings

MinMax.h — Full Header with RigL Extensions

The MinMax.h header declares all functions for finding extreme values in tensors. The two RigL-specific additions (tensorAbsKthSmallestActive and tensorAbsKthLargestInactive) are shown below in context with the full header.

```
#ifndef ODT_MINMAX_H
#define ODT_MINMAX_H                    // Include guard
#include "Tensor.h"                     // For tensor_t
#include &lt;stddef.h&gt;                 // For size_t
/* Existing functions */
float tensorAbsMax(const tensor_t *t,
const tensor_t *mask);                  // NULL mask = all elements
float tensorAbsMin(const tensor_t *t,
const tensor_t *mask);
void tensorClip(tensor_t *t,
float lo, float hi);                    // In-place clip
/* RigL additions — K-th order statistics */
float tensorAbsKthSmallestActive(
const tensor_t *weights,                // Weight tensor (float)
const tensor_t *mask,                   // BOOL mask tensor
size_t K);                              // Rank (1-indexed)
float tensorAbsKthLargestInactive(
const tensor_t *grads,                  // Gradient tensor (float)
const tensor_t *mask,                   // BOOL mask tensor
size_t K);                              // Rank (1-indexed)
#endif /* ODT_MINMAX_H */
```

MinMax.c — K-th Order Statistics Implementation

The implementation uses a partial selection sort. We maintain a sorted scratch array of the top-K values seen so far. This avoids allocating a full N-element buffer, at the cost of O(N*K) comparisons.

```
#include "MinMax.h"
#include "DataStorage.h"                // For scratch alloc
#include &lt;math.h&gt;                   // For fabsf
#include &lt;string.h&gt;                 // For memmove
float tensorAbsKthSmallestActive(
const tensor_t *w, const tensor_t *mask, size_t K) {
size_t N = tensorNumel(w);
float *scratch = dataStorageAlloc(K*sizeof(float));
```

```
size_t found = 0;                                   // Elements in scratch so far
for (size_t i = 0; i &lt; N; i++) {                 // Scan all elements
if (!tensorBoolGet(mask, i)) continue;              // Skip inactive
float v = fabsf(tensorGetF32(w, i));                // Absolute value
if (found &lt; K) {                                 // Scratch not full yet
scratch[found++] = v;                               // Add to scratch
if (found == K) {                                   // Just filled: sort it
/* insertion sort scratch */
for (size_t a=1;a&lt;K;a++) {
float key = scratch[a];
size_t b = a;
while (b&gt;0 && scratch[b-1]&gt;key) {
scratch[b]=scratch[b-1];b--;
}
scratch[b]=key;
}
}
} else if (v &lt; scratch[K-1]) {                   // New value smaller than max
/* Insert v into sorted scratch */
size_t pos = K-1;
while (pos&gt;0 && scratch[pos-1]&gt;v) {
scratch[pos]=scratch[pos-1]; pos--;
}
scratch[pos] = v;
}
}
float result = (found &gt;= K) ? scratch[K-1] :
scratch[found-1];
return result;                                      // K-th smallest abs value
}
```

RigL.h — Module Header

```
#ifndef ODT_RIGL_H
#define ODT_RIGL_H
#include "Layer.h"                                  // For layer_t
#include &lt;stddef.h&gt;                             // For size_t
/**
* rigLStep: perform one RigL topology update.
* Call every rigLInterval gradient steps.
* @param model Array of layer pointers.
* @param numLayers Number of layers.
* @param sparsity Target fraction of zeros.
* @param step Current training step (0-indexed).
* @param totalSteps Total steps for alpha decay.
```

```
*/
void rigLStep(layer_t **model, size_t numLayers,
float sparsity, size_t step, size_t totalSteps);
#endif /* ODT_RIGL_H */
```

RigL.c — Complete rigLStep() Implementation

The full rigLStep() function with detailed inline comments. This is the most complex function in the ODT library -- it coordinates MinMax, Tensor, and Layer APIs to execute the RigL DROP + GROW topology update.

```
#include "RigL.h"
#include "MinMax.h"
#include "Tensor.h"
#include "Layer.h"
#include &lt;math.h&gt;  // For cosf, M_PI
void rigLStep(layer_t **model, size_t numLayers,
float sparsity, size_t step, size_t totalSteps) {
/* Cosine-decayed drop fraction */
float cosDecay = 0.5f*(1.0f+cosf((float)M_PI
*step/totalSteps));
float alpha = 0.3f * cosDecay;  // alpha: 0.3 -> 0 over training
for (size_t li = 0; li &lt; numLayers; li++) {  // Iterate layers
layer_t *lay = model[li];
size_t nParams = layerNumParams(lay);
for (size_t pi=0; pi&lt;nParams; pi++) {  // Iterate parameters
parameter_t *par = layerGetParam(lay,pi);
if (par-&gt;mask == NULL) continue;  // Dense param: skip
tensor_t *w = par-&gt;param;
tensor_t *g = par-&gt;grad;
tensor_t *m = par-&gt;mask;
size_t N = tensorNumel(w);  // Total weights
size_t numActive = 0;
for (size_t i=0;i&lt;N;i++)
if (tensorBoolGet(m,i)) numActive++;  // Count active
size_t K = (size_t)(alpha * numActive);  // Drop/grow count
if (K == 0) continue;  // Nothing to do
/* === DROP: deactivate K smallest |w| active === */
float dropThr = tensorAbsKthSmallestActive(w,m,K);
size_t dropped = 0;
for (size_t i=0;i&lt;N&&dropped&lt;K;i++) {
if (!tensorBoolGet(m,i)) continue;  // Skip inactive
if (fabsf(tensorGetF32(w,i)) &lt;= dropThr) {
tensorBoolSet(m,i,false);  // Deactivate
tensorSetF32(w,i,0.0f);  // Zero weight
dropped++;
```

```
}
}
/* === GROW: activate K largest |grad| inactive === */
float growThr = tensorAbsKthLargestInactive(g,m,K);
size_t grown = 0;
for (size_t i=0;i&lt;N&&grown&lt;K;i++) {
if (tensorBoolGet(m,i)) continue;                    // Skip active
if (fabsf(tensorGetF32(g,i)) &gt;= growThr) {
tensorBoolSet(m,i,true);                             // Activate
tensorSetF32(w,i,0.0f);                              // Init to zero
grown++;
}
}
}
}
}
}
```

■ **RigL: rigLStep Invariants**	The rigLStep guarantees: dropped == grown (same K) => numActive is preserved exactly. All newly inactive weights are zeroed. All newly active weights are initialised to 0.0f (they will receive their first gradient update on the next backward pass).

Appendix B: Complete Sparse SGD and Linear Layer Listings

Sgd.h — Sparse-Aware SGD Header

```
#ifndef ODT_SGD_H
#define ODT_SGD_H
#include "Tensor.h"
typedef struct {
float lr;                       // Learning rate
float momentum;                 // Momentum coefficient (0=off)
float weightDecay;              // L2 regularisation coefficient
tensor_t *velocity;             // Momentum buffer (NULL if momentum=0)
tensor_t *weightMask;           // RigL BOOL mask (NULL=dense)
} sgdUpdateCtx_t;
void sgdStep(tensor_t *param, tensor_t *grad,
sgdUpdateCtx_t *cfg);           // One parameter update
#endif
```

Sgd.c — Sparse-Aware SGD Implementation

```
#include "Sgd.h"
#include &lt;string.h&gt;
void sgdStep(tensor_t *param, tensor_t *grad, sgdUpdateCtx_t
*cfg) {
size_t N = tensorNumel(param);
float lr = cfg-&gt;lr;
float mom = cfg-&gt;momentum;
float wd = cfg-&gt;weightDecay;
tensor_t *mask = cfg-&gt;weightMask;     // NULL = dense
tensor_t *vel = cfg-&gt;velocity;       // NULL if no momentum
for (size_t i = 0; i &lt; N; i++) {
/* Skip inactive weights (mask == false) */
if (mask && !tensorBoolGet(mask, i)) {  // RigL: inactive
if (vel) tensorSetF32(vel,i,0.0f);      // Zero velocity for inactive
continue;                               // Do not update weight
}
float w = tensorGetF32(param, i);       // Current weight
float g = tensorGetF32(grad, i);        // Gradient
/* Weight decay (L2 regularisation) */
g += wd * w;                            // Effective gradient with WD
/* Momentum update */
if (vel) {
```

```
float v = tensorGetF32(vel, i);
v = mom * v + g;                    // EMA of gradients
tensorSetF32(vel, i, v);
g = v;                              // Use velocity as effective grad
}
/* Weight update */
tensorSetF32(param, i, w - lr * g); // SGD step: w = w - lr * g
}
}
```

The critical RigL change is the three-line block at the top of the loop body: if the weight is masked out (inactive), zero its velocity buffer and skip the update. Without zeroing the velocity: after a weight is grown back (mask set to 1 again), it would immediately receive a large velocity-driven update from the accumulated velocity during the period it was inactive. This would likely cause an undesirable large weight jump. Zeroing velocity for inactive weights ensures grown weights start with zero velocity.

Linear.h — Configuration and Forward/Backward Signatures

```
#ifndef ODT_LINEAR_H
#define ODT_LINEAR_H
#include "Layer.h"
#include "Tensor.h"
typedef struct {
parameter_t *weights;               // Weight param (has .mask for RigL)
parameter_t *bias;                  // Bias param (no mask, always dense)
tensor_t *input;                    // Saved input for backward
tensor_t *weightMask;               // Convenience alias: weights->mask
size_t inFeatures;
size_t outFeatures;
bool useBias;
} linearConfig_t;
layer_t *createLinearLayer(linearConfig_t *cfg);
/* Forward: out = W*x + b (sparse if mask set) */
/* Backward: grad_x = W^T * grad_out
* grad_W = grad_out outer x
* grad_b = grad_out */
#endif
```

Linear.c — Sparse Forward Pass

```
static tensor_t *linearForwardFn(void *cfg_v, tensor_t
*input) {
linearConfig_t *cfg = cfg_v;
tensor_t *W = cfg-&gt;weights-&gt;param;          // [outFeat x inFeat]
tensor_t *b = cfg-&gt;useBias ? cfg-&gt;bias-&gt;param :
NULL;
```

```
tensor_t *mask = cfg-&gt;weightMask;                    // NULL = dense
size_t out = cfg-&gt;outFeatures;
size_t in = cfg-&gt;inFeatures;
cfg-&gt;input = input;                                 // Save for backward
tensor_t *output = allocActivation(out);
/* output[i] = sum_j W[i,j] * input[j] */
for (size_t i = 0; i &lt; out; i++) {
float acc = 0.0f;
for (size_t j = 0; j &lt; in; j++) {
/* Skip inactive weights */
if (mask) {
size_t idx = i*in + j;                              // Flat index into W
if (!tensorBoolGet(mask, idx)) continue;
}
acc += tensorGetF32(W, i*in+j)
* tensorGetF32(input, j);
}
if (b) acc += tensorGetF32(b, i);                   // Add bias
tensorSetF32(output, i, acc);
}
return output;
}
```

The mask check inside the inner loop is the performance bottleneck at high density. For large dense layers (no mask), the branch is compiled away by the null check before the loop. For sparse layers, consider the 8-at-a-time byte scan optimisation described in the Performance chapter to eliminate per-element tensorBoolGet overhead.

Appendix C: Serialize and Deserialize Complete Listings

ODTS v2 Binary Format Specification

The ODTS v2 format is a flat binary file. All integers are little-endian. Floats use IEEE 754 binary32. The file layout:

Offset Size Field	
0 4 Magic: 0x4F 0x44 0x54 0x53 ("ODTS")	
4 4 Version: 0x00000002	// Always 2 for ODTS v2
8 4 num_layers (uint32)	
--- For each layer (num_layers entries): ---	
12+ 4 layer_type (uint32, enum layer_type_t)	
+4 4 num_params (uint32)	
--- For each param (num_params entries): ---	
+0 4 shape_ndim (uint32)	// Number of dimensions
+4 4*ndim shape_dims[ndim] (uint32 each)	// Dimension sizes
+4*ndim 4 dtype (uint32, enum dtype_t)	
+4 4 scale (float32)	
+4 4 zero_point (int32)	
+4 N*elemBytes weight_data	// N=product of dims
+... 1 has_mask (uint8, 0 or 1)	
(if has_mask==1):	
+1 ceil(N/8) mask_bytes	// 1 bit per weight

Serialize.c — Complete serializeModel() Implementation

#include "Serialize.h"	
#include "Layer.h"	
#include <stdio.h>	
#include <stdint.h>	
static const uint8_t MAGIC[4] = {0x4F,0x44,0x54,0x53};	
static const uint32_t VERSION = 2;	
void serializeModel(layer_t **model, size_t n, const char *path) {	
FILE *fp = fopen(path, "wb");	
ODT_ASSERT(fp != NULL, "cannot open file");	
fwrite(MAGIC, 1, 4, fp);	// Magic bytes
fwrite(&VERSION, 4, 1, fp);	// Version
uint32_t nl = (uint32_t)n;	
fwrite(&nl, 4, 1, fp);	// num_layers
for (size_t li = 0; li < n; li++) {	

```
uint32_t lt = (uint32_t)model[li]-&gt;type;
fwrite(&lt, 4, 1, fp);                                  // layer_type
uint32_t np_ = (uint32_t)layerNumParams(model[li]);
fwrite(&np_, 4, 1, fp);                                 // num_params
for (size_t pi = 0; pi &lt; np_; pi++) {
parameter_t *par = layerGetParam(model[li], pi);
serializeWeights(par-&gt;param, fp);                    // Weights
uint8_t hm = (par-&gt;mask != NULL);
fwrite(&hm, 1, 1, fp);                                  // has_mask
if (hm) serializeSparsity(par-&gt;mask, fp);
}
}
fclose(fp);
}
```

```
void serializeWeights(const tensor_t *t, FILE *fp) {
uint32_t ndim = (uint32_t)t-&gt;shape.ndim;
fwrite(&ndim, 4, 1, fp);
for (size_t d = 0; d &lt; ndim; d++) {
uint32_t dim = (uint32_t)t-&gt;shape.dims[d];
fwrite(&dim, 4, 1, fp);
}
uint32_t dt = (uint32_t)t-&gt;dtype;
fwrite(&dt, 4, 1, fp);
fwrite(&t-&gt;scale, 4, 1, fp);
fwrite(&t-&gt;zeroPoint, 4, 1, fp);
size_t nb = tensorBytes(t);
fwrite(t-&gt;data, 1, nb, fp);                          // Raw weight bytes
}
void serializeSparsity(const tensor_t *mask, FILE *fp) {
size_t N = tensorNumel(mask);
size_t maskBytes = (N + 7) / 8;
fwrite(mask-&gt;data, 1, maskBytes, fp);                // Packed bit mask
}
```

Deserialize.c — deserializeModel() Implementation

```
void deserializeModel(layer_t **model, size_t n, const char
*path) {
FILE *fp = fopen(path, "rb");
ODT_ASSERT(fp != NULL, "cannot open");
uint8_t magic[4]; fread(magic, 1, 4, fp);
/* verify magic */
for (int i = 0; i &lt; 4; i++)
ODT_ASSERT(magic[i]==MAGIC[i],"bad magic");
uint32_t ver; fread(&ver, 4, 1, fp);
```

ODT_ASSERT(ver == 2, "unsupported version");	
uint32_t nl; fread(&nl, 4, 1, fp);	
ODT_ASSERT(nl == n, "layer count mismatch");	
for (size_t li = 0; li < n; li++) {	
uint32_t lt; fread(&lt, 4, 1, fp);	
/* verify lt matches model[li]->type */	
uint32_t np_; fread(&np_, 4, 1, fp);	
for (size_t pi = 0; pi < np_; pi++) {	
parameter_t *par=layerGetParam(model[li],pi);	
deserializeWeights(par->param, fp);	
uint8_t hm; fread(&hm, 1, 1, fp);	
if (hm) deserializeSparsity(par->mask, fp);	
}	
}	
fclose(fp);	
}	

Appendix D: Complete Training Loop Example

Full Training Loop with RigL for Speaker ID

Below is a complete, annotated training loop integrating all ODT modules: DataLoader, NPYLoader, LinearLayer (sparse), SGD with momentum, RigL topology updates, cross-entropy loss, accuracy tracking, and checkpoint saving.

```
#include "Common.h"
#include "DataStorage.h"
#include "Tensor.h"
#include "Layer.h"
#include "Linear.h"
#include "Loss.h"
#include "Sgd.h"
#include "RigL.h"
#include "DataLoader.h"
#include "Serialize.h"
#define NUM_SPEAKERS 8
#define EMBED_DIM 128
#define NUM_EPOCHS 20
#define BATCH_SIZE 16
#define LR 0.01f
#define RIGL_INTERVAL 100                    // Topology update every 100 steps
#define SPARSITY 0.9f                        // 90% sparse
```

```
int main(void) {
/* --- Memory pool --- */
static uint8_t pool[320*1024];               // 320 KB SRAM pool
dataStorageInit(pool, sizeof(pool));
/* --- Allocate weight tensors --- */
shape_t wShape = shape2d(NUM_SPEAKERS, EMBED_DIM);
float *wBuf = dataStorageAlloc(tensorBytesForShape(wShape, FLOAT32));
float *gBuf = dataStorageAlloc(tensorBytesForShape(wShape, FLOAT32));
float *vBuf = dataStorageAlloc(tensorBytesForShape(wShape, FLOAT32));   // Momentum
size_t mBytes = (NUM_SPEAKERS*EMBED_DIM+7)/8;
uint8_t *mBuf = dataStorageAlloc(mBytes);
memset(mBuf, 0xFF, mBytes);                  // All active initially
tensor_t *wTens = createTensor(wBuf,wShape,FLOAT32,1.0f,0);
tensor_t *gTens = createTensor(gBuf,wShape,FLOAT32,1.0f,0);
tensor_t *vTens = createTensor(vBuf,wShape,FLOAT32,1.0f,0);
```

```
tensor_t *mTens = createBoolTensor(mBuf,wShape);
/* Xavier init */
float bound = sqrtf(6.0f/(NUM_SPEAKERS+EMBED_DIM));
for (size_t i=0;i&amp;lt;NUM_SPEAKERS*EMBED_DIM;i++)
tensorSetF32(wTens,i,(rngFloat()*2-1)*bound);
fillTensor(gTens,0.0f); fillTensor(vTens,0.0f);
```

```
/* --- Build layer --- */
parameter_t wParam = {wTens, gTens, mTens};
linearConfig_t linCfg = {
.weights = &wParam, .bias = NULL,
.inFeatures = EMBED_DIM,
.outFeatures = NUM_SPEAKERS,
.useBias = false,
.weightMask = mTens
};
layer_t *classifier = createLinearLayer(&linCfg);
layer_t *model[] = {classifier};
/* --- SGD context --- */
sgdUpdateCtx_t sgdCtx = {
.lr = LR, .momentum = 0.9f,
.weightDecay = 1e-4f,
.velocity = vTens,
.weightMask = mTens
};
/* --- Training loop --- */
size_t step = 0;
size_t totalSteps = NUM_EPOCHS * (DATASET_SIZE/BATCH_SIZE);
for (size_t epoch = 0; epoch &amp;lt; NUM_EPOCHS; epoch++) {
dataLoaderShuffle(&loader);                // Shuffle each epoch
float epochLoss = 0.0f;
size_t correct = 0;
while (dataLoaderHasNext(&loader)) {
tensor_t *x = dataLoaderNextX(&loader);    // Input embedding
int label = dataLoaderNextY(&loader);      // Speaker label
/* Forward */
tensor_t *logits = layerForward(classifier, x);
float loss = crossEntropyLoss(logits, label);
epochLoss += loss;
if (argmax(logits)==label) correct++;
/* Backward */
tensor_t *dLogits = crossEntropyGrad(logits, label);
layerBackward(classifier, dLogits);
/* SGD update */
sgdStep(wTens, gTens, &sgdCtx);
fillTensor(gTens, 0.0f);                   // Zero gradients
```

```
/* RigL topology update */
if (++step % RIGL_INTERVAL == 0)
rigLStep(model, 1, SPARSITY,
step, totalSteps);
}
float acc = 100.0f*correct/DATASET_SIZE;
/* Log epoch result via ITM */
printf("Epoch %zu loss=%.4f acc=%.1f%%\n",
epoch, epochLoss/BATCH_SIZE, acc);
}
serializeModel(model, 1, "model.odts");  // Save checkpoint
return 0;
}
```

Key design decisions in this training loop: (1) fillTensor(gTens, 0.0f) after every sgdStep -- gradients are accumulated across the batch inside layerBackward, so they must be zeroed before the next batch. (2) rigLStep is called every RIGL_INTERVAL steps with the current step and totalSteps for cosine alpha decay. (3) The model is saved to SD card after training using serializeModel. (4) The DataStorage pool is allocated statically (not with malloc) to avoid heap fragmentation on the STM32.

■ **WARNING**	fillTensor(gTens, 0.0f) MUST be called after every optimizer step. Failing to zero gradients causes gradient accumulation across batches, effectively increasing the batch size geometrically and causing training instability or divergence.

Appendix E: Common Errors and Debugging Guide

HardFault on STM32

HardFault is the most common error when developing ODT on STM32. Causes and fixes:

1. Unaligned memory access: tensor_t data pointers must be 4-byte aligned for float32 access. dataStorageAlloc returns 4-byte aligned pointers (if the pool buffer is aligned). Check: ODT_ASSERT((uintptr_t)buf % 4 == 0, "misaligned"). Fix: declare pool with ALIGN4 attribute.

2. Stack overflow: rigLStep scratch array placed on stack. Fix: use dataStorageAlloc for scratch buffers larger than 256 bytes. Monitor stack high-water mark with the FreeRTOS uxTaskGetStackHighWaterMark() or by filling the stack with a magic pattern and scanning for the last non-magic word.

3. NULL pointer dereference: layerGetParam returns NULL if pi >= numParams. Check: always validate pi < layerNumParams before calling layerGetParam. The ODT_ASSERT in layerGetParam catches this in debug builds.

```
/* Stack overflow diagnosis (no RTOS) */
/* Fill stack canary at init time */
extern uint32_t _stack_base;                    // From linker script
memset(&_stack_base, 0xAB, CANARY_SIZE);        // Paint with pattern
/* At end of training, check: */
uint32_t *p = &_stack_base;
while (*p == 0xABABABAB) p++;                   // Find first non-canary
uint32_t usedStack = (uint32_t)(&_stack_top - p) * 4;
printf("Peak stack: %lu bytes\n", usedStack);
```

NaN or Inf in Gradients

NaN (Not a Number) or Inf in gradients causes the entire model to become NaN after one SGD step. Common causes: (1) Learning rate too large -- gradient explodes. (2) Log(0) in cross-entropy loss when a class logit is -Inf. (3) Division by zero in layer norm when the input variance is 0.

```
/* Gradient NaN check (debug mode) */
for (size_t i = 0; i &lt; N; i++) {
float g = tensorGetF32(grad, i);
if (!isfinite(g)) {                             // Catches NaN and Inf
printf("NaN at grad[%zu]\n", i);
ODT_ASSERT(0, "NaN gradient");
}
}
```

Fix for gradient explosion: gradient clipping. After backward pass, compute the global gradient norm and scale all gradients down if the norm exceeds a threshold (e.g., 1.0). This is the standard fix for RNN training but useful for any deep network with large gradients.

RigL Bugs and Their Signatures

Common RigL implementation bugs: (1) Active count changes after rigLStep: dropped != grown. Usually caused by the drop threshold and grow threshold being computed with different K values, or by off-by-one in the partial sort. Fix: add ODT_ASSERT(dropped == K && grown == K) inside rigLStep.

(2) Inactive weights become nonzero: the SGD update forgot to check the mask. Fix: add the mask check in sgdStep and verify with the RigL topology invariant test. (3) Accuracy does not improve with RigL: the grow step is growing connections with zero gradient (inactive weights whose input was 0). Fix: verify that input data has non-zero values and that the gradient computation path is correct.

ITM Printf for Embedded Debugging

```
/* retarget printf to ITM in syscalls.c */
int __io_putchar(int ch) {
ITM_SendChar((uint32_t)ch);                    // Send over SWD
return ch;
}
/* In STM32CubeIDE: Run &gt; Debug Configurations */
/* Enable SWO (Serial Wire Output) at 2 MHz */
/* Window &gt; Show View &gt; SWV &gt; SWV ITM Data Console */
/* Select ITM channel 0, start trace */
```

■ TIP ITM printf is much faster than UART printf for debug output during training: ITM uses the SWD interface (no CPU interrupt) and buffers through the trace hardware. At 2 MHz SWO: about 200,000 characters per second. A typical epoch log line of 50 characters takes 0.25 ms vs 5 ms for 115200-baud UART.

Appendix F: Line-by-Line Walkthrough — Tensor.c

createTensor — Annotated

createTensor is the constructor for all ODT tensor objects. It does not allocate memory -- it wraps an existing buffer. The caller is responsible for allocating buf with sufficient size (tensorBytesForShape(shape, dtype) bytes).

tensor_t *createTensor(void *buf, shape_t shape,	
dtype_t dtype, float scale, int32_t zeroPoint) {	
tensor_t *t = dataStorageAlloc(sizeof(tensor_t));	// Alloc struct from pool
ODT_ASSERT(t != NULL, "DataStorage full");	// Fail fast if pool exhausted
t->data = (uint8_t *)buf;	// Raw byte pointer to data
t->shape = shape;	// Copy shape struct by value
t->dtype = dtype;	// Set quantisation type
t->scale = scale;	// SYM/ASYM scale factor
t->zeroPoint = zeroPoint;	// ASYM zero point (0 for SYM)
/* Initialise transpose state */	
for (int d = 0; d < shape.ndim; d++)	
t->shape.orderOfDimensions[d] = d;	// Identity permutation
return t;	
}	

The orderOfDimensions field initialised to {0,1,...,ndim-1} represents the identity permutation -- no transpose. After transposeTensor(t, 0, 1): orderOfDimensions becomes {1,0}. Element access using shape index (i,j) maps to physical index via orderOfDimensions: physical = i * shape.dims[orderOfDimensions[1]] + j * 1.

tensorBoolGet / tensorBoolSet — Annotated

bool tensorBoolGet(const tensor_t *t, size_t i) {	
ODT_ASSERT(t->dtype == BOOL, "not BOOL tensor");	
size_t byte_idx = i >> 3;	// i / 8: which byte
size_t bit_idx = i & 7u;	// i % 8: which bit in byte
return (t->data[byte_idx] >> bit_idx) & 1u;	// Extract bit (LSB first)
}	
void tensorBoolSet(tensor_t *t, size_t i, bool v) {	
ODT_ASSERT(t->dtype == BOOL, "not BOOL tensor");	
size_t byte_idx = i >> 3;	// i / 8
size_t bit_idx = i & 7u;	// i % 8
if (v)	
t->data[byte_idx] |= (1u << bit_idx);	// Set bit to 1
else	
t->data[byte_idx] &= ~(1u << bit_idx);	// Clear bit to 0

```
}
```

Performance analysis: tensorBoolGet compiles to 4-5 ARM Thumb-2 instructions: LSR (right shift by 3 = divide by 8), AND with 7, LDRB (load byte), LSR (extract bit), AND with 1. On Cortex-M7 with D-cache hot: 4 cycles total. The LDRB instruction loads a byte from D-cache (0 wait states if cache hit). For N=32768 elements: 4 cycles * 32768 = 131,072 cycles = 0.61 ms just for mask scanning. This is why vectorised mask processing (loading a full byte and processing 8 bits) is critical for performance.

quantiseTensor — Annotated

```
void quantiseTensor(tensor_t *dst, const tensor_t *src,
roundingMode_t rm) {
ODT_ASSERT(src-&gt;dtype == FLOAT32, "src must be F32");
size_t N = tensorNumel(src);
/* Step 1: compute scale from abs max */
float absMax = tensorAbsMax(src, NULL);          // Max |value| in src
if (absMax &lt; 1e-8f) absMax = 1e-8f;           // Guard against div by zero
int32_t qMax = (1 &lt;&lt; (dst-&gt;qBits - 1)) - 1;   // e.g. 2047 for 12-bit
dst-&gt;scale = absMax / qMax;                   // Minimax optimal scale
/* Step 2: quantise each element */
for (size_t i = 0; i &lt; N; i++) {
float real = tensorGetF32(src, i);
float scaled = real / dst-&gt;scale;             // Divide by scale
if (rm == SR_HALF_AWAY) {                        // Stochastic rounding
scaled += rngFloat() - 0.5f;                     // Add U[-0.5, 0.5)
}
int32_t m = (int32_t)roundf(scaled);             // Round to nearest int
m = CLAMP(m, -qMax, qMax);                       // Saturate to valid range
tensorSetI32(dst, i, m);                         // Store mantissa
}
}
```

The two-pass approach (first find absMax, then quantise) requires two full scans over the tensor. An alternative one-pass approach with online max tracking: maintain running_max = 0, on each element update running_max = MAX(running_max, |real|), accumulate scaled values. But the scale depends on absMax, which is unknown until all elements have been seen -- so the one-pass approach requires storing scaled values with a provisional scale and rescaling at the end. The two-pass approach is simpler and the extra scan is O(N) which is acceptable.

■ WARNING If quantiseTensor is called with rm=ROUND_HALF_AWAY_FROM_ZERO (deterministic) on a gradient tensor, small gradients will be systematically rounded to zero. This creates dead weights that never receive gradient updates. Always use rm=SR_HALF_AWAY for gradient quantisation in FQT training.

transposeTensor — Zero-Copy Implementation

```
void transposeTensor(tensor_t *t, int dim0, int dim1) {
ODT_ASSERT(dim0 &lt; t-&gt;shape.ndim, "dim0 OOB");
```

```
ODT_ASSERT(dim1 &lt; t-&gt;shape.ndim, "dim1 OOB");
/* Swap dimension sizes */
size_t tmp = t-&gt;shape.dims[dim0];
t-&gt;shape.dims[dim0] = t-&gt;shape.dims[dim1];
t-&gt;shape.dims[dim1] = tmp;
/* Swap orderOfDimensions */
int otmp = t-&gt;shape.orderOfDimensions[dim0];
t-&gt;shape.orderOfDimensions[dim0] =
t-&gt;shape.orderOfDimensions[dim1];
t-&gt;shape.orderOfDimensions[dim1] = otmp;
}
```

transposeTensor is O(1): it swaps two integers in the shape struct. No data movement. The data buffer remains identical. All subsequent element accesses that use the 2D index (row, col) go through the orderOfDimensions mapping to find the physical offset. For a 2D tensor: physical_offset = orderOfDimensions[0]*dim1_size * row + orderOfDimensions[1] * col. After transposeTensor(t, 0, 1): orderOfDimensions = {1,0}, so physical = col*dim0_orig_size + row -- equivalent to accessing the original layout in column-major order.

Appendix G: Line-by-Line Walkthrough — Rng.c

XorShift32 State and Period

The XorShift32 generator maintains a single 32-bit state variable. The three-constant variant (13, 17, 5) is due to Marsaglia (2003) and has a period of 2^32 - 1 (all 32-bit values except 0 are visited exactly once before repeating). The state must be initialised to a non-zero value -- if state=0, all XOR operations produce 0 and the generator is stuck.

```
static uint32_t rng_state;                  // Module-level PRNG state
void rngInit(uint32_t seed) {
rng_state = (seed == 0) ? 1 : seed;         // Never allow state=0
}
uint32_t rngNext(void) {
uint32_t x = rng_state;                     // Load current state
x ^= x &lt;&lt; 13;                          // Shift-XOR mix: upper bits
x ^= x &gt;&gt; 17;                          // Shift-XOR mix: lower bits
x ^= x &lt;&lt; 5;                           // Shift-XOR mix: final avalanche
rng_state = x;                              // Save new state
return x;                                   // Return new random word
}
```

The three XOR-shift operations provide good statistical mixing: the left-shift-13 propagates lower bits into higher positions, the right-shift-17 propagates upper bits down, and the left-shift-5 provides a final avalanche. The combination passes the Diehard and TestU01 statistical test suites for most test cases.

On Cortex-M7: the three XOR-shift instructions compile to EOR (exclusive-OR with shifted register): EOR r0, r0, r0, LSL #13; EOR r0, r0, r0, LSR #17; EOR r0, r0, r0, LSL #5. Each EOR with shift executes in 1 cycle. Total: 3 cycles per rngNext() call.

rngFloat — Converting uint32 to [0,1)

```
float rngFloat(void) {
uint32_t raw = rngNext();                   // Raw random bits
/* Scale to [0,1) by dividing by 2^32 */
return (float)(raw &gt;&gt; 1) * (1.0f / (float)(1u &lt;&lt; 31));
}
```

Why right-shift by 1 before converting? Converting a 32-bit unsigned int to float: the float32 mantissa has 24 bits, so only the top 24 bits of raw are significant. The bottom 8 bits are lost in the conversion. By shifting right by 1, we use bits 31 down to 1 of raw, and divide by 2^31 to map to [0,1). An alternative: use the IEEE 754 trick of OR-ing the bits into the mantissa field of a float: (raw >> 9) | 0x3F800000 gives a float in [1,2), then subtract 1.0. This avoids the integer-to-float conversion and is 1 cycle faster.

rngBounded — Rejection Sampling

```
uint32_t rngBounded(uint32_t bound) {
/* Rejection sampling to avoid modulo bias */
uint32_t threshold = (-bound) % bound;  // 2^32 mod bound
uint32_t r;
do {
r = rngNext();
} while (r &lt; threshold);  // Reject biased values
return r % bound;
}
```

Why rejection sampling? Naive r % bound is biased when 2^32 is not divisible by bound. For bound=5: 2^32 = 4294967296 = 858993459*5 + 1. Values 0,1,2,3 appear 858993460 times and value 4 appears 858993459 times -- a tiny but nonzero bias. For Fisher-Yates shuffle correctness, even tiny bias accumulates over N shuffle steps. The rejection threshold = (2^32 mod bound) ensures only values in [threshold, 2^32) are accepted, which is exactly divisible by bound. Expected rejections: threshold/2^32 = (bound-1)/(2*bound) < 50%. Average calls per result: less than 2.

rngShuffle — Fisher-Yates Implementation

```
void rngShuffle(size_t *arr, size_t n) {
for (size_t i = n - 1; i &gt; 0; i--) {  // From last to second
size_t j = rngBounded((uint32_t)(i+1));  // j in [0, i]
size_t tmp = arr[i];
arr[i] = arr[j];  // Swap arr[i] and arr[j]
arr[j] = tmp;
}
}
```

Fisher-Yates produces uniformly random permutations: after iteration i, elements arr[i..n-1] form a uniformly random arrangement of the last n-i elements of the original array. After all n-1 iterations: every permutation of {0,...,n-1} is equally likely, with probability 1/n!. For n=1000 training samples: rngShuffle costs approximately 1000 * 2 (expected rngBounded calls) * 3 cycles = 6000 cycles = 0.03 ms. Negligible overhead.

Appendix H: Line-by-Line Walkthrough — DataLoader.c

DataLoader Internals

DataLoader.c manages the dataset iteration for training. It maintains: (1) a pointer to the dataset tensor (all samples), (2) a pointer to the labels array, (3) an index permutation array (for shuffled access), (4) a current position counter. The permutation array is initialised to {0,1,...,N-1} and shuffled at the start of each epoch.

```
typedef struct {
tensor_t *data;                    // All samples, shape [N, featDim]
int32_t *labels;                   // Label for each sample
size_t *perm;                      // Index permutation for shuffle
size_t N;                          // Total number of samples
size_t pos;                        // Current position in perm
size_t batchSize;
} dataLoader_t;
```

```
void dataLoaderShuffle(dataLoader_t *dl) {
rngShuffle(dl-&gt;perm, dl-&gt;N);          // Fisher-Yates permutation
dl-&gt;pos = 0;                            // Reset to start
}
bool dataLoaderHasNext(dataLoader_t *dl) {
return dl-&gt;pos + dl-&gt;batchSize &lt;= dl-&gt;N;   // At least one batch remains
}
tensor_t *dataLoaderNextX(dataLoader_t *dl) {
/* View into dataset at perm[pos] */
size_t idx = dl-&gt;perm[dl-&gt;pos];          // Shuffled index
return tensorSlice(dl-&gt;data, idx);       // 1D slice of row idx
}
int dataLoaderNextY(dataLoader_t *dl) {
int label = dl-&gt;labels[dl-&gt;perm[dl-&gt;pos]];
dl-&gt;pos++;                              // Advance position
return label;
}
```

tensorSlice returns a view (not a copy) of one row of the dataset tensor. It creates a new tensor_t struct (allocated from DataStorage) pointing into the middle of the dataset buffer. The view's data pointer = &dataset-;>data[idx * featDim * bytesPerElement]. This zero-copy approach avoids copying each sample during iteration.

Memory lifetime: tensorSlice allocates a tensor_t struct from DataStorage. After iterating the entire epoch (N calls to dataLoaderNextX), the DataStorage pool has N extra tensor_t structs. If dataStorageReset() is called between epochs, these are reclaimed. But if the training loop does not reset DataStorage, the pool gradually fills up. Design decision: reset DataStorage after each epoch but before the next epoch's shuffle, since all previous activation tensors (from the completed epoch) are no longer needed.

NPYLoader: Loading .npy Files on STM32

NumPy .npy format: a magic string (\x93NUMPY), version (1 or 2), header length (2 or 4 bytes), then a Python dict literal as ASCII text specifying dtype, fortran_order, and shape. After the header, raw array data follows in C or Fortran order. NPYLoader parses this format via fread on an open FatFS file.

`/* NPY header parsing (simplified) */`	
`uint8_t magic[6];`	
`fread(magic, 1, 6, fp);`	`// Read magic + version`
`/* magic should be 0x93 0x4E 0x55 0x4D 0x50 0x59 */`	
`uint16_t headerLen;`	
`fread(&headerLen, 2, 1, fp);`	`// Header dict length`
`char header[512];`	`// Stack buffer for header text`
`fread(header, 1, headerLen, fp);`	`// Read ASCII dict`
`/* Parse dtype string e.g. "float32" or "int32" */`	
`/* Parse shape tuple e.g. (1000, 128) */`	
`/* Parse fortran_order: always False for ODT */`	
`/* Read raw data */`	
`fread(tensor-&gt;data, 1, tensorBytes(tensor), fp);`	`// Load data directly into tensor buffer`

The critical optimisation: fread into tensor->data directly from the SD card. No intermediate copy buffer needed. FatFS buffers the SD card reads internally (512-byte sector cache). For a 1000-sample x 128-feature dataset: 1000 * 128 * 4 = 512,000 bytes. At SD card read speed of 1 MB/s (typical for SDv1 cards): 0.5 seconds to load. At 10 MB/s (SDv2): 0.05 seconds. This is a one-time cost at startup.

Appendix I: Line-by-Line Walkthrough — Loss Functions and Activations

Softmax and Numerical Stability

Softmax: out_i = exp(x_i) / sum_j exp(x_j). The naive implementation suffers from numerical overflow for large x_i (exp(88) = 1.65e38, near float32 max). The standard fix: subtract the maximum logit before exponentiating. softmax(x)_i = exp(x_i - max_x) / sum_j exp(x_j - max_x). This is equivalent (numerator and denominator both divided by exp(max_x)) but numerically stable.

```
void softmaxForward(tensor_t *out, const tensor_t *in) {
size_t N = tensorNumel(in);
float max_val = tensorGetF32(in, 0);
for (size_t i=1;i&amp;lt;N;i++)
if (tensorGetF32(in,i)&amp;gt;max_val)
max_val = tensorGetF32(in,i);                    // Find max logit
float sum = 0.0f;
for (size_t i=0;i&amp;lt;N;i++) {
float e = expf(tensorGetF32(in,i)-max_val);      // Stable exp
tensorSetF32(out,i,e);                           // Store unnormalised
sum += e;
}
for (size_t i=0;i&amp;lt;N;i++)
tensorSetF32(out,i,tensorGetF32(out,i)/sum);     // Normalise
}
```

expf on Cortex-M7 FPU: the VFP instruction set does not have a hardware expf instruction. expf is computed in software by the ARM math library (CMSIS-DSP or newlib). Typical latency: 20-50 cycles. For N=8 classes: 8 * 35 cycles = 280 cycles = 1.3 us. Negligible for a single inference call.

Cross-Entropy Loss

Cross-entropy loss: L = -log(softmax(x)[label]) = -log(exp(x_label - max_x) / sum_j exp(x_j - max_x)) = -(x_label - max_x) + log(sum_j exp(x_j - max_x)). This is the log-sum-exp formulation, numerically identical to applying softmax then log, but computed in one pass.

```
float crossEntropyLoss(const tensor_t *logits, int label) {
size_t N = tensorNumel(logits);
float max_val = tensorGetF32(logits, 0);
for (size_t i=1;i&amp;lt;N;i++)
max_val = fmaxf(max_val, tensorGetF32(logits,i));
float sum_exp = 0.0f;
for (size_t i=0;i&amp;lt;N;i++)
sum_exp += expf(tensorGetF32(logits,i)-max_val);
float log_sum = logf(sum_exp) + max_val;         // Log-sum-exp
```

```
float label_logit = tensorGetF32(logits, label);
return log_sum - label_logit;                          // Cross-entropy
}
```

Cross-Entropy Backward Pass

The gradient of cross-entropy loss with respect to logits: dL/dx_i = softmax(x)_i - (i == label ? 1 : 0). This is the softmax output minus the one-hot label. This elegant formula is the reason cross-entropy + softmax is always paired: their combined gradient is much simpler than either gradient separately.

```
tensor_t *crossEntropyGrad(const tensor_t *logits, int label)
{
size_t N = tensorNumel(logits);
tensor_t *sm = allocActivation(N);
softmaxForward(sm, logits);                            // Compute softmax
/* Subtract 1.0 from the true class */
float v = tensorGetF32(sm, label);
tensorSetF32(sm, label, v - 1.0f);                     // dL/dx_label -= 1
return sm;                                             // Gradient of loss w.r.t. logits
}
```

ReLU Forward and Backward

```
void reluForward(tensor_t *out, const tensor_t *in) {
size_t N = tensorNumel(in);
for (size_t i=0;i&amp;lt;N;i++) {
float v = tensorGetF32(in,i);
tensorSetF32(out,i, v &amp;gt; 0.0f ? v : 0.0f);                // max(0, x)
}
}
void reluBackward(tensor_t *grad_in, const tensor_t *in,
const tensor_t *grad_out) {
size_t N = tensorNumel(in);
for (size_t i=0;i&amp;lt;N;i++) {
float mask = tensorGetF32(in,i) &amp;gt; 0.0f ? 1.0f : 0.0f;     // STE for ReLU
float go = tensorGetF32(grad_out,i);
tensorSetF32(grad_in,i, mask * go);                    // Straight-through
}
}
```

ReLU backward uses the straight-through estimator: gradient is passed through for positive inputs (the gate is open) and blocked for negative inputs (the gate is closed). The saved input tensor from the forward pass is required for the backward pass -- this is why Linear.c saves cfg->input = input before calling the matmul kernel.

Batch Normalisation Backward (Layer Norm)

Layer normalisation normalises each sample independently (unlike batch norm which normalises across the batch). Forward: y = (x - mean(x)) / sqrt(var(x) + eps) * gamma + beta. Backward: the gradient of layer norm is complex but the key insight is that dL/dx depends on both dL/dy and x (via the normalisation factor).

For embedded inference (forward pass only, no training of norm parameters): layer norm reduces to a simple per-sample mean subtraction and variance normalisation. The gamma and beta parameters can be absorbed into the subsequent linear layer weight and bias: W_eff = W * gamma / std, b_eff = b + W * (beta - mean*gamma/std). This fusion eliminates the layer norm from the inference path entirely.

■ **TIP**	Layer norm fusion for inference: pre-compute W_eff and b_eff once during model export. The merged linear layer performs W_eff*x + b_eff in one matmul -- no separate normalisation step. This reduces one full layer of O(N) operations to zero during inference.

Appendix J: AdamW Deep Dive — Algorithm, Derivation, and Code

Adam Algorithm: Intuition and Derivation

Adam (Adaptive Moment Estimation) was introduced by Kingma and Ba (2015). The core idea: maintain a per-parameter exponentially weighted moving average (EMA) of gradients (first moment m) and squared gradients (second moment v). Use these statistics to adapt the learning rate for each parameter individually.

First moment (mean): m_t = beta1 * m_{t-1} + (1-beta1) * g_t. Interprets as: "where is the gradient pointing on average?" High beta1 = more momentum, smoother trajectory. Default beta1=0.9: each step is 10% current gradient, 90% accumulated history.

Second moment (variance): v_t = beta2 * v_{t-1} + (1-beta2) * g_t^2. Interprets as: "how variable is the gradient?" High beta2 = slow adaptation to gradient variance. Default beta2=0.999: each step is 0.1% current squared gradient, 99.9% history.

Effective learning rate: lr / sqrt(v_t) * m_t. For parameters with consistent, large gradients: v_t is large -> small effective lr (prevents overshooting). For parameters with small, noisy gradients: v_t is small -> large effective lr (accelerates learning). This is the "adaptive" in Adam.

Bias correction: at step t, m_t = (1-beta1^t)^{-1} * (true mean) approximately. Similarly for v_t. Without correction, early steps have m_t close to 0 (no history) causing tiny updates. Bias correction: m_hat = m_t / (1 - beta1^t), v_hat = v_t / (1 - beta2^t). After t=100 steps with beta1=0.9: 1-0.9^100 = 1-2.66e-5 ≈ 1.0. Correction is negligible. At t=1: 1-0.9 = 0.1. Correction factor = 10x -- critical for the first few steps.

AdamW: Decoupled Weight Decay

Standard Adam applies weight decay as gradient regularisation: g_eff = g + wd*w. This means weight decay is adapted by the second moment v_t: the effective L2 penalty is wd*w / sqrt(v_t). For parameters with large gradients (large v_t): the effective weight decay is small. For parameters with small gradients (small v_t): the effective weight decay is large. This coupling is generally undesirable -- weight decay should penalise weight magnitude uniformly.

AdamW (Loshchilov and Hutter 2019) decouples weight decay from the gradient update: w_t = (1 - wd*lr) * w_{t-1} - lr * m_hat_t / sqrt(v_hat_t + eps). The weight decay term (1-wd*lr) is applied directly to the weight, not to the gradient. This is equivalent to L2 regularisation in SGD but not in Adam (where the gradient is scaled by 1/sqrt(v)). AdamW generally outperforms Adam+L2 for neural network training.

```
/* AdamW.c -- one parameter update step */
void adamwStep(tensor_t *param, tensor_t *grad,
adamwUpdateCtx_t *cfg) {
size_t N = tensorNumel(param);
double b1t = cfg-&gt;beta1_t;          // beta1^t (double precision)
double b2t = cfg-&gt;beta2_t;          // beta2^t (double precision)
double corr1 = 1.0 - b1t;             // 1 - beta1^t
double corr2 = 1.0 - b2t;             // 1 - beta2^t
float lr = cfg-&gt;lr;
float wd = cfg-&gt;weightDecay;
```

```
float eps = (float)cfg-&gt;eps;
float b1 = cfg-&gt;beta1;
float b2 = cfg-&gt;beta2;
tensor_t *mask = cfg-&gt;weightMask;                    // NULL = dense
for (size_t i = 0; i &lt; N; i++) {
if (mask && !tensorBoolGet(mask, i)) {                 // Skip inactive
tensorSetF32(cfg-&gt;m, i, 0.0f);                      // Zero stale m
tensorSetF32(cfg-&gt;v, i, 0.0f);                      // Zero stale v
continue;
}
float g = tensorGetF32(grad, i);                       // Gradient
float w = tensorGetF32(param, i);                      // Current weight
float m = b1*tensorGetF32(cfg-&gt;m,i)+(1-b1)*g;       // Update m
float v = b2*tensorGetF32(cfg-&gt;v,i)+(1-b2)*g*g;     // Update v
tensorSetF32(cfg-&gt;m, i, m);                         // Save m
tensorSetF32(cfg-&gt;v, i, v);                         // Save v
float mh = m / (float)corr1;                           // Bias-correct m
float vh = v / (float)corr2;                           // Bias-correct v
/* AdamW decoupled weight decay */
w = w*(1.0f-wd*lr) - lr*mh/(sqrtf(vh)+eps);
tensorSetF32(param, i, w);
}
/* Advance bias correction */
cfg-&gt;beta1_t *= cfg-&gt;beta1;                      // beta1^(t+1)
cfg-&gt;beta2_t *= cfg-&gt;beta2;                      // beta2^(t+1)
}
```

The double-precision beta1_t and beta2_t are critical. At step t=1000: beta1_t = 0.9^1000. In float32: 0.9^1000 = 2.66e-46 which underflows to 0.0f. Then corr1 = 1.0 - 0.0 = 1.0, disabling bias correction. In double: 0.9^1000 = 2.66e-46 is representable (double min = 2.23e-308). Bias correction remains accurate throughout training.

Zeroing m and v for inactive weights: when a weight is deactivated by RigL (mask cleared), its moment estimates m and v may be large (from active training). When RigL grows the weight back (mask set), the stale m/v would cause an incorrect large update. Zeroing ensures newly grown weights start with m=0, v=0, receiving a pure gradient signal on their first active step.

Learning Rate Schedulers

ODT provides three learning rate schedulers: (1) Constant: lr stays fixed throughout training. Simple but often suboptimal -- early training benefits from large lr (fast convergence), late training benefits from small lr (fine-tuning). (2) Step decay: lr = lr_initial * gamma^(step // step_size). Reduces lr by gamma every step_size steps. Common in image classification. (3) Cosine annealing: lr = lr_min + 0.5*(lr_max - lr_min)*(1 + cos(pi*step/total)). Smooth decay from lr_max to lr_min over training.

```
float lrSchedulerGet(lrScheduler_t *sch, size_t step) {
switch (sch-&gt;type) {
case CONSTANT:
```

```
return sch-&gt;lr_initial;
case STEP_DECAY:
return sch-&gt;lr_initial
* powf(sch-&gt;gamma, step / sch-&gt;step_size);
case COSINE_ANNEALING:
return sch-&gt;lr_min + 0.5f*(sch-&gt;lr_max-sch-&gt;lr_min)
* (1.0f + cosf(M_PI*step/sch-&gt;total));
default:
return sch-&gt;lr_initial;
}
}
```

Cosine annealing is recommended for RigL training because: (1) the high initial lr drives exploration of the loss landscape and supports aggressive topology changes; (2) the decaying lr allows fine-tuning of the final sparse topology; (3) the smooth decay avoids the abrupt accuracy drops that step decay can cause. The alpha decay schedule in rigLStep is also cosine-shaped, so both the topology and learning rate decay synchronously.

Appendix K: Conv1d and Conv1dTransposed Deep Dive

Conv1d Forward Pass

1D convolution for audio/time-series: input shape [inChannels, length], kernel shape [outChannels, inChannels, kernelSize], output shape [outChannels, outLength] where outLength = (length - kernelSize) / stride + 1. The convolution computes: out[c_out, t] = sum_{c_in} sum_{k} kernel[c_out, c_in, k] * input[c_in, t*stride + k].

```
/* Conv1d forward (simplified, no padding) */
for (size_t co=0; co&amp;lt;outCh; co++) {  // Output channel
for (size_t t=0; t&amp;lt;outLen; t++) {  // Time step
float acc = bias[co];  // Start with bias
for (size_t ci=0; ci&amp;lt;inCh; ci++) {  // Input channel
for (size_t k=0; k&amp;lt;kSize; k++) {  // Kernel position
acc += kernel[co][ci][k]
* input[ci][t*stride+k];
}
}
output[co][t] = acc;
}
}
```

The four-loop structure has complexity O(outCh * outLen * inCh * kSize). For a typical audio model: outCh=64, outLen=100, inCh=1, kSize=16: 102,400 MACs. At 216 MHz: 0.47 ms. Much smaller than a dense linear layer, since the kernel is much smaller than the input.

im2col optimisation: reshape the input into a matrix of shape [inCh*kSize, outLen], then the convolution becomes a matmul: output = kernel_matrix @ input_col, where kernel_matrix shape is [outCh, inCh*kSize]. This allows using the optimised matmul kernel. Memory cost: the im2col buffer requires inCh * kSize * outLen * 4 bytes. For above: 1*16*100*4 = 6400 bytes -- acceptable from DataStorage.

Conv1dTransposed (Upsampling)

Conv1dTransposed (transposed convolution or "deconvolution") is the gradient of Conv1d with respect to the input. It upsamples: input shape [inChannels, length], kernel shape [outChannels, inChannels, kernelSize], output shape [outChannels, length*stride]. Used in autoencoders (decoder) and generative models (PPCA noise injection).

Forward pass: for each input element input[ci, t], add kernel[co, ci, k] * input[ci, t] to output[co, t*stride + k] for all co and k. This is the "scatter" operation (vs Conv1d's "gather"). The output elements receive contributions from multiple input positions -- they must be accumulated (+=), so the output buffer must be pre-zeroed.

```
/* Conv1dTransposed forward */
fillTensor(output, 0.0f);  // Must zero before accumulation
for (size_t ci=0; ci&amp;lt;inCh; ci++) {
for (size_t t=0; t&amp;lt;inLen; t++) {
```

```
for (size_t co=0; co&lt;outCh; co++) {
for (size_t k=0; k&lt;kSize; k++) {
output[co][t*stride+k] +=
kernel[co][ci][k] * input[ci][t];
}
}
}
}
```

Global Average Pooling

Global average pooling reduces a feature map of shape [channels, length] to [channels]: out[c] = mean(input[c, :]). Used to collapse the temporal dimension before the final classifier. Backward: the gradient is distributed uniformly: d_input[c, t] = d_out[c] / length for all t.

```
/* Global average pooling forward */
for (size_t c=0; c&lt;channels; c++) {
float sum = 0.0f;
for (size_t t=0; t&lt;length; t++)
sum += input[c*length+t];
output[c] = sum / length;
}
```

Global average pooling is parameter-free and extremely fast: O(channels * length) additions plus one division per channel. For channels=64, length=100: 6400 additions + 64 divisions = 6464 operations = 30 us at 216 MHz. The lack of parameters means no weight storage and no gradient computation for pooling weights -- only the gradient w.r.t. the input is needed for backpropagation.

Appendix L: Memory Layout Diagram and Allocation Guide

STM32F746ZG Full Memory Map

The complete memory map of the STM32F746ZG as relevant to ODT training:

```
Address Range Size Name Usage
0x00000000- 512B ITCM Boot code, vector table alias
0x08000000- 1MB Flash Compiled ODT code, .rodata
0x20000000- 64KB DTCM C stack, hot activation buffers
0x20010000- 256KB AXI SRAM DataStorage pool, weight tensors
0x40000000- Periph Bus UART, SPI, SDMMC registers
0xE0001000- DWT Cycle counter
0xE0000000- ITM Printf semihosting
```

Flash (.text + .rodata): all compiled ODT code resides here. The ARM ART accelerator provides 0 wait-state access up to 216 MHz via a pre-fetch buffer. The weight tensors and dataset are NOT stored in Flash (they change during training). Only the model checkpoint on the SD card uses FatFS -- never directly mapped into the memory map.

DataStorage Allocation Order

The recommended allocation order from the 256 KB AXI SRAM DataStorage pool. Allocate in this order at startup (before any dynamic allocations):

```
/* 1. Weight tensors (static, persist forever) */
w1_buf = alloc(outFeat1*inFeat1 * sizeof(float));  // ~64KB for 128x128
w2_buf = alloc(outFeat2*inFeat2 * sizeof(float));
/* 2. Gradient tensors (same size as weights) */
g1_buf = alloc(outFeat1*inFeat1 * sizeof(float));
/* 3. Momentum/variance buffers (Adam: 2x weights) */
m1_buf = alloc(outFeat1*inFeat1 * sizeof(float));
v1_buf = alloc(outFeat1*inFeat1 * sizeof(float));
/* 4. Mask tensors (1/32 the size of weights) */
mask_buf = alloc((N+7)/8);  // ~2KB for N=16384
/* 5. Dataset buffer (loaded from SD card) */
dataset_buf = alloc(N_samples * featDim * sizeof(float));
/* Remaining space for activations (forward pass) */
/* -- allocated and freed each forward-backward pass -- */
```

Typical memory budget for a speaker ID model (128->64->8 architecture, 90% sparse, 20 speakers, 1000 training samples):

```
Weights 128*64*4 + 64*8*4 = 34816 bytes 34 KB
Gradients same as weights = 34816 bytes 34 KB
Moments 2x weights (Adam) = 69632 bytes 68 KB
```

```
Masks (128*64+64*8+7)/8 = 1088 bytes 1 KB
Dataset 1000 * 128 * 4 = 512000 bytes 500 KB
--- Dataset exceeds AXI SRAM! Use streaming from SD card ---
Activations 128 + 64 + 8 (x3 for FW+BW) = ~600 bytes 1 KB
PPCA models 20 * (128*8 + 128 + 1) * 4 = 82880 bytes 81 KB
```

■ WARNING The 1000-sample dataset at 500 KB exceeds the 256 KB AXI SRAM. Solution: stream samples from SD card in mini-batches. Load one batch (e.g., 32 samples = 16 KB) at a time, process (forward+backward+sgd), then load the next batch. This eliminates the need for full dataset in SRAM but increases SD card read frequency.

Linker Script Configuration

The linker script STM32F746ZGTx_FLASH.ld defines the memory regions and section placement. Key sections to understand for ODT:

```
MEMORY {
RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 320K                  // All SRAM
DTCM (xrw) : ORIGIN = 0x20000000, LENGTH = 64K                  // DTCM first
AXISRAM(xrw): ORIGIN = 0x20010000, LENGTH = 256K                // AXI SRAM
FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 1024K                // Flash
}
SECTIONS {
.text : { *(.text) *(.text*) } &gt; FLASH                       // Code in Flash
.data : { *(.data) } &gt; AXISRAM AT &gt; FLASH                 // Init data in AXI SRAM
.bss : { *(.bss) } &gt; AXISRAM                                 // BSS in AXI SRAM
._stack : { . = ALIGN(8); . += 0x2000; } &gt; DTCM              // 8 KB stack in DTCM
}
```

The DataStorage pool should be declared as a global uint8_t array in .bss (uninitialised, goes to AXI SRAM). The C stack (8 KB in the example) is placed in DTCM for zero-wait-state stack access. Increasing stack size: change 0x2000 to 0x4000 (16 KB) if rigLStep scratch arrays or deep recursion require more stack space.

Appendix M: Speaker Identification Pipeline on STM32

Speaker ID System Architecture

The speaker ID system on STM32 Nucleo-F746ZG consists of five pipeline stages: (1) Audio capture via I2S microphone (or simulated from SD card samples). (2) MFCC feature extraction: framing, windowing, FFT, mel filterbank, DCT. (3) Embedding extraction: forward pass through a trained encoder network producing a 128-dimensional speaker embedding. (4) Classification: dot product of embedding with stored speaker centroids (or the ODT sparse linear classifier). (5) Continual registration: when a new speaker is enrolled, PPCA is fitted to their embeddings.

Audio capture: the STM32F746ZG Discovery board includes an onboard MEMS microphone (MP34DT01) connected via the I2S/SAI interface. Sampling rate: 16 kHz (standard for speech). Frame size: 25 ms * 16000 = 400 samples. Frame shift: 10 ms = 160 samples. This gives approximately 100 frames per second of audio.

```
/* I2S audio capture (simplified) */
#define SAMPLE_RATE 16000                    // 16 kHz
#define FRAME_SIZE 400                       // 25 ms
#define FRAME_SHIFT 160                      // 10 ms
#define N_MFCC 40                            // 40 MFCC coefficients
static int16_t audio_buf[FRAME_SIZE];        // Capture buffer
static float mfcc_buf[N_MFCC];               // MFCC output
/* DMA circular buffer for continuous capture */
HAL_I2S_Receive_DMA(&hi2s1, audio_buf, FRAME_SIZE);  // Start DMA
/* Process each frame when DMA half/full complete */
```

MFCC Feature Extraction

Mel-Frequency Cepstral Coefficients (MFCCs) are the standard features for speech and speaker recognition. Computation steps: (1) Pre-emphasis: x[n] = x[n] - 0.97*x[n-1]. Boosts high frequencies. (2) Framing: extract overlapping frames of 400 samples. (3) Hamming window: multiply by w[n] = 0.54 - 0.46*cos(2*pi*n/(N-1)). Reduces spectral leakage. (4) FFT: 512-point FFT (real input, hermitian output). (5) Magnitude spectrum: |X[k]|^2. (6) Mel filterbank: 40 triangular filters spaced on mel scale. (7) Log: log(filterbank_energy + 1e-8). (8) DCT: 40-point DCT-II, keep first 40 coefficients.

CMSIS-DSP FFT on Cortex-M7: arm_rfft_fast_f32() computes a real 512-point FFT in approximately 2000 cycles = 9 us at 216 MHz. The full MFCC pipeline (pre-emphasis + FFT + filterbank + log + DCT) takes approximately 50,000 cycles = 0.23 ms per frame. For 100 frames per second: 0.23 ms * 100 = 23 ms per second of audio, leaving 977 ms per second for inference and training.

Mel scale: mel(f) = 2595 * log10(1 + f/700). Humans perceive pitch logarithmically at low frequencies and linearly at high frequencies. The mel scale captures this by being approximately linear below 1 kHz and logarithmic above 1 kHz. Triangular filterbank on the mel scale: filter k spans [f_low_k, f_center_k, f_high_k] on the mel scale, linearly ramping from 0 to 1 to 0. For 40 filters spanning 0-8000 Hz: first few filters are very narrow (50-100 Hz wide) and later filters are wide (hundreds of Hz).

Speaker Embedding Network Architecture

The encoder network maps variable-length audio (sequence of MFCCs) to a fixed-length 128-dimensional embedding. For the STM32 deployment, the architecture is: Conv1d(40, 64, kernel=16, stride=4) -> ReLU -> Conv1d(64, 128, kernel=8, stride=2) -> ReLU -> Global Average Pooling -> Linear(128, 128) -> L2-normalise.

L2 normalisation: embedding = embedding / ||embedding||_2. This maps all embeddings to the unit hypersphere. Speaker ID then becomes a nearest-centroid problem on the sphere, equivalent to maximum cosine similarity. Cosine similarity is independent of magnitude, so L2-normalised embeddings remove the effect of speaking volume on the embedding.

```
/* L2 normalisation (in-place) */
float norm_sq = 0.0f;
for (size_t i = 0; i &lt; 128; i++)
norm_sq += emb[i] * emb[i];
float inv_norm = 1.0f / sqrtf(norm_sq + 1e-8f);
// Guard against divide-by-zero
for (size_t i = 0; i &lt; 128; i++)
emb[i] *= inv_norm;
```

Classification: for K registered speakers, maintain centroids c_k = mean of all embeddings for speaker k (updated as new enrollments arrive). For a new utterance embedding e: predicted speaker = argmax_k (e . c_k). Since both e and c_k are L2-normalised, e . c_k = cos(angle between e and c_k). The argmax finds the speaker whose centroid is most cosine-similar to the new embedding.

Speaker Registration Procedure

To register a new speaker: (1) Collect 10-20 utterances of diverse content from the new speaker. (2) For each utterance: extract MFCCs, compute encoder embedding. (3) L2-normalise each embedding. (4) Fit PPCA to the collected embeddings (PpcaReplay.c). (5) Compute centroid = mean of embeddings. (6) Store PPCA model and centroid in the speaker database (SRAM or SD card). (7) Fine-tune the classifier with the new speaker's embeddings + PPCA replay from old speakers.

Minimum enrollment utterances: 5 utterances is typically sufficient for a reliable centroid estimate. 20 utterances allows fitting a meaningful PPCA model (q=4 principal components from 20 points in 128D space). Below 5 utterances, the centroid estimate is noisy and the PPCA model is underdetermined.

■ **TIP**	Phonetically balanced text for enrollment: ask new speakers to read specific sentences covering all phonemes (e.g., "The quick brown fox jumps over the lazy dog"). This ensures the embedding covers the full range of that speaker's voice characteristics, leading to a more representative centroid.

Appendix N: Frequently Asked Questions

Q: Why does ODT use float32 weights rather than int8?

A: ODT uses float32 weights during training because: (1) Gradient updates (SGD/AdamW) require high precision -- small weight changes (lr * gradient) must accumulate over thousands of steps without rounding to zero. (2) The moment estimates in AdamW require approximately the same precision as the gradients. (3) The stochastic rounding in quantiseTensor handles the quantisation noise more gracefully when the underlying weights are in float32. For inference-only deployment, weights would be stored as INT8 or INT16 and dequantised on the fly.

Q: What is the difference between SYM and SYM_INT32?

A: Both use symmetric quantisation (zero_point=0, scale = absMax / qMax). The difference is the data type used to store mantissas: SYM stores mantissas as compact integers (e.g., int8 for 8-bit SYM), SYM_INT32 stores mantissas as int32 regardless of bit width. SYM_INT32 is used when the intermediate accumulator in matmul would overflow smaller integer types, or when the computing unit natively uses 32-bit integers (like Cortex-M7 without SIMD). SYM is used for compact storage (one int8 per element vs four bytes for int32).

Q: Why is the BOOL tensor 1-indexed from LSB?

A: Element 0 is stored in bit 0 (LSB) of byte 0, element 1 in bit 1, etc. This LSB-first (little-endian bit order) convention matches the NumPy packed bit format when creating boolean arrays with numpy.packbits(order="little"). Consistency with NumPy allows saving/loading masks as .npy files without bit-reversal. The alternative (MSB-first) is also common but would require a bit-reversal when interfacing with NumPy.

Q: Why does rigLStep use selection sort instead of quicksort?

A: For the K-th smallest element problem, selection sort of K elements from N is O(N*K). Quickselect is O(N) expected but O(N^2) worst case and requires partitioning in-place (modifying the input array). Since the weight array must not be modified during the scan, quickselect would need a copy of all N values -- requiring N*4 bytes of scratch. Selection sort maintains only K values in the scratch buffer, costing K*4 bytes. For K = alpha*numActive = 0.1 * 3277 = 328 values and N=32768: selection sort costs 328*32768 = 10.7 million comparisons; quickselect costs ~32768 comparisons. The performance gap is significant for large layers -- consider quickselect for N > 4096.

Q: Can ODT train on multiple tasks simultaneously (multi-task learning)?

A: Not in the current implementation. ODT trains one task at a time, with PPCA replay for continual learning across sequential tasks. Multi-task learning would require: (1) Multiple loss functions (one per task) summed with weighting factors. (2) Multiple label arrays in the DataLoader. (3) A task identifier passed to each forward pass. This is architecturally straightforward to add but not currently implemented.

Q: How does ODT handle class imbalance?

A: Not explicitly. If one speaker has far more training samples than others, the model will be biased toward that speaker. Mitigations: (1) Oversample under-represented speakers in the DataLoader (sample with replacement from small classes). (2) Use weighted cross-entropy: weight each sample's loss by 1/class_count_k. (3) PPCA replay automatically balances: generate equal numbers of synthetic samples per old speaker regardless of original enrollment count.

Q: Why is there no GPU acceleration for on-device training?

A: STM32 microcontrollers do not have GPUs. The Cortex-M7 FPU provides single-precision floating-point at up to 1 GFLOP/s (1 FMA/cycle * 216 MHz * 2 ops/FMA = 432 MFLOPS). A mobile GPU (e.g., Mali-G52) provides 60 GFLOPS -- 140x faster. However, mobile GPUs require a full application processor SoC (e.g., STM32MP1 or NXP i.MX RT1170) which consumes orders of magnitude more power than the Cortex-M7. ODT targets ultra-low-power always-on training scenarios where a GPU SoC is impractical.

Q: What is the inference latency for a full speaker ID prediction?

A: For a 128->64->8 model at 90% sparsity on STM32F746ZG at 216 MHz with optimised sparse matmul: MFCC extraction: 0.23 ms/frame. Encoder forward (Conv1d+GlobalPool): ~0.3 ms. Classifier forward (128->8 sparse linear): ~0.02 ms. L2 normalisation: ~0.01 ms. Centroid dot products (8 speakers): ~0.01 ms. Total: ~0.56 ms per frame, or approximately 2 ms for a decision averaged over a 20ms window. This is well within real-time requirements for a speaker ID system.

Q: How to verify the RigL implementation is correct?

A: Four verification steps: (1) Topology invariant test: active count unchanged after rigLStep (dropped == grown == K). (2) Zero check: all inactive weights are exactly 0.0f. (3) Gradient guided growth: manually verify the K grown connections have the largest |gradient| among inactive connections. (4) Accuracy comparison: run the same model in PyTorch RigL and ODT RigL from the same starting weights; accuracy should be within 2% after the same number of topology updates.

Q: What causes "DataStorage full" assertion failure?

A: dataStorageAlloc returns NULL when the pool is exhausted. ODT_ASSERT catches this and halts. Causes: (1) Pool size too small for the model and dataset. (2) Activation tensors not freed between batches (if using a pooling rather than bump allocator). (3) tensor_t structs for each batch sample accumulated without reset. Fix: call dataStorageReset at the end of each epoch to reclaim all temporary allocations, then re-allocate the static weight/gradient/momentum tensors at the start of the next epoch -- or better, allocate all long-lived tensors before training begins and only use DataStorage for ephemeral activations during the forward-backward pass.

Q: How to add a new layer type to ODT?

A: Follow these steps: (1) Create NewLayer.h with the config struct and function declarations. (2) Create NewLayer.c with newLayerForwardFn(), newLayerBackwardFn(), newLayerZeroGradFn(), newLayerGetParamFn(). (3) Add a new enum value to layer_type_t in Layer.h. (4) Register the new layer in

createLayer() in Layer.c: add a case to the switch statement setting layer->forward, layer->backward, etc. (5) Add NewLayer.c to CMakeLists.txt. (6) Add the new layer type to Serialize.c (write layer_type as uint32) and Deserialize.c (read layer_type and call the correct createLayer function).

Appendix O: ExecuteOp Dispatch Engine — Complete Analysis

What ExecuteOp Does

ExecuteOp is the central dispatch and type-management engine in ODT. All tensor operations (matmul, elementwise add, reduce, etc.) flow through ExecuteOp rather than being called directly. Its responsibilities: (1) Convert input tensors to the required computation dtype if they do not match. (2) Allocate a scratch output buffer of the correct size and dtype. (3) Dispatch to the appropriate typed kernel (e.g., matmulInt32Core for SYM_INT32 or matmulFloatCore for FLOAT32). (4) Convert the output back to the target dtype. (5) Write the result to the destination tensor.

This design separates the type logic from the computation logic: each kernel only handles one specific combination of input/output types, keeping individual kernels simple. ExecuteOp handles all the type conversion edge cases centrally.

```
/* ExecuteOp high-level pseudocode */
void executeOp(opType_t op,
tensor_t *dst,
tensor_t *src1,
tensor_t *src2,
opConfig_t *cfg) {
/* Step 1: Determine computation dtype */
dtype_t compDtype = resolveComputeDtype(op, dst-&gt;dtype,
src1-&gt;dtype, src2-&gt;dtype);
/* Step 2: Convert inputs if needed */
tensor_t *a = ensureDtype(src1, compDtype);          // May alloc+copy
tensor_t *b = ensureDtype(src2, compDtype);
/* Step 3: Allocate scratch output */
tensor_t *out = allocScratch(dst-&gt;shape, compDtype);
/* Step 4: Dispatch to kernel */
kernelFn_t fn = getKernel(op, compDtype);
fn(out, a, b, cfg);
/* Step 5: Convert result to dst dtype */
convertAndWrite(dst, out);
}
```

ensureDtype checks if src1->dtype == compDtype. If yes, returns src1 directly (no copy). If no, allocates a scratch tensor from DataStorage, calls quantiseTensor or dequantiseTensor to fill it, and returns the scratch tensor. The original src1 is unchanged. This "as-needed conversion" avoids unnecessary copies for the common case where all tensors are already in the correct dtype.

Type Promotion Rules

resolveComputeDtype implements ODT's type promotion rules. The rules, in priority order: (1) If either source is FLOAT32, compute in FLOAT32 (safest, no precision loss). (2) If both sources are SYM_INT32, compute

in SYM_INT32 (uses integer matmul kernel). (3) If either source is BOOL, the operation must be mask-aware (e.g., maskedMatmul). (4) Default: compute in FLOAT32 (safe fallback).

These rules mean that mixing dtypes (e.g., SYM_INT32 weights with FLOAT32 input) always promotes to FLOAT32. This is conservative but ensures correctness. A more aggressive implementation could promote SYM_INT32 + FLOAT32 to SYM_INT32 by quantising the float input -- but this risks quantisation errors accumulating in the intermediate computations.

Scratch Buffer Management in ExecuteOp

allocScratch allocates from DataStorage. The scratch tensors live for the duration of one ExecuteOp call. After the call returns, the scratch tensors are no longer referenced. They are not freed (DataStorage has no free()). Instead, ODT relies on a "high-water mark" pattern: at the start of each forward pass, record the DataStorage position with dataStorageUsed(). After the forward pass (before the backward pass starts), optionally reset to that position to free all activation tensors and scratch buffers.

This works because activations are not needed after the backward pass: the backward pass computes gradients from activations and then the activations are discarded. The only data that must persist between forward and backward passes are: (1) layer configs (saved input tensors in linearConfig_t->input), and (2) the loss value.

```
/* Forward-backward with DataStorage management */
size_t mark = dataStorageUsed();          // Record pool position
tensor_t *out = layerForward(layer, input);  // Forward (allocs scratch)
float loss = crossEntropyLoss(out, label);
tensor_t *dOut = crossEntropyGrad(out, label);
layerBackward(layer, dOut);               // Backward (allocs scratch)
/* Gradients are in layer-&gt;weights-&gt;grad tensor */
/* NOT in DataStorage scratch -- grad tensor is pre-allocated */
/* Now safe to reclaim all scratch: */
dataStorageResetTo(mark);                 // Free all scratch tensors
```

> ■ **WARNING** dataStorageResetTo(mark) invalidates ALL pointers returned after the mark was taken. Do NOT hold pointers to any scratch tensor (activation, intermediate result) across this reset. The only safe pointers after reset are to tensors whose buffers were allocated before the mark (weights, gradients, momentum).

Kernel Dispatch Table

getKernel(op, dtype) looks up the appropriate kernel function pointer. The dispatch table is a 2D array indexed by [opType][dtype]. opType values: OP_MATMUL, OP_ADD, OP_MUL, OP_REDUCE_SUM, OP_REDUCE_MAX, OP_CONV1D, etc. dtype values: FLOAT32, SYM_INT32, INT32.

```
/* Kernel dispatch table (simplified) */
static kernelFn_t kernelTable[NUM_OPS][NUM_DTYPES] = {
/* OP_MATMUL: */
[OP_MATMUL][FLOAT32] = matmulFloatCore,
[OP_MATMUL][SYM_INT32] = matmulInt32Core,
/* OP_ADD: */
```

```
[OP_ADD][FLOAT32] = addFloatCore,
[OP_ADD][INT32] = addInt32Core,
/* OP_REDUCE_SUM: */
[OP_REDUCE_SUM][FLOAT32] = reduceSumFloatCore,
/* ... more ops ... */
};
kernelFn_t getKernel(opType_t op, dtype_t dt) {
kernelFn_t fn = kernelTable[op][dt];
ODT_ASSERT(fn != NULL, "no kernel for (op,dt)");
return fn;
}
```

The NULL check in getKernel catches attempts to use an operation/dtype combination that has no kernel (e.g., OP_MATMUL with BOOL dtype). This is a development-time error that should never occur in deployed code -- adding the ODT_ASSERT catches it early during testing.

Appendix P: Backward Pass Mathematics and Implementation

Backpropagation Through a Linear Layer

Forward pass of Linear layer: y = W*x + b. Dimensions: W in R^{m x n}, x in R^n, b in R^m, y in R^m. Given dL/dy (gradient of loss w.r.t. output y, computed by the next layer in the chain rule), compute: dL/dx = W^T * dL/dy (gradient w.r.t. input), dL/dW = dL/dy * x^T (gradient w.r.t. weights), dL/db = dL/dy (gradient w.r.t. bias).

dL/dx derivation: L depends on x only through y = Wx + b. By chain rule: dL/dx_j = sum_i (dL/dy_i * dy_i/dx_j) = sum_i (dL/dy_i * W_{ij}) = (W^T * dL/dy)_j. In matrix form: dL/dx = W^T * dL/dy. Dimension check: W^T is n x m, dL/dy is m x 1, result is n x 1. Correct.

dL/dW derivation: dL/dW_{ij} = sum_k (dL/dy_k * dy_k/dW_{ij}) = dL/dy_i * x_j (only term k=i survives since y_k = sum_l W_{kl}*x_l and dy_k/dW_{ij} = x_j * delta_{ki}). In matrix form: dL/dW = dL/dy * x^T (outer product). Dimension: dL/dy is m x 1, x^T is 1 x n, result is m x n. Correct.

`static void linearBackwardFn(void *cfg_v, tensor_t *dOut) {`	
`linearConfig_t *cfg = cfg_v;`	
`tensor_t *W = cfg->weights->param;`	`// [outFeat x inFeat]`
`tensor_t *dW = cfg->weights->grad;`	`// Accumulate into this`
`tensor_t *db = cfg->useBias ? cfg->bias->grad : NULL;`	
`tensor_t *x = cfg->input;`	`// Saved from forward`
`tensor_t *mask = cfg->weightMask;`	
`size_t m = cfg->outFeatures;`	
`size_t n = cfg->inFeatures;`	
`/* dL/dW = dOut (m x 1) outer x (1 x n) */`	
`for (size_t i=0;i<m;i++) {`	
`float dy_i = tensorGetF32(dOut, i);`	
`for (size_t j=0;j<n;j++) {`	
`if (mask && !tensorBoolGet(mask,i*n+j)) continue;`	`// Skip inactive`
`float xj = tensorGetF32(x, j);`	
`float dw = tensorGetF32(dW, i*n+j);`	`// Accumulate`
`tensorSetF32(dW, i*n+j, dw + dy_i*xj);`	
`}`	
`}`	
`/* dL/db = dOut */`	
`if (db) for (size_t i=0;i<m;i++) {`	
`float d=tensorGetF32(db,i);`	
`tensorSetF32(db,i,d+tensorGetF32(dOut,i));`	
`}`	
`/* dL/dx = W^T * dOut (returned for previous layer) */`	
`/* (computed in layerBackward which stores dIn) */`	
`}`	

The gradient accumulation (dw + dy_i*xj) is important: layerBackward does NOT zero gradients before calling linearBackwardFn. Multiple forward-backward passes on different samples in a batch accumulate their gradients in dW. The training loop zeros gradients (fillTensor(dW, 0)) after each optimizer step, not before each backward pass. This allows gradient accumulation across a batch without explicit batching.

Gradient Flow Through RigL Sparse Mask

For RigL sparse layers: inactive weights (mask=0) do not participate in the forward pass computation. Their gradients should be non-zero (the gradient tells us how much the loss would decrease if the connection were activated). The backward pass computes dL/dW = dOut * x^T for all (i,j) pairs including inactive ones.

Wait -- the code above shows `continue` for inactive weights in the dL/dW computation. Is this correct? For the DROP-GROW algorithm: the GROW step needs |dL/dW_{ij}| for INACTIVE weights. If we skip inactive weights in the backward pass (not computing their gradient), we cannot compare inactive gradients to active gradients for the GROW decision.

The resolution: the `continue` in the dL/dW loop should NOT be applied to the gradient computation. Gradients for inactive weights should be computed. The mask should only be applied in the FORWARD pass (skip computing the output contribution from inactive weights) and the OPTIMIZER step (skip updating the value of inactive weights). The gradient dL/dW is always computed for all weights, active or not.

```
/* Correct backward: compute grad for ALL weights */
for (size_t i=0;i&lt;m;i++) {
float dy_i = tensorGetF32(dOut, i);
for (size_t j=0;j&lt;n;j++) {
/* NO mask check here -- compute grad for inactive too */
float xj = tensorGetF32(x, j);
float dw = tensorGetF32(dW, i*n+j);
tensorSetF32(dW, i*n+j, dw + dy_i*xj);
}
}
/* mask check only in SGD/AdamW update kernel */
```

■ **RigL: Gradient for Inactive Weights**	RigL critically depends on computing |dL/dW_{ij}| for INACTIVE weights in the GROW step. If linearBackwardFn skips inactive weights in the gradient computation, the GROW step cannot function correctly (all inactive gradients would be zero). Always compute gradients for all weights; apply the mask only in the optimizer update step.

Appendix Q: PPCA Implementation Deep Dive

PpcaModel Structure

The PPCA model for one speaker is stored in a ppcaModel_t struct. Fields: mu (p-dimensional mean vector), W (p x q factor loading matrix), sigma2 (isotropic noise variance), p (observation dimensionality = embedding dim), q (latent dimensionality = number of principal components).

```
typedef struct {
float *mu;                  // Mean: p floats
float *W;                   // Loadings: p*q floats (row-major)
float sigma2;               // Noise variance
size_t p;                   // Observation dim (e.g. 128)
size_t q;                   // Latent dim (e.g. 8)
} ppcaModel_t;
```

Fitting PPCA: Mean and Covariance

```
void ppcaFit(ppcaModel_t *model, float *data,
size_t N, size_t p, size_t q) {
model-&gt;p = p; model-&gt;q = q;
/* Step 1: Compute mean */
for (size_t j=0;j&lt;p;j++) model-&gt;mu[j]=0.0f;
for (size_t i=0;i&lt;N;i++)
for (size_t j=0;j&lt;p;j++)
model-&gt;mu[j] += data[i*p+j]/N;
/* Step 2: Compute covariance C = (1/N) sum (x-mu)(x-mu)^T */
float *C = dataStorageAlloc(p*p*sizeof(float));
for (size_t i=0;i&lt;N;i++) {
for (size_t j=0;j&lt;p;j++) {
for (size_t k=0;k&lt;p;k++) {
float cj = data[i*p+j]-model-&gt;mu[j];
float ck = data[i*p+k]-model-&gt;mu[k];
C[j*p+k] += cj*ck/N;
}
}
}
/* Step 3: Eigendecompose C via Jacobi */
float *eigvals = dataStorageAlloc(p*sizeof(float));
float *eigvecs = dataStorageAlloc(p*p*sizeof(float));
jacobiEig(C, eigvals, eigvecs, p);
/* Step 4: Sort eigenvalues descending */
sortEigPairs(eigvals, eigvecs, p);
/* Step 5: sigma2 = mean of remaining p-q eigenvalues */
```

```
float sumRem = 0.0f;
for (size_t i=q;i&lt;p;i++) sumRem += eigvals[i];
model-&gt;sigma2 = sumRem / (p-q);
/* Step 6: W = V[:, :q] * diag(sqrt(lambda[:q] - sigma2)) */
for (size_t j=0;j&lt;p;j++)
for (size_t k=0;k&lt;q;k++)
model-&gt;W[j*q+k] = eigvecs[j*p+k]
* sqrtf(eigvals[k] - model-&gt;sigma2);
}
```

Time complexity: Step 2 (covariance) is O(N * p^2). For p=128, N=20: 327,680 multiplications. Step 3 (Jacobi eigendecomposition) is O(p^3 * sweeps). For p=128, sweeps=25: 53 million multiplications. The Jacobi step dominates. At 216 MHz: 53e6 / 216e6 = 0.25 seconds. This is a one-time cost per speaker registration -- acceptable.

Generating Synthetic Embeddings

```
void ppcaGenerate(const ppcaModel_t *m, float *out) {
/* Sample z ~ N(0, I_q) */
float z[32];                                    // Stack: max q=32
for (size_t k=0;k&lt;m-&gt;q;k++)
z[k] = rngNormal(0.0f, 1.0f);
/* Compute x = W*z + mu */
for (size_t j=0;j&lt;m-&gt;p;j++) {
float val = m-&gt;mu[j];
for (size_t k=0;k&lt;m-&gt;q;k++)
val += m-&gt;W[j*m-&gt;q+k] * z[k];
val += rngNormal(0.0f, sqrtf(m-&gt;sigma2));   // Add isotropic noise
out[j] = val;
}
/* L2-normalise */
float norm2 = 0.0f;
for (size_t j=0;j&lt;m-&gt;p;j++) norm2 += out[j]*out[j];
float inv = 1.0f / sqrtf(norm2 + 1e-8f);
for (size_t j=0;j&lt;m-&gt;p;j++) out[j] *= inv;
}
```

The generated embedding is L2-normalised, matching the normalisation applied during enrollment. This ensures that synthetic replay samples are on the unit sphere and can be used directly as inputs to the classifier without any preprocessing.

For q=8 latent dims, p=128: ppcaGenerate costs 8 rngNormal calls for z (8 * ~20 cycles = 160 cycles), 128*8=1024 multiply-accumulates for W*z (1024 cycles), 128 rngNormal calls for noise (2560 cycles), one sqrtf (14 cycles), 128 multiplications for normalisation (128 cycles). Total: ~3886 cycles = 18 us per synthetic embedding.

Appendix R: Jacobi Eigendecomposition Algorithm

Algorithm Overview

The Jacobi eigendecomposition algorithm finds all eigenvalues and eigenvectors of a real symmetric matrix A. It iteratively applies Givens rotations to zero out off-diagonal elements. After convergence, the diagonal elements are the eigenvalues and the accumulated rotation matrix is the matrix of eigenvectors.

Convergence criterion: stop when the Frobenius norm of the off-diagonal elements is below a tolerance (e.g., 1e-10). For a p x p matrix: the off-diagonal Frobenius norm is sqrt(sum_{i!=j} A[i,j]^2). Each sweep (pass over all p*(p-1)/2 off-diagonal pairs) reduces this norm by a factor of approximately (1 - 2/(p*(p-1))). For p=16: convergence in ~10 sweeps. For p=128: ~25 sweeps.

```
void jacobiEig(float *A, float *vals, float *vecs, size_t p) {
/* Initialise vecs = identity */
for (size_t i=0;i&lt;p;i++)
for (size_t j=0;j&lt;p;j++)
vecs[i*p+j] = (i==j) ? 1.0f : 0.0f;
/* Iterative sweeps */
for (int sweep=0; sweep&lt;50; sweep++) {                // Max 50 sweeps
float offnorm = 0.0f;
for (size_t i=0;i&lt;p;i++)
for (size_t j=i+1;j&lt;p;j++)
offnorm += 2*A[i*p+j]*A[i*p+j];
if (offnorm &lt; 1e-20f) break;                          // Converged
/* Zero each off-diagonal pair (i,j) */
for (size_t i=0;i&lt;p;i++) {
for (size_t j=i+1;j&lt;p;j++) {
if (fabsf(A[i*p+j]) &lt; 1e-12f) continue;               // Skip tiny
/* Givens rotation angle */
float tau = (A[j*p+j]-A[i*p+i]) / (2*A[i*p+j]);
float t = (tau&gt;=0) ? 1.0f/(tau+sqrtf(1+tau*tau))
: 1.0f/(tau-sqrtf(1+tau*tau));
float c = 1.0f/sqrtf(1+t*t);                           // cosine
float s = t*c;                                         // sine
/* Apply rotation to A */
applyGivens(A, vecs, p, i, j, c, s);
}
}
}
for (size_t i=0;i&lt;p;i++) vals[i] = A[i*p+i];          // Extract eigenvalues
}
```

applyGivens updates rows/columns i and j of A and column i, j of the eigenvector matrix vecs. Each Givens rotation zeros the (i,j) and (j,i) off-diagonal elements while potentially creating non-zero values in other

positions -- hence the iterative approach.

```
void applyGivens(float *A, float *V, size_t p,
size_t i, size_t j, float c, float s) {
/* Update column i and j of A (symmetric) */
for (size_t k=0;k&amp;lt;p;k++) {
float Aki = A[k*p+i], Akj = A[k*p+j];
A[k*p+i] = c*Aki + s*Akj;
A[k*p+j] = -s*Aki + c*Akj;
A[i*p+k] = A[k*p+i];                    // Maintain symmetry
A[j*p+k] = A[k*p+j];
}
/* Zero the (i,j) element */
A[i*p+j] = 0.0f; A[j*p+i] = 0.0f;
/* Update eigenvectors */
for (size_t k=0;k&amp;lt;p;k++) {
float Vki = V[k*p+i], Vkj = V[k*p+j];
V[k*p+i] = c*Vki + s*Vkj;
V[k*p+j] = -s*Vki + c*Vkj;
}
}
```

Complexity per Givens rotation: 6p multiplications and 4p additions. Total per sweep: p*(p-1)/2 rotations * 6p ops = 3*p^2*(p-1) ops. For p=128: 3*128^2*127 = 6.2 million operations per sweep. For 25 sweeps: 155 million operations. At 216 MHz with FPU: approximately 0.72 seconds for ppcaFit on p=128. This is the bottleneck for speaker registration, but it occurs only once per new speaker.

■ **TIP**	Reduce q (latent dimension) to speed up PPCA fitting. For q=4 instead of q=8: W is p x 4 instead of p x 8 -- halves the GROW step memory for W and halves the generation cost. Accuracy impact: synthetic embeddings with q=4 are less diverse (lower variance), potentially reducing replay quality. Empirically, q=6-8 is a good range for p=128 embeddings.

Appendix S: Batch Norm, Pooling, and Misc Layers

Batch Normalisation vs Layer Normalisation

Batch normalisation (BN) normalises across the batch dimension: for each feature j, subtract the batch mean and divide by batch standard deviation. BN requires batch statistics: mu_j = (1/B) * sum_i x_{ij}, sigma_j^2 = (1/B) * sum_i (x_{ij} - mu_j)^2. Problem on embedded systems: batch size B is typically 1 (online learning, one sample at a time). BN with B=1 has undefined statistics (sigma = 0) -- the normalisation collapses.

Layer normalisation (LN) normalises across the feature dimension for each individual sample: mu = mean(x), sigma^2 = var(x), y = (x - mu) / sqrt(sigma^2 + eps). LN does not depend on batch statistics -- it works with B=1 and is therefore compatible with online/on-device training. ODT uses LN (via normConfig_t with type=LAYER_NORM) for all normalisation layers.

```
/* Layer norm forward */
float mu = 0.0f, var = 0.0f;
size_t n = tensorNumel(input);
for (size_t i=0;i&amp;lt;n;i++)
mu += tensorGetF32(input,i)/n;  // Compute mean
for (size_t i=0;i&amp;lt;n;i++) {
float d = tensorGetF32(input,i) - mu;
var += d*d/n;  // Compute variance
}
float inv_std = 1.0f/sqrtf(var + 1e-5f);
for (size_t i=0;i&amp;lt;n;i++) {
float norm = (tensorGetF32(input,i)-mu)*inv_std;
float y = gamma[i]*norm + beta[i];  // Scale and shift
tensorSetF32(output,i,y);
}
```

Gamma and beta are learnable parameters (one per feature). Their gradients: dL/d(gamma_i) = dL/dy_i * norm_i, dL/d(beta_i) = dL/dy_i. These are computed during the backward pass alongside the gradient of the input.

Max Pooling vs Average Pooling

Max pooling: out[i] = max(input[i*stride : i*stride + poolSize]). Backward: gradient flows only to the position of the maximum. This is a non-differentiable operation (the max is not smooth), but the subgradient is well-defined: dL/d(input[i*stride+k]) = dL/d(out[i]) if input[i*stride+k] was the maximum, 0 otherwise.

Average pooling: out[i] = mean(input[i*stride : i*stride + poolSize]). Backward: gradient is distributed equally to all positions: dL/d(input[i*stride+k]) = dL/d(out[i]) / poolSize. Smoother gradient than max pooling, often better for training.

Global max/average pooling (pool over the entire temporal dimension): reduces [channels, length] to [channels]. Used at the end of the encoder to produce a fixed-size embedding regardless of input length. Global average pooling is preferred over global max pooling because it: (1) produces smoother embeddings

less sensitive to single outlier frames, (2) has a uniform backward gradient, (3) is less likely to overfit to particular time positions.

Dropout (Training Regularisation)

Dropout randomly zeros elements of its input with probability p during training (inference: pass-through). This prevents co-adaptation of neurons and acts as regularisation. In ODT, dropout is applied to intermediate activations (e.g., after the first linear layer in the classifier).

```
void dropoutForward(tensor_t *out, const tensor_t *in,
float p, bool training) {
size_t N = tensorNumel(in);
float scale = training ? 1.0f/(1.0f-p) : 1.0f;  // Inverted dropout scale
for (size_t i=0;i&lt;N;i++) {
float v = tensorGetF32(in,i);
if (training && rngFloat() &lt; p) v = 0.0f;  // Drop with prob p
tensorSetF32(out,i,v*scale);  // Scale surviving units
}
}
```

Inverted dropout: multiply surviving values by 1/(1-p) during training. This keeps the expected value of the output constant (E[out] = v * (1-p) * 1/(1-p) = v). Without scaling, inference outputs would have different magnitude than training outputs (since all units are active at inference time).

> ■ **TIP** Save the dropout mask (which elements were zeroed) during the forward pass for use in the backward pass. The backward pass applies the same mask: gradient is zeroed for dropped elements and scaled by 1/(1-p) for surviving elements. Store the mask as a BOOL tensor in the dropout config struct.

Leaky ReLU and GELU

Leaky ReLU: f(x) = x if x > 0, alpha*x otherwise (typically alpha=0.01). Fixes the "dying ReLU" problem where neurons with consistently negative input have zero gradient and never recover. Backward: df/dx = 1 if x > 0, alpha otherwise.

GELU (Gaussian Error Linear Unit): f(x) = x * Phi(x) where Phi is the Gaussian CDF. Approximation: f(x) ≈ 0.5 * x * (1 + tanh(sqrt(2/pi) * (x + 0.044715 * x^3))). GELU is smooth everywhere (differentiable at x=0, unlike ReLU). Used in transformer models. On STM32: computing tanh requires approximately 20 cycles (software FPU) -- significantly slower than ReLU (1 cycle comparison + 1 cycle conditional move). Use ReLU or Leaky ReLU for embedded deployment unless GELU is specifically required.

```
float leakyReLU(float x, float alpha) {
return x &gt;= 0.0f ? x : alpha*x;  // 1 compare + 1 multiply
}
float geluApprox(float x) {
float c = sqrtf(2.0f/M_PI);
float t = tanhf(c*(x+0.044715f*x*x*x));  // ~20 cycles for tanhf
return 0.5f*x*(1.0f+t);
}
```

Appendix T: Build System, CMake, and Cross-Compilation

CMakeLists.txt Structure

The ODT project uses CMake as the build system. CMakeLists.txt is at the root of the repository. The build is configured for cross-compilation to ARM Cortex-M7 using the arm-none-eabi toolchain. A separate CMakeLists.txt in the tests/ directory configures the host (x86-64) build for unit testing.

```
cmake_minimum_required(VERSION 3.20)
project(ODT C)
# Cross-compilation toolchain
set(CMAKE_SYSTEM_NAME Generic)
set(CMAKE_SYSTEM_PROCESSOR arm)
set(CMAKE_C_COMPILER arm-none-eabi-gcc)
set(CMAKE_ASM_COMPILER arm-none-eabi-gcc)
# CPU and FPU flags
set(CPU "-mcpu=cortex-m7 -mthumb -mfpu=fpv5-d16
-mfloat-abi=hard")
set(CMAKE_C_FLAGS "${CPU} -O2 -Wall -Wextra -std=c99")
set(CMAKE_EXE_LINKER_FLAGS "${CPU} -T ${LINKER_SCRIPT}
-specs=nano.specs")
# ODT library sources
add_library(odt STATIC
src/Common.c src/Tensor.c src/TensorConversion.c
src/DataStorage.c src/Rng.c src/MinMax.c
src/Matmul.c src/Arithmetic.c src/Reduce.c
src/Layer.c src/Linear.c src/Conv1d.c
src/Sgd.c src/AdamW.c src/LrScheduler.c
src/DataLoader.c src/NPYLoader.c
src/PpcaReplay.c src/Serialize.c src/Deserialize.c
src/RigL.c                                  // New RigL module
)
target_include_directories(odt PUBLIC include/)
# Main firmware executable
add_executable(firmware src/main.c)
target_link_libraries(firmware odt m)       // Link math library
```

The -specs=nano.specs flag links against newlib-nano, a minimal C library for embedded systems. It provides printf (no floating-point by default -- add -u _printf_float to enable), malloc (wraps around _sbrk), and basic string functions. Total code size overhead of newlib-nano: approximately 20 KB.

-O2 optimisation is recommended for ODT training performance. -O3 can provide additional speedup (loop unrolling, vectorisation hints) but increases code size. -Os optimises for code size -- slower matmul but smaller Flash footprint. Profile with DWT before choosing.

Toolchain Installation

```
# Ubuntu/Debian: install ARM toolchain
sudo apt-get install gcc-arm-none-eabi
sudo apt-get install gdb-multiarch
# Or use the official ARM toolchain from developer.arm.com
# Verify installation:
arm-none-eabi-gcc --version
# Expected output: arm-none-eabi-gcc 12.x.x ... arm-none-eabi
# CMake build for STM32:
mkdir build_stm32 && cd build_stm32
cmake -DCMAKE_TOOLCHAIN_FILE=../cmake/stm32.cmake ..
make -j4
# Flash to STM32 using OpenOCD:
openocd -f interface/stlink.cfg -f target/stm32f7x.cfg \
-c "program firmware.elf verify reset exit"
```

Host Build for Unit Testing

```
# tests/CMakeLists.txt (host build)
cmake_minimum_required(VERSION 3.20)
project(ODT_Tests C)
# Use host GCC (no cross-compilation)
set(CMAKE_C_FLAGS "-O0 -g -Wall -Wextra -std=c99")
add_library(odt_host STATIC
../src/Tensor.c ../src/Rng.c ../src/MinMax.c
# ... all ODT sources ...
hal_stub.c                                  // Stub HAL functions
)
add_executable(test_runner
test_minmax.c test_rng.c test_tensor.c
test_rigl.c test_serialize.c main_test.c
)
target_link_libraries(test_runner odt_host m)
# Run tests:
# ./test_runner
# Expected: all assertions pass, exit code 0
```

hal_stub.c replaces the STM32 HAL functions with host implementations: HAL_GetTick() returns clock_gettime() in milliseconds; HAL_UART_Transmit() calls write(STDOUT_FILENO, buf, size); HAL_RNG_GenerateRandomNumber() reads from /dev/urandom. FatFS file I/O is replaced by standard POSIX fopen/fread/fwrite. The ODT library code is identical -- only the HAL layer changes.

Continuous Integration Setup

A typical CI pipeline for ODT using GitHub Actions: (1) Build for host with -fsanitize=address,undefined (catches memory errors and UB). (2) Run unit tests on the host. (3) Build for STM32 to verify cross-compilation success. (4) Optional: run in QEMU emulator for basic integration testing (QEMU supports Cortex-M4 but not M7 precisely -- use for basic logic, not performance).

```
# .github/workflows/ci.yml
name: ODT CI
on: [push, pull_request]
jobs:
test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3
- name: Install ARM toolchain
run: sudo apt-get install -y gcc-arm-none-eabi
- name: Build and test (host)
run: |
cmake -S tests/ -B build_test -DCMAKE_BUILD_TYPE=Debug
cmake --build build_test
./build_test/test_runner
- name: Build for STM32 (cross-compile check)
run: |
cmake -S . -B build_stm32
-DCMAKE_TOOLCHAIN_FILE=cmake/stm32.cmake
cmake --build build_stm32
```

Appendix U: RigL Integration Checklist

Complete RigL Integration Checklist

Use this checklist to verify a complete and correct RigL integration into the ODT library.

1. Data Structure Changes

[ ] parameter_t in Tensor.h has a tensor_t *mask field (defaults to NULL for dense parameters). [ ] linearConfig_t in Linear.h has a tensor_t *weightMask convenience field. [ ] sgdUpdateCtx_t in Sgd.h has a tensor_t *weightMask field. [ ] adamwUpdateCtx_t in AdamW.h has a tensor_t *weightMask field.

2. MinMax.c / MinMax.h

[ ] tensorAbsKthSmallestActive(weights, mask, K) implemented and declared. [ ] tensorAbsKthLargestInactive(grads, mask, K) implemented and declared. [ ] Both functions use DataStorage for scratch allocation (not stack). [ ] Unit test passes: known weight vector, known K, correct threshold returned.

3. RigL.c / RigL.h

[ ] rigLStep(model, numLayers, sparsity, step, totalSteps) implemented. [ ] Alpha computed as alpha_initial * cosine_decay. [ ] K = floor(alpha * numActive), skip layer if K == 0. [ ] DROP: K weights with smallest |w| deactivated (mask cleared, w zeroed). [ ] GROW: K inactive weights with largest |grad| activated (mask set, w set to 0). [ ] dropped == grown == K (count invariant). [ ] declared in RigL.h with correct signature. [ ] Added to CMakeLists.txt.

4. Linear.c Modifications

[ ] Forward pass: mask check in inner loop (tensorBoolGet per weight OR optimised byte scan). [ ] Backward weight gradient: NO mask check -- compute dW for all weights including inactive. [ ] Backward input gradient: same as before (W^T * dOut, ignoring mask since the forward contribution from inactive weights was zero anyway).

5. Sgd.c / AdamW.c Modifications

[ ] sgdStep checks weightMask; skips inactive weights; zeros velocity for inactive. [ ] adamwStep checks weightMask; skips inactive weights; zeros m and v for inactive. [ ] Unit test: after rigLStep deactivates a weight, subsequent sgdStep does not change that weight's value.

6. Serialize.c / Deserialize.c

[ ] serializeSparsity() writes ceil(N/8) bytes of packed mask bits. [ ] deserializeSparsity() reads mask bytes into pre-allocated mask tensor. [ ] Checkpoint round-trip test: save model, load model, verify mask is identical (bit-for-bit comparison).

7. Training Loop Integration

[ ] rigLStep called every rigLInterval steps. [ ] step counter incremented after every gradient step. [ ] totalSteps set correctly (NUM_EPOCHS * STEPS_PER_EPOCH). [ ] alpha decays from alpha_initial to 0 over totalSteps. [ ] After final rigLStep (step=totalSteps): K=0, no topology changes (alpha=0).

8. Verification Tests

[ ] Active count invariant: numActive unchanged after rigLStep. [ ] Zero check: all inactive weights are exactly 0.0f. [ ] Gradient is computed for inactive weights (backward pass does not skip them). [ ] Accuracy comparison with PyTorch RigL: within 5% after 100 topology updates. [ ] Checkpoint round-trip:

save-load-verify equal accuracy.

9. Performance Targets (STM32F746ZG at 216 MHz)

[ ] rigLStep for one 128x128 layer at 90% sparsity, alpha=0.1: under 50 ms. [ ] One forward pass (128->8 sparse linear): under 1 ms. [ ] One backward + SGD step: under 5 ms. [ ] One complete training epoch (1000 samples): under 60 seconds. [ ] PPCA fitting (N=20 samples, p=128, q=8): under 2 seconds.

10. Power Considerations

[ ] Training is paused when battery below threshold (requires ADC monitoring). [ ] Model checkpoint saved to SD card after each epoch (prevents data loss on power failure). [ ] STM32 enters STOP mode between training epochs if latency allows (reduces power from 150 mA to 2 mA during idle). [ ] Wake from STOP triggered by RTC alarm or external interrupt (e.g., new audio available).

Appendix V: Numerical Recipes for Embedded ML

Fast Inverse Square Root on Cortex-M7

The L2-normalisation in the speaker ID pipeline requires computing 1/sqrt(norm^2). The Cortex-M7 FPU has a VSQRT.F32 instruction (14 cycles) and no VRSQRT (reciprocal square root) instruction. The classic Quake fast inverse square root trick uses Newton-Raphson refinement starting from a magic constant initial guess.

```
/* Fast inverse square root (Lomont 2003 variant) */
float fastInvSqrt(float x) {
float xhalf = 0.5f * x;
union { float f; uint32_t i; } u;  // Type-pun float to int
u.f = x;
u.i = 0x5f375a86 - (u.i &gt;&gt; 1);  // Magic constant initial guess
/* One Newton-Raphson step: y = y*(1.5 - xhalf*y*y) */
u.f *= (1.5f - xhalf * u.f * u.f);  // ~0.175% error after 1 step
/* Two steps for higher precision */
u.f *= (1.5f - xhalf * u.f * u.f);  // ~0.00001% error
return u.f;
}
```

Performance: fastInvSqrt with 2 Newton-Raphson steps costs approximately 10 cycles on Cortex-M7 (type-pun + 2 Newton steps) vs 14 cycles for VSQRT.F32 + 1 cycle VDIV.F32 = 15 cycles. Marginal speedup, but the Newton-Raphson approach also works on Cortex-M0+ (no hardware sqrt) where it saves 50+ cycles compared to software sqrt.

> ■ **WARNING** The type-pun via union (float/uint32_t) is defined behaviour in C99 and C11 but was undefined behaviour in C89. ODT uses C99 (-std=c99), so the union type-pun is valid. Do NOT use pointer casting (float *ptr = &x; uint32_t i = *(uint32_t*)ptr) which is a strict aliasing violation and may be optimised incorrectly by GCC.

Numerically Stable Log-Sum-Exp

Log-sum-exp: LSE(x_1,...,x_n) = log(sum_i exp(x_i)). Used in cross-entropy loss and softmax. Direct computation overflows for large x_i (exp(89) = INF in float32). Stable form: LSE(x) = max(x) + log(sum_i exp(x_i - max(x))). The subtracted max ensures all arguments to exp are <= 0, preventing overflow.

```
float logSumExp(const float *x, size_t n) {
float mx = x[0];
for (size_t i=1;i&lt;n;i++) if (x[i]&gt;mx) mx=x[i];  // Find max
float sum = 0.0f;
for (size_t i=0;i&lt;n;i++) sum += expf(x[i]-mx);  // Shifted exp
return mx + logf(sum);  // Log-sum-exp
}
```

Kahan Summation for Accurate Float Accumulation

When accumulating many float32 values (e.g., computing the mean of 1000 gradient values), rounding errors can accumulate. The Kahan summation algorithm maintains a compensation term that corrects for the rounding error of each addition.

```
float kahanSum(const float *x, size_t n) {
float sum = 0.0f, comp = 0.0f;
for (size_t i=0;i&lt;n;i++) {
float y = x[i] - comp;    // Compensate for error
float t = sum + y;
comp = (t - sum) - y;    // Error of last addition
sum = t;
}
return sum;
}
```

Kahan summation reduces the accumulated error from O(n * eps) to O(eps) (independent of n). For n=1000 and eps=6e-8 (float32 machine epsilon): naive sum error ~6e-5; Kahan sum error ~6e-8. The extra 3 floating-point operations per iteration (subtraction, addition, subtraction) cost approximately 3 extra cycles, less than a 10% overhead.

Fixed-Point Sine and Cosine for Cosine Decay

The cosine decay schedule in rigLStep uses cosf() from libm. On Cortex-M7: cosf() costs approximately 30-50 cycles (software implementation). Since rigLStep is called infrequently (every 100 steps), this overhead is negligible -- 50 cycles / 100 * 32768 steps per matmul = 0.015% overhead. No approximation needed.

For applications where cosf is called in a tight inner loop (e.g., generating sinusoidal test signals for audio): use the CMSIS-DSP arm_cos_f32() function which uses a lookup table with linear interpolation. Performance: approximately 10 cycles, trading 0.002% accuracy for 5x speedup over libm cosf.

Integer Power of Two Operations

```
/* Powers of 2 via bit shift (faster than powf) */
/* 2^n for integer n: */
uint32_t pow2_uint(int n) { return 1u &lt;&lt; n; }    // 1 cycle on M7
/* 2^x for float x: use expf(x * logf(2)) */
/* But for n-bit quantisation: */
/* qMax = (1 &lt;&lt; (qMaxBits-1)) - 1 */
/* For qMaxBits=12: 1&lt;&lt;11 = 2048, qMax=2047 */
/* For qMaxBits=8: 1&lt;&lt;7 = 128, qMax=127 */
/* For qMaxBits=4: 1&lt;&lt;3 = 8, qMax=7 */
```

The (1 << (qMaxBits-1)) - 1 pattern computes the maximum positive value for a symmetric qMaxBits-bit signed integer. For 12-bit: 2047. For 8-bit: 127. For 16-bit: 32767. The corresponding minimum is -(qMax+1): for 12-bit: -2048. Symmetric quantisation uses [-qMax, qMax] (not the full [-2048, 2047]) to maintain symmetry (positive and negative ranges equal).

Appendix W: Dataset Preparation for On-Device Training

LibriSpeech Dataset Subset Preparation

LibriSpeech is a 1000-hour English speech corpus from audiobooks. For on-device training, we use a small subset: 8 speakers, 50 utterances each, 5-10 seconds per utterance. This gives 400 utterances, approximately 2500 seconds of audio. The dataset fits on a 2 GB microSD card.

Preprocessing pipeline (run on a host PC, results saved to SD card as .npy files): (1) Resample all audio to 16 kHz mono. (2) Extract 40-MFCC features with 25ms frames, 10ms shift, Hamming window. (3) Stack 30 consecutive frames into one feature vector (30 * 40 = 1200 dimensions). (4) L2-normalise each feature vector. (5) Split into train (80%) and test (20%). (6) Save as npy: X_train.npy (shape [320, 1200], dtype float32), y_train.npy (shape [320], dtype int32).

```
# Python preprocessing script (run on host)
import numpy as np, librosa, os
SPEAKERS = ["84", "174", "251", "422", ...]  // 8 speaker IDs
X, y = [], []
for spk_id, spk in enumerate(SPEAKERS):
files = glob.glob(f"LibriSpeech/{spk}/**/*.flac")
for fpath in files[:50]:  // 50 utterances per speaker
audio, sr = librosa.load(fpath, sr=16000)
mfcc = librosa.feature.mfcc(y=audio, sr=sr,
n_mfcc=40, n_fft=512,
hop_length=160)
# Stack 30 frames
for t in range(0, mfcc.shape[1]-30, 15):
feat = mfcc[:, t:t+30].flatten()  // 1200-dim feature
feat /= np.linalg.norm(feat) + 1e-8  // L2-normalise
X.append(feat); y.append(spk_id)
np.save("X_train.npy", np.array(X, dtype=np.float32))
np.save("y_train.npy", np.array(y, dtype=np.int32))
```

The resulting .npy files: X_train.npy is [320, 1200] float32 = 320*1200*4 = 1.54 MB. y_train.npy is [320] int32 = 1.25 KB. Both fit easily on the SD card and can be loaded into the DataStorage pool if available. If 1.54 MB exceeds SRAM, stream from SD card in 32-sample batches.

Data Augmentation for Speaker ID

Data augmentation increases effective training set size without collecting more real data. For speaker ID, relevant augmentations: (1) Additive noise: add scaled white noise (SNR 10-30 dB). Simulates real microphone noise. (2) Speed perturbation: resample audio by 0.9x or 1.1x (equivalent to speaking slower/faster). Use librosa.effects.time_stretch. (3) Room reverberation: convolve with a random room impulse response (RIR). Simulates different acoustic environments.

For on-device augmentation (applied during training, not pre-processing): additive noise is easy to implement in C using rngNormal: for each feature x_i, add sigma * rngNormal(0,1) where sigma is the noise level. This doubles or triples the effective training set with minimal storage overhead.

```
/* On-device additive noise augmentation */
void augmentNoise(tensor_t *feat, float snr_db) {
/* snr_db in [10, 30]: sigma in [0.03, 0.3] */
float sigma = powf(10.0f, -snr_db/20.0f);  // Convert SNR to sigma
size_t N = tensorNumel(feat);
for (size_t i=0;i&lt;N;i++) {
float v = tensorGetF32(feat,i);
v += sigma * rngNormal(0.0f, 1.0f);  // Add Gaussian noise
tensorSetF32(feat,i,v);
}
}
```

Evaluation Metrics for Speaker ID

Key metrics for evaluating speaker ID performance: (1) Accuracy: fraction of test utterances correctly identified. Simple but ignores the difficulty of each speaker pair. (2) Equal Error Rate (EER): the threshold at which false accept rate equals false reject rate. Lower EER is better. Typical target: EER below 5% for high-quality enrollment.

Confusion matrix analysis: for K=8 speakers, the confusion matrix is 8x8. C[i,j] = number of utterances from speaker i predicted as speaker j. The diagonal (C[i,i]) is correct predictions. Off-diagonal entries reveal which speakers are confused with each other (acoustically similar voices). Use the confusion matrix to identify hard pairs that may need more training samples or better augmentation.

```
/* Compute confusion matrix */
int C[K][K];  // K x K confusion matrix
memset(C, 0, sizeof(C));
for (size_t i=0;i&lt;N_test;i++) {
tensor_t *x = testX[i];
int true_label = testY[i];
tensor_t *logits = layerForward(classifier, x);
int pred = argmax(logits, K);
C[true_label][pred]++;
}
/* Print confusion matrix */
for (int i=0;i&lt;K;i++) {
for (int j=0;j&lt;K;j++)
printf("%4d", C[i][j]);
printf("\n");
}
```

Appendix X: Sparse Matrix Formats and Trade-offs

Overview of Sparse Formats

Several sparse matrix formats exist for representing matrices with many zero entries. ODT uses the BOOL mask format (dense matrix + bit-packed mask). Alternative formats have different trade-offs in memory, access pattern, and modification cost.

(1) Dense + mask (ODT approach): store all N values in a dense array (including zeros), plus a ceil(N/8)-byte bit mask. Memory: N*4 + N/8 bytes. Access: O(1) for weight[i] (direct index). Mask check: 4 cycles. Drop/grow: O(1) (flip one bit). Best for: dynamic sparsity (RigL) where the set of active weights changes frequently.

(2) CSR (Compressed Sparse Row): store only non-zero values, column indices, and row pointers. Memory for 10% density: 0.1*N*4 (values) + 0.1*N*4 (col indices) + m*4 (row pointers) = 0.8*N + 4*m bytes. Access: O(1) for row scan, O(nnz/m) per row. Drop/grow: O(nnz) (rebuild the arrays). Best for: static sparsity with rare topology changes.

(3) COO (Coordinate format): store (row, col, value) triples. Memory: 3 * nnz * 4 bytes. Access: O(nnz) for row scan. Drop/grow: O(1) for grow (append), O(nnz) for drop (scan and remove). Best for: incremental construction of sparse matrices.

```
/* Memory comparison for N=32768, 10% density (3277 active) */
Dense+mask: 32768*4 + 32768/8 = 131072 + 4096 = 135168 bytes
132 KB
CSR: 3277*(4+4) + 128*4 = 26216 + 512 = 26728 bytes 26 KB
COO: 3277*3*4 = 39324 bytes 38 KB
```

CSR saves 5x memory over Dense+mask for 10% density. However, CSR makes RigL topology updates expensive: growing a weight requires inserting a new (row, col, value) tuple while maintaining CSR structure -- O(nnz) operation in the worst case. For RigL's frequent topology updates (every 100 steps), Dense+mask is preferred despite the 5x memory overhead.

Why ODT Chose Dense + Mask

The design decision to use Dense+mask in ODT was driven by: (1) Simplicity of implementation: existing dense matmul kernels needed only a mask check added, not a complete rewrite. (2) RigL compatibility: the mask format allows O(1) DROP and GROW operations (just flip a bit, clear/set a weight). (3) Gradient computation: inactive weights need their gradients computed (for GROW decisions) -- dense storage makes this natural. (4) Memory access: sequential access to the weight array is cache-friendly, even with the mask check overhead.

Trade-off: for very high sparsity (>95%) or very large models (N > 10^6), CSR or COO become attractive. For the STM32 embedded target with N typically under 100,000 weights and sparsity 70-90%, Dense+mask is the right choice.

Quantised Sparse Representation for Inference

For inference-only deployment (after training is complete), the optimal representation is: store only the active weights in a quantised format, with column indices for the CSR structure. For 10% density and 8-bit quantisation: 3277 * (1 byte value + 2 byte column index) = 9831 bytes = 9.6 KB for the weight part. Compare to Dense+mask: 135 KB (float32) or 33.8 KB (int8+mask). CSR int8 saves 3.5x over Dense+mask int8 at 10% density.

Implementation note: for inference, the sparse format is fixed (no topology updates). The matmul can use a CSR-style loop: for each output element i, sum over the non-zero weights in row i using the stored column indices. This avoids the mask check overhead and directly accesses only active weights.

Appendix Y: Quantisation Types Reference

INT32 dtype

INT32: 32-bit signed integer, no scale, no zero-point. Used for: (1) accumulator type in integer matmul (SYM_INT32 * SYM_INT32 -> INT32 sum). (2) Raw mantissas of SYM/ASYM quantised tensors before dequantisation. (3) Label arrays (class indices). Not a quantisation type per se -- it is the native integer type for accumulation.

Range: [-2^31, 2^31-1] = [-2147483648, 2147483647]. For SYM_INT32 matmul accumulator: sum of 128 terms of 2047^2 = 537,395,072 < 2^31. Safe. For 256 terms: 1,074,790,144 < 2^31. Still safe. For 512 terms: 2,149,580,288 > 2^31 = overflow! Limit inner product length to 500 with 12-bit SYM_INT32 operands.

FLOAT32 dtype

FLOAT32: 32-bit IEEE 754 floating point. Mantissa: 23 explicit bits + 1 implicit = 24 bits (~7 decimal digits). Exponent: 8 bits, range ~10^-38 to 10^38. Used for: weight tensors during training, gradient tensors, momentum/variance tensors, activation tensors during inference. Default type for all non-quantised operations.

Special values in float32: NaN (Not a Number) -- result of 0/0, sqrt(-1), etc. Inf (positive/negative infinity) -- result of overflow. Denormals (subnormal numbers) -- very small values near zero with reduced precision; can cause performance degradation (flush-to-zero mode on Cortex-M7 FPU avoids this). Check: at startup, set FPU->FPCCR to enable flush-to-zero (FPDSCR_FZ_Msk) if denormal performance is a concern.

SYM_INT32 dtype

SYM_INT32: symmetric quantisation where mantissas are stored as int32 and there is no zero-point (zeroPoint=0). The real value: real = mantissa * scale. The scale is stored as a float32 in the tensor_t struct. The qMaxBits parameter determines the effective precision: for qMaxBits=12, mantissas are in [-2047, 2047]. The upper 20 bits of the int32 are used as sign extension.

SYM_INT32 vs SYM: SYM stores mantissas compactly (e.g., 1 byte for 8-bit SYM). SYM_INT32 always stores in int32 (4 bytes). SYM is more memory-efficient for storage; SYM_INT32 is more compute-efficient since no packing/unpacking is needed for 32-bit arithmetic. ODT uses SYM_INT32 for the matmul computation type and SYM for checkpoint storage.

SYM dtype

SYM: symmetric quantisation with compact mantissa storage. For qMaxBits=8: mantissa stored as int8 (1 byte). For qMaxBits=4: mantissa stored as int8 (4 bits used, 4 bits wasted -- or packed 2 per byte with byteConversionAppend). For qMaxBits=16: mantissa stored as int16 (2 bytes). Formula: scale = absMax / qMax, mantissa = round(real / scale).

Use cases: (1) Compressed model checkpoint (save to SD card). (2) Quantised inference (int8 matmul). (3) Gradient quantisation for bandwidth-limited applications (not relevant for single-device ODT but relevant for federated learning). Default for new ODT quantisation: SYM with qMaxBits=8 gives good accuracy/memory trade-off.

ASYM dtype

ASYM: asymmetric quantisation with both scale and zero_point. real = (mantissa - zeroPoint) * scale. The zero_point allows the quantised range to be offset from zero. Use case: ReLU activations are always >= 0 (range [0, maxVal]). Symmetric quantisation would waste half the range on negative values. ASYM can represent [0, maxVal] using all qMax bits: zeroPoint = 0 (representing real=0), mantissa max = 2*qMax (representing real = maxVal).

ASYM is more complex: every dequantisation requires subtracting zeroPoint before multiplying by scale. Every quantisation requires computing (real/scale + zeroPoint). These extra operations have minimal cycle cost but add code complexity. For ODT's primary use case (speaker ID with ReLU activations in a simple classifier), ASYM provides marginal accuracy benefit -- SYM is simpler and sufficient.

BOOL dtype

BOOL: 1 bit per element, packed into bytes (8 elements per byte), LSB-first ordering. Only used for RigL weight masks. tensorNumel returns the number of mask bits. tensorBytes returns ceil(numBits/8) bytes. Operations: tensorBoolGet(i) and tensorBoolSet(i, value). No scale or zeroPoint -- the value is 0 (inactive) or 1 (active).

A BOOL tensor for N=32768 weights uses exactly 32768/8 = 4096 bytes = 4 KB. Initialise with memset(buf, 0xFF, 4096) to set all bits to 1 (all weights active). memset(buf, 0x00, 4096) sets all bits to 0 (all weights inactive -- never do this for training). A specific sparsity of 90% (10% active): initialise with a random mask setting exactly 3277 bits to 1 (or start fully dense and let rigLStep prune to the target sparsity).

Appendix Z: Error Analysis and Accuracy Recovery Techniques

Sources of Error in On-Device FQT Training

Five primary sources of error accumulate during FQT training on STM32: (1) Weight quantisation error: when storing float32 weights as SYM_INT32 mantissas, each weight incurs an error of at most scale/2. For 12-bit SYM_INT32 with scale=4.88e-4: max error per weight = 2.44e-4. (2) Gradient quantisation error: if gradients are quantised (SYM with 8-bit), small gradients are rounded to zero. Stochastic rounding mitigates but does not eliminate this. (3) Accumulation rounding: float32 accumulation of N products has rounding error proportional to sqrt(N) * eps. For N=128: rounding error ≈ 11 * 6e-8 = 6.6e-7 per output element. Negligible. (4) Learning rate sensitivity: the effective learning rate is lr / scale (in mantissa space). If scale changes between steps (due to weight magnitude changes), the effective LR changes unexpectedly. (5) Dead weight syndrome: weights stuck at zero mantissa (quantised away) that never receive gradient updates.

Mitigation strategies: (1) Use SR_HALF_AWAY for all quantisation operations in the training path. (2) Use a fixed scale (not re-computed from absMax each step) for gradient tensors to avoid LR instability. (3) Monitor active weight count: if the number of non-zero weights decreases unexpectedly (outside of rigLStep calls), dead weight syndrome is occurring -- reduce the quantisation precision or increase LR.

Accuracy Recovery After Catastrophic Forgetting

If accuracy on old speakers drops significantly after registering a new speaker, try: (1) Increase replay ratio: generate more synthetic samples per old speaker per batch. (2) Reduce learning rate for fine-tuning new speaker: use lr=0.001 instead of lr=0.01. (3) Increase PPCA quality: collect more enrollment samples (30+ utterances) and increase q (latent dim) to 16. (4) Use EWC (Elastic Weight Consolidation) as a supplementary regulariser: add a penalty proportional to the change in important weights.

```
/* EWC penalty (simplified) */
/* fisher[i] = importance of weight i for old tasks */
float ewcLoss = 0.0f;
float ewcLambda = 1000.0f;
for (size_t i=0;i&lt;N;i++) {
float dw = tensorGetF32(w,i) - old_w[i];  // Change from old weights
ewcLoss += fisher[i] * dw * dw;  // Weighted L2 penalty
}
/* Add ewcLambda * ewcLoss to cross-entropy loss */
/* Backward pass propagates EWC gradient */
```

Fisher information matrix diagonal estimate: for each training sample from old tasks, compute (dL/dw_i)^2 averaged over all samples. High fisher[i] means weight i was important for predicting old task labels and should not change much. The EWC penalty discourages changes to high-fisher weights while still allowing changes to low-fisher weights (which can be repurposed for the new speaker).

Handling NaN and Inf During Training

NaN propagates: if one weight becomes NaN (from e.g. 0/0 in layer norm with zero variance), ALL subsequent weights in the same forward pass become NaN (since NaN + anything = NaN). The entire model becomes corrupt in a single step. Prevention: (1) Add epsilon to all divisors (scale + 1e-8, variance + 1e-5). (2) Gradient clipping: if global grad norm > threshold, scale all gradients down. (3) Gradient NaN detection: after backward pass, scan all gradients for NaN/Inf and abort training with a useful error message.

```
/* Gradient clipping */
float gradNorm = 0.0f;
for (size_t i=0;i&lt;N;i++) {
float g = tensorGetF32(grad,i);
gradNorm += g*g;
}
gradNorm = sqrtf(gradNorm);
if (gradNorm &gt; MAX_GRAD_NORM) {
float scale = MAX_GRAD_NORM / gradNorm;
for (size_t i=0;i&lt;N;i++) {
float g = tensorGetF32(grad,i);
tensorSetF32(grad,i,g*scale);
}
}
```

```
// e.g. MAX_GRAD_NORM=1.0
```

Appendix AA: Power Analysis for On-Device Training

STM32F746ZG Power Consumption

Power consumption on STM32F746ZG (all domains at 216 MHz, all peripherals active): Run mode: approximately 100-150 mA at 3.3V = 330-500 mW. With FPU active (matmul training): approximately 120-160 mA. Idle/WFI mode: approximately 20-30 mA (clock gating, peripherals off). STOP mode: approximately 1-2 mA (only low-power oscillator and SRAM retention). STANDBY mode: approximately 50 uA (no SRAM retention).

Energy per training epoch: if one epoch (1000 samples, one pass) takes 60 seconds at 150 mA: 150 mA * 3.3V * 60s = 29.7 J. For a 2000 mAh battery at 3.3V: 2000 mAh * 3.3V * 3600 s/h = 23,760 J total energy. Epochs per battery charge: 23760 / 29.7 ≈ 800 epochs. At 20 epochs per speaker registration: 40 speaker registrations per battery charge.

Battery Life	For a typical coin cell (CR2032, 225 mAh at 3V): total energy = 225 * 3 * 3600 = 2.43 kJ. One training epoch costs 29.7 J. Epochs per charge: 2430/29.7 = 81. Not practical for coin cell -- use a LiPo pack (1000-2000 mAh) for training-capable embedded systems.

Power Optimisation Strategies

(1) Reduce CPU frequency for non-critical phases: drop from 216 MHz to 48 MHz during SD card loading (IO-bound, not compute-bound). Power scales approximately linearly with frequency (for dynamic power): 48/216 = 22% of full-speed power. Loading a 500 KB dataset at 48 MHz takes 4.5x longer but uses 4.5x less energy (same total energy -- no gain in energy, only in peak power).

(2) Enable sleep between training steps: after each SGD step, insert WFI (Wait For Interrupt). The SYSTICK interrupt wakes the CPU for the next step. This reduces run-mode to sleep-mode when the CPU is waiting for SD card IO. Power reduction: 20-30% in IO-bound training phases.

(3) Batch training during charging: detect USB charger or external power supply via ADC/GPIO. Only run training while charging (device plugged in) to avoid battery drain. This is the most practical power strategy for devices that are regularly charged (e.g., overnight).

(4) Reduce matmul precision: switch from float32 to SYM_INT32 (12-bit) for training. Integer operations consume less FPU energy than float operations. On Cortex-M7: the FPU and integer pipeline have similar energy per operation, so the savings are modest. The main benefit of SYM_INT32 is reduced memory bandwidth (if using compact mantissa storage), which reduces SRAM access energy.

```
/* Enter STOP mode between epochs */
__HAL_RCC_PWR_CLK_ENABLE();                    // Enable Power controller
HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON,
PWR_STOPENTRY_WFI);                            // Wait for interrupt
/* After waking: restore clocks (PLL may be off) */
SystemClock_Config();                          // Re-initialise PLL
```

STOP mode wakeup sources: RTC alarm (for scheduled training), UART idle detection (wake on new voice command), GPIO edge (wake on button press for enrollment). After waking from STOP, re-call SystemClock_Config() to restore the PLL and clocks to their training-phase frequencies.

Energy Efficiency Comparison: Dense vs Sparse

Energy consumption is proportional to operations (FMAs) plus memory accesses. For dense matmul: N FMAs + N load-stores. For sparse matmul at 90% sparsity (optimised byte scan): 0.1*N FMAs + N/8 mask byte loads + 0.1*N load-stores (for active weights). Energy ratio: (0.1*N FMAs + N/8 mask + 0.1*N LS) / (N FMAs + N LS). With FMA energy = load-store energy and mask energy = 0.5 load-store: ratio ≈ (0.1+0.0625+0.1) / (1+1) ≈ 0.13. Sparse uses 13% of dense energy for the matmul. 7.7x energy reduction at 90% sparsity with optimised mask scan.

This energy reduction means: if one dense training epoch consumes 29.7 J, one sparse epoch (90% sparsity) consumes approximately 3.9 J (for the matmul-dominated phases). Other phases (loading data, SGD update, rigLStep) are unaffected. If matmul is 70% of epoch time: sparse epoch energy = 29.7 * (0.3 + 0.7*0.13) = 29.7 * 0.391 = 11.6 J. Saving: 18.1 J per epoch, 60% reduction. Epochs per battery: 800 * (29.7/11.6) = 2050 epochs per charge.

Appendix AB: Federated Learning Context for ODT

Federated Learning Overview

Federated Learning (FL) is a distributed machine learning paradigm where multiple devices train on their local data and share only model updates (gradients or weights), not the raw data. ODT on STM32 can participate in a federated learning system as an edge device. The server aggregates updates from many devices using FedAvg (federated averaging).

FedAvg algorithm: (1) Server broadcasts the global model to all K devices. (2) Each device trains on its local data for E epochs. (3) Each device sends updated weights to the server. (4) Server computes the weighted average of the device weights: w_global = sum_k (n_k / n_total) * w_k, where n_k is the number of samples on device k. (5) Repeat for R communication rounds.

ODT's role in FL: ODT handles the local training (steps 2 above). The communication step (sending/receiving weights) would require a networking layer (WiFi, BLE, or cellular) connected to the STM32. The STM32F746ZG has no built-in WiFi -- an external module (e.g., ESP8266, BG77 LTE module) would be needed, connected via UART or SPI.

Differential Privacy in On-Device Training

Differential privacy (DP) adds calibrated noise to gradients before sharing, providing a formal privacy guarantee. ODT's XorShift32 PRNG can generate the Gaussian noise needed for DP-SGD: instead of g_final = g_clipped, use g_final = g_clipped + N(0, sigma^2). The sigma is chosen based on the privacy budget (epsilon, delta).

```
/* DP-SGD noise injection */
float sensitivity = MAX_GRAD_NORM;                    // From gradient clipping
float sigma = sensitivity * sqrt(2 * log(1.25/delta)) / epsilon;
for (size_t i=0;i&lt;N;i++) {
float g = tensorGetF32(grad,i);
g += sigma * rngNormal(0.0f, 1.0f);                   // DP noise
tensorSetF32(grad,i,g);
}
```

Practical DP parameters for speaker ID: epsilon=1 (strict privacy), delta=1e-5 (standard choice for neural networks), sensitivity=1.0 (after gradient clipping to norm 1.0): sigma = 1.0 * sqrt(2 * ln(125000)) ≈ 6.8. This large noise essentially prevents learning useful gradients. More practical: epsilon=10 (relaxed privacy), giving sigma ≈ 0.68. The trade-off between privacy and utility is a key challenge in private federated learning.

Model Compression for Transmission

For federated learning, only the changes to weights need to be transmitted. With RigL 90% sparsity: (1) Only 10% of weights are active and potentially changed. (2) Transmit only the changed active weights (delta encoding). (3) Compress using the BOOL mask to identify which weights changed. Transmission size: 3277 * 4 bytes (weight values) + 4096 bytes (mask) = 17.2 KB vs 131 KB for full dense model. 7.6x compression

from sparsity alone.

Additional compression: quantise the weight delta to INT8 before transmission. If weight changes are typically small (< 0.01 per step), quantise with scale=0.0001: 8-bit mantissa covers range +/- 0.0127. Transmission size: 3277 bytes (INT8 deltas) + 4096 bytes (mask) = 7.4 KB. 17.7x compression vs uncompressed dense float32. This makes STM32-based federated learning practical even over narrow-band radio links (LoRaWAN, SIGFOX).

Appendix AC: Historical Context and Related Work

The Journey to On-Device Training

Neural network training has historically required significant compute resources: early neural networks (Rosenblatt 1958 Perceptron, Rumelhart 1986 backpropagation) ran on room-sized computers. The deep learning era (LeCun 1989 convolutional networks, Hinton 2006 deep belief networks) accelerated with GPU training (Raina 2009). By 2012, ResNet and AlexNet required millions of GPU cycles.

The push toward the edge: (1) Privacy concerns: user data sent to cloud servers for training raises privacy issues. (2) Latency: cloud round-trip latency (50-200ms) is unacceptable for real-time applications. (3) Connectivity: devices in remote locations have unreliable or expensive connectivity. (4) Personalisation: cloud models are generic; on-device training enables personalisation to individual users.

Key milestones in on-device training: (1) Google's Federated Learning (McMahan 2017): the first large-scale deployment of on-device training, on Android phones with 4+ GB RAM. (2) TinyML movement (2019-present): training and inference on microcontrollers with kilobytes of RAM. (3) Deutel et al. FQT (the ODT library's precursor): fixed-point quantisation training on STM32, 320 KB SRAM, 2020. (4) Evci et al. RigL (2020): gradient-guided dynamic sparse training, enabling further model compression. (5) ODT: combines FQT + RigL for continual on-device speaker ID training.

RigL in Context: Other Dynamic Sparse Methods

RigL is one of several dynamic sparse training algorithms. Key predecessors and contemporaries: (1) Lottery Ticket Hypothesis (Frankle 2019): dense networks contain sparse subnetworks ("winning tickets") that can be trained from scratch with 10-100x fewer parameters. Finding the ticket requires training the full dense network. (2) SNIP (Lee 2019): one-shot pruning at initialisation based on connection sensitivity. No retraining needed, but accuracy gap is larger than iterative methods. (3) GraNet (Liu 2021): combines RigL with gradual pruning, starting dense and ramping up to target sparsity during training. (4) SET (Mocanu 2018): Sparse Evolutionary Training -- grow connections randomly (not gradient-guided). Simpler than RigL, 3-5% accuracy gap. (5) AC/DC (Peste 2021): alternates between dense and sparse training phases.

Why RigL is best for ODT: RigL provides the best accuracy for a given sparsity level among the above methods, because gradient-guided growth is theoretically grounded (greedy coordinate descent approximation) and empirically superior to random growth. The implementation complexity is moderate (K-th order statistics + mask manipulation) and was implemented in approximately 200 lines of C in ODT.

FQT in Context: Quantisation Aware Training Methods

Quantisation-Aware Training (QAT) methods train neural networks while simulating the effects of quantisation. Key methods: (1) Straight-Through Estimator (Bengio 2013): the canonical trick for backpropagating through quantisation. Used in ODT's quantiseTensor backward path. (2) DoReFa-Net (Zhou 2016): quantises weights, activations, and gradients to arbitrary bit widths. (3) XNOR-Net (Rastegari 2016): 1-bit binary networks. (4) Deutel FQT: the specific QAT framework implemented in ODT, targeting STM32 with SYM_INT32 (12-bit).

The 12-bit operand choice in SYM_INT32 (qMaxBits=12) is specific to ODT's target (STM32 32-bit integer accumulator). Most published QAT works target GPUs with INT8 SIMD units (e.g., NVIDIA TensorRT INT8).

ODT's 12-bit choice provides higher precision than INT8 while still fitting within INT32 accumulation without overflow -- a constraint unique to the embedded setting with no INT64 hardware acceleration.

The LibriSpeech Speaker ID Task

LibriSpeech (Panayotov 2015) is a 1000-hour English audiobook corpus. The full LibriSpeech dataset contains 2484 speakers. For the ODT speaker ID task, a 8-speaker subset is used (small enough to fit on an SD card, large enough to be a meaningful classification problem). The speaker ID task on LibriSpeech is a well-studied problem: state-of-the-art systems (x-vectors, ECAPA-TDNN) achieve EER below 1% on the full 2484-speaker set. The ODT system, with its resource constraints, targets EER below 10% on the 8-speaker subset -- a much easier task but representative of the on-device personalization use case.